

第22章: CI/CD & デプロイ

この章の対象読者: デプロイやCI/CDが初めての方。コードは書けるが、本番公開の経験がない方。 **前提知識:** Git の基本操作 (commit, push, pull request)、第1章～第21章の内容 **所要時間:** 約4～5時間 (実践含む)

22.0 概要

この章では、BON-LOGアプリケーションを本番環境にデプロイする方法を学びます。Vercel、Supabase、GitHub Actionsを使ったCI/CDパイプラインの構築から、本番運用のベストプラクティスまでを解説します。

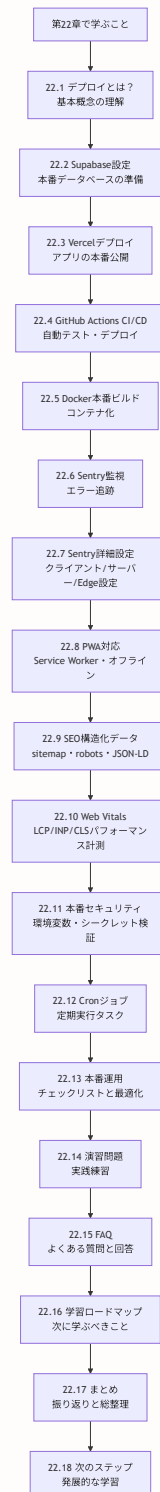
22.0 実習手順の進め方と手順マップ

手順に沿って進めると、**どのファイルに何を入力し、何を確認すればよいか** が分かります。形式の説明は [チュートリアルの進め方](#) を参照してください。

手順	主な対象ファイル (例)	完了時に確認すること
デプロイの概念	—	開発/本番・CI/CD の違いが分かる
Supabase	本番 DB 設定	本番用 DB に接続できる
Vercel	プロジェクト連携	プッシュで本番にデプロイされる
GitHub Actions	<code>.github/workflows/*.yml</code>	プッシュ時に lint・test・build が通る
Docker・Sentry・PWA・SEO 等	各セクション参照	コンテナ・監視・PWA・sitemap が動く

各セクションで **対象ファイル・入力するコード（サンプルコード）・実行方法・実行するとうなる・このあと変わること・確認方法**を確認しながら進めてください。

この章の全体像



22.1 デプロイとは

▶ このセクションで学ぶこと（クリックして展開）

デプロイとは？ `npm run dev` で動かしているのは「自分のPC上の開発サーバー」であり、他の人はアクセスできません。デプロイとは、コードをインターネット上のサーバーに配置し、URLで誰でもアクセスできるようにすることです。

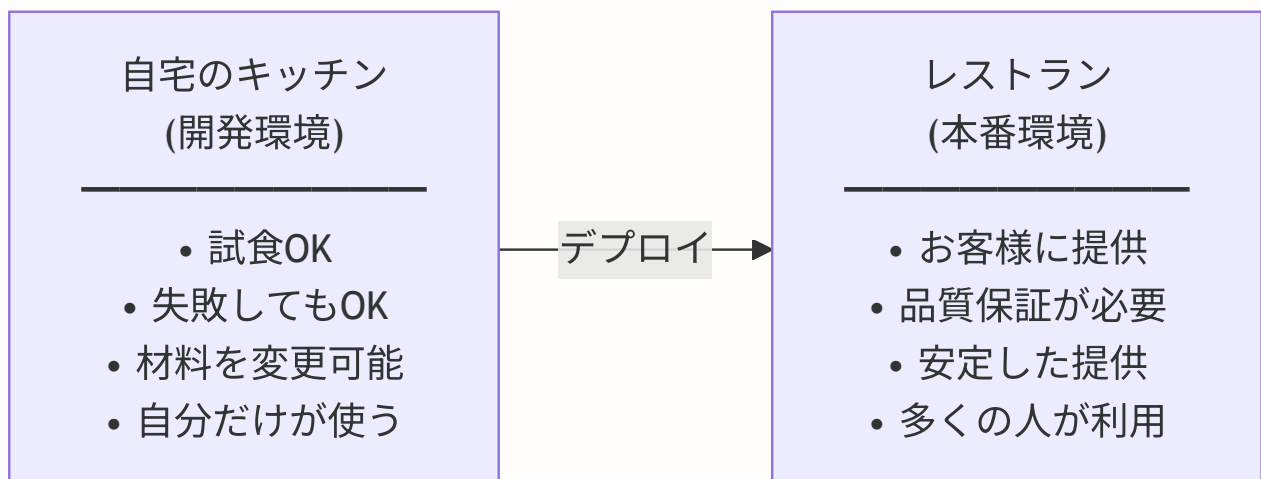
開発: localhost:3000 (自分のPCのみ)
↓ デプロイ
本番: https://www.bon-log.com (世界中からアクセス可能)

22.1.1 デプロイの基本概念

デプロイ（Deploy）とは、開発したアプリケーションを本番環境（プロダクション）に公開するプロセスです。

日常生活のたとえで理解しよう

デプロイは、料理に例えると「キッチンで作った料理をお客さんのテーブルに運ぶこと」に似ています。

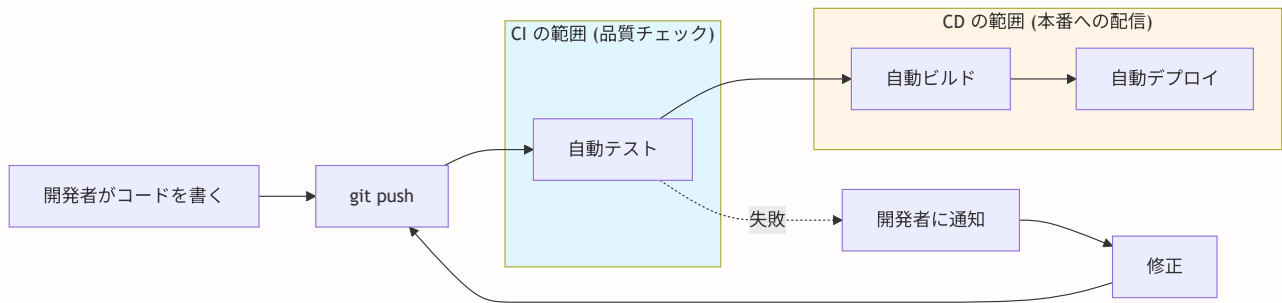


- **開発環境（キッチン）**：自分のPC上で動かすアプリ。エラーが出ても問題ない
- **本番環境（レストラン）**：世界中のユーザーがアクセスするサーバー上のアプリ。安定稼働が必須

22.1.2 CI/CDとは

CI (Continuous Integration: 継続的インテグレーション) は、コードの変更を頻繁に統合し、自動テストで品質を確認する仕組みです。

CD (Continuous Delivery / Deployment: 継続的デリバリー / デプロイメント) は、テスト済みのコードを自動的に本番環境にデプロイする仕組みです。

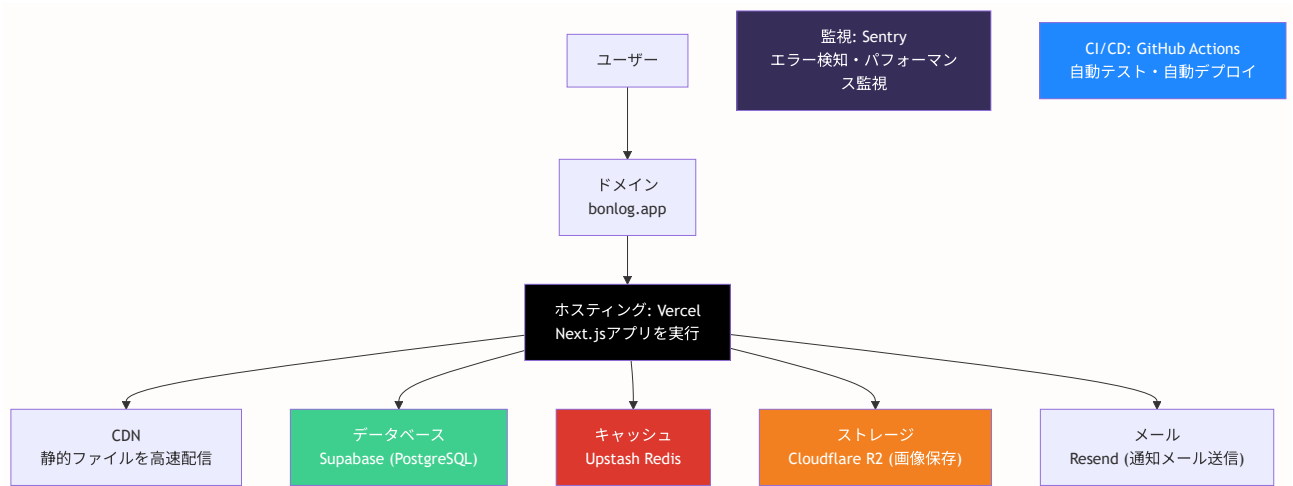


なぜCI/CDが必要なのか？

手動デプロイの問題点	CI/CDで解決
テストし忘れてバグが本番に	自動テストで必ずチェック
人によって手順が違う	毎回同じ手順で自動実行
デプロイに時間がかかる	数分で自動完了
深夜のデプロイでミスしやすい	人間の介入なしで正確に実行
誰がいつデプロイしたか不明	すべてログが残る

22.1.3 デプロイの主な構成要素

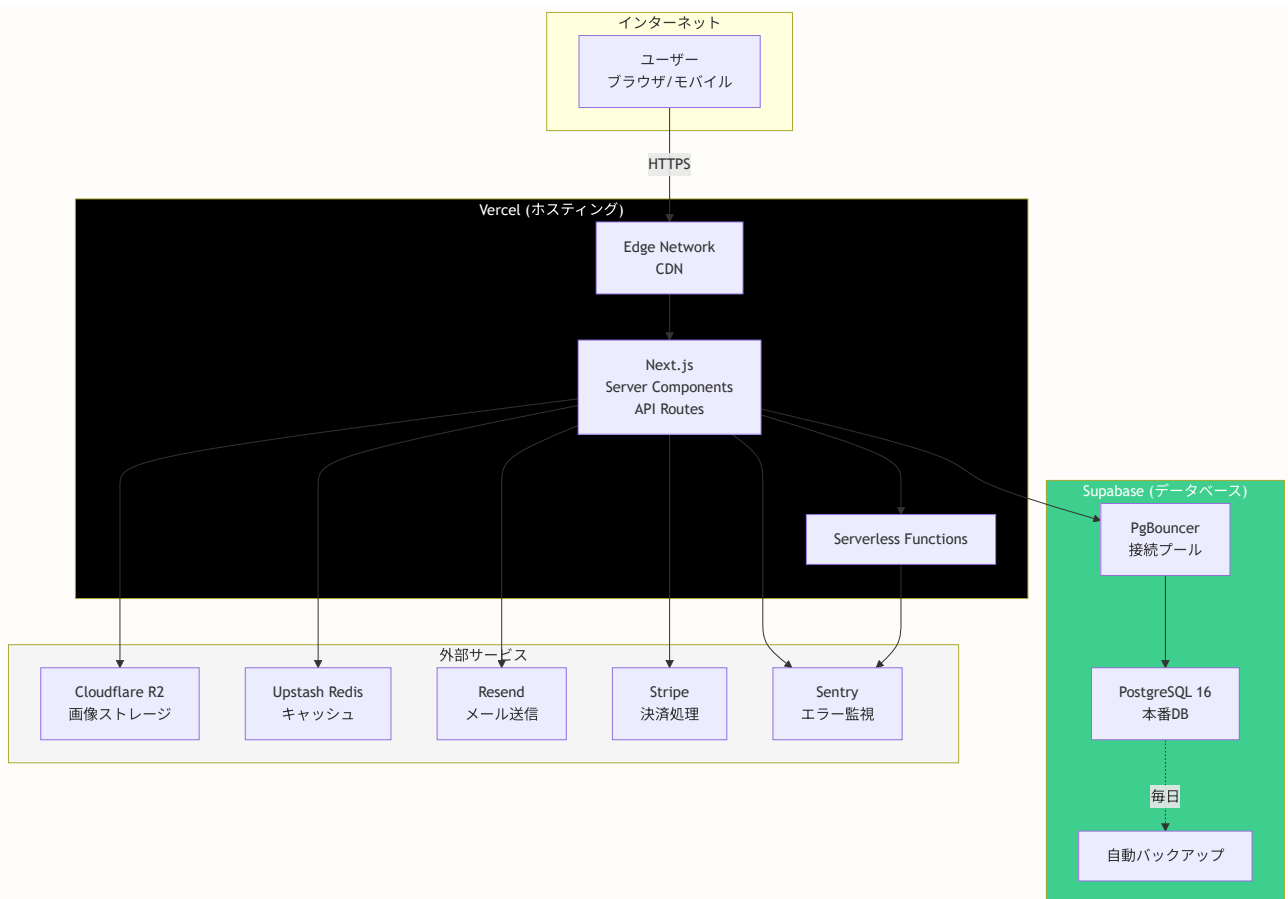
本番環境には、以下の要素が必要です。



構成要素	サービス	役割
ホスティング	Vercel	アプリケーションを実行するサーバー
データベース	Supabase	本番用のPostgreSQLデータベース
CI/CD	GitHub Actions	自動テスト・デプロイのパイプライン
モニタリング	Sentry	エラー追跡とパフォーマンス監視
ドメイン	任意のレジストラ	カスタムドメインの設定
キャッシュ	Upstash Redis	データのキャッシュで高速化
ストレージ	Cloudflare R2	画像・動画ファイルの保存
メール	Resend	ユーザーへの通知メール

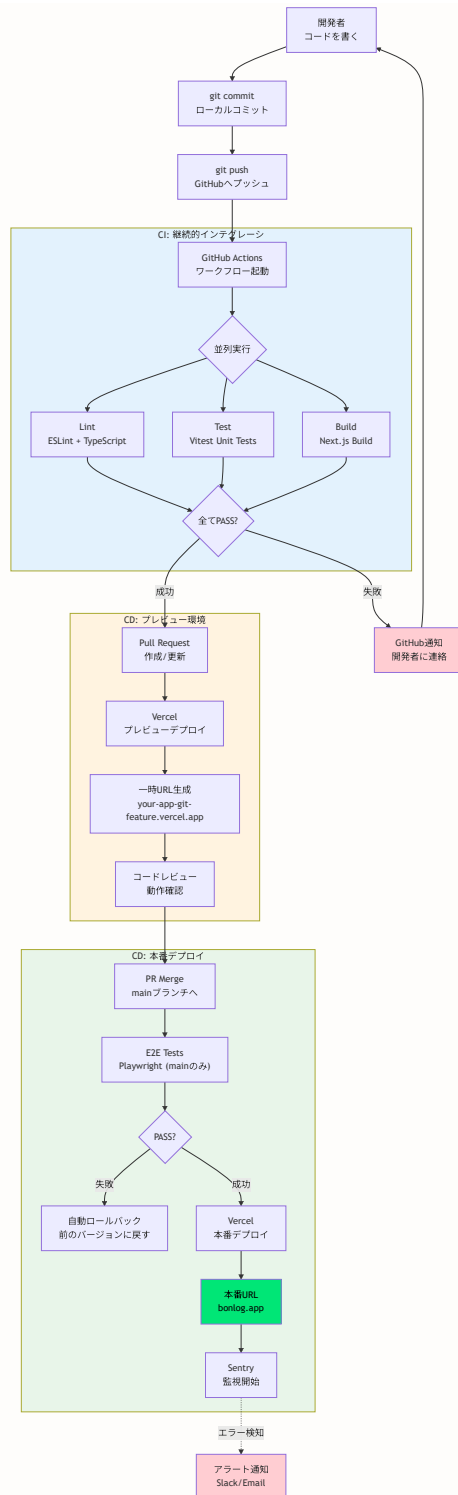
22.1.4 本番環境の詳細アーキテクチャ

以下の図は、BON-LOGアプリケーションの本番環境における完全なアーキテクチャを示しています。



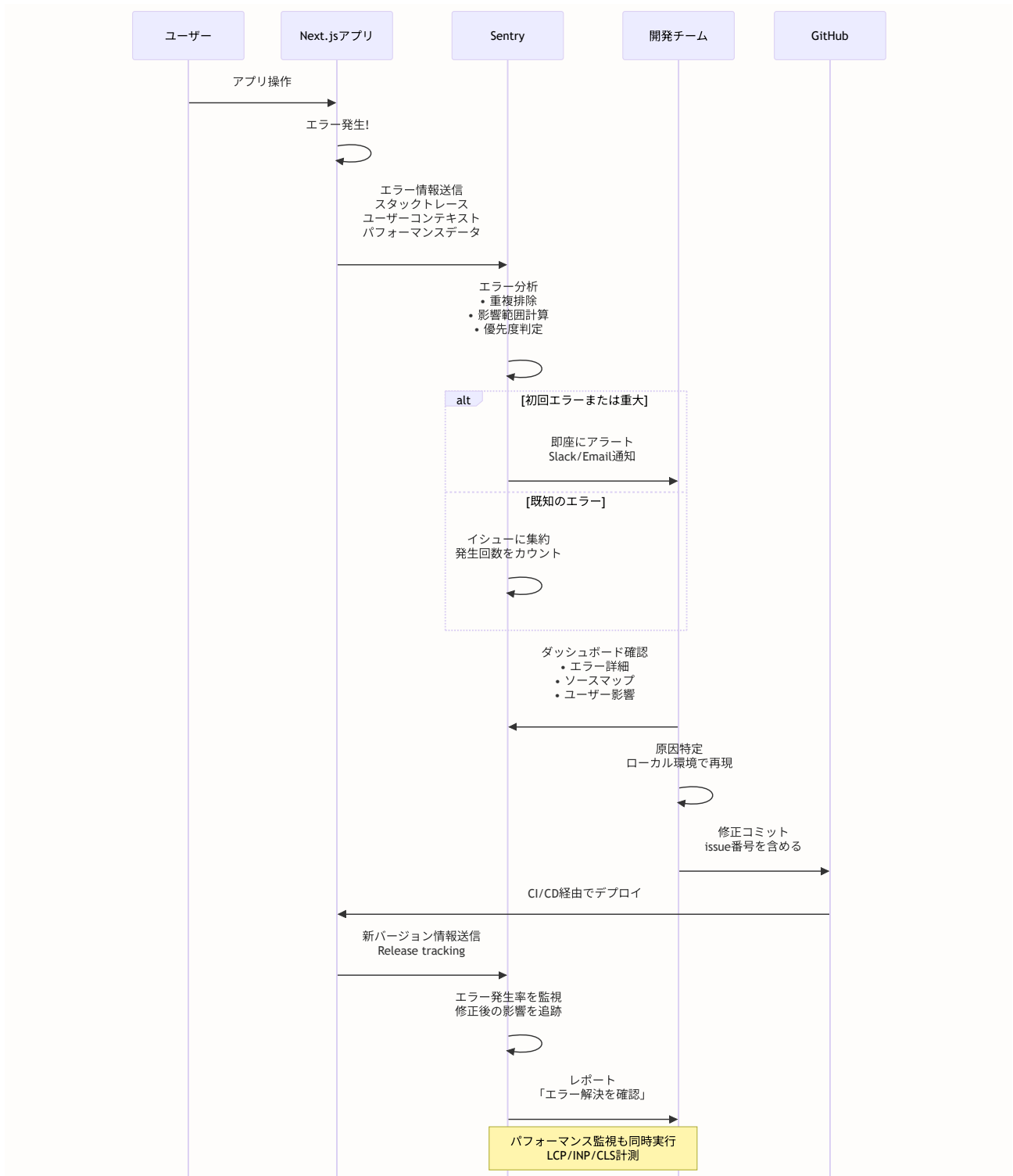
22.1.5 デプロイメントパイプライン

コードが本番環境に届くまでの完全な自動化プロセスを示します。



22.1.6 Sentryによる監視とエラー追跡フロー

本番環境でエラーが発生した際の検知から解決までの流れを示します。



▶ 理解度チェック（クリックして展開）

22.2 Supabase（PostgreSQL本番環境）

▶ このセクションで学ぶこと（クリックして展開）

BON-LOGでの使用箇所: 本番環境のデータベースとしてSupabase PostgreSQLを使用しています。

`DATABASE_URL`（Transaction mode: ポート6543）と `DIRECT_URL`（Session mode: ポート5432）の2つの接続文字列を `.env.local` に設定します。 `prisma/schema.prisma` の `datasource db` ブロックで両方が参照されます。マイグレーションは `npx prisma migrate deploy` で本番DBに適用します。

実装しない場合の影響: Vercelへのデプロイ時は必ず本番用PostgreSQLが必要です。ローカルのDockerでは本番環境からアクセスできません。Supabaseの無料プランは1プロジェクト・500MB・月50,000アクティブユーザーまで利用できます。別のPostgreSQLプロバイダー（Neon、Railway、PlanetScaleなど）でも `DATABASE_URL` を変更するだけで代替できます。

22.2.1 Supabaseとは

Supabaseは、PostgreSQLデータベースをクラウド上で簡単に管理できるサービスです。開発中はDockerでローカルにPostgreSQLを動かしていましたが、本番環境ではクラウド上のデータベースが必要です。

なぜSupabaseを選ぶのか？

本番環境

Supabase上の PostgreSQL

- クラウドで24時間稼働
- 世界中からアクセス可能
 - 自動バックアップ
 - スケーラブル

開発環境

Docker上の PostgreSQL

- 自分のPCで動作
- PCを閉じると停止
- 自分だけアクセス

Supabaseの特徴	説明
無料プランあり	小規模プロジェクトなら無料で利用可能
PostgreSQL互換	開発で使ったPrismaがそのまま使える
東京リージョン	日本のユーザーに高速レスポンス
自動バックアップ	データ消失のリスクを軽減
ダッシュボード	ブラウザからデータを確認・操作可能

22.2.2 プロジェクト作成

以下の手順で、Supabase上に本番用データベースを作成します。

ステップ1: アカウント作成・ログイン

1. [Supabase](#)にアクセス
2. GitHubアカウントでサインアップ（推奨）またはメールで登録

ステップ2: 新しいプロジェクトを作成

1. ダッシュボードで「New Project」をクリック
2. 以下の情報を入力します

```

+-----+
|           New Project           |
+-----+
| Organization: [あなたの組織名]   |
|                                   |
| Name:          bonlog-production |
| (プロジェクト名。わかりやすい名前をつける) |
|                                   |
| Database Password: [強力なパスワード] |
| (※ 必ずメモしておく！後から確認不可) |
|                                   |
| Region:        Northeast Asia (Tokyo) |
| (ユーザーの多い地域を選ぶ) |
|                                   |
| Pricing Plan:  Free              |
| (無料プランで開始) |
|                                   |
|           [ Create new project ] |
+-----+

```

重要: Database Passwordは後から確認できません。必ずパスワードマネージャーなどに保存してください。

ステップ3: プロジェクトの初期化を待つ

プロジェクト作成後、数分で初期化が完了します。ダッシュボードに「Project is ready」と表示されるまで待ちましょう。

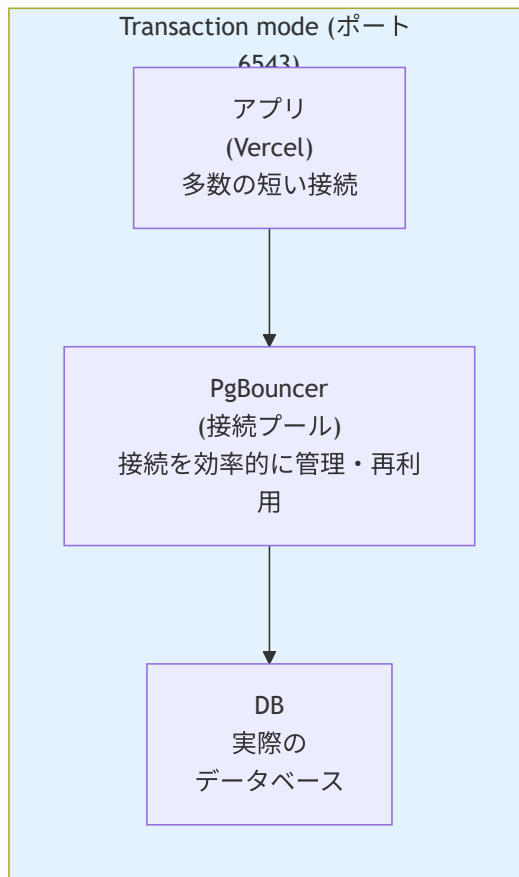
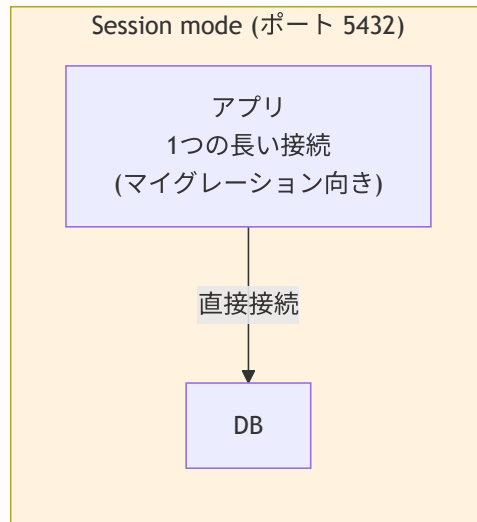
22.2.3 接続文字列の取得

データベースに接続するための「接続文字列 (Connection String)」を取得します。接続文字列とは、データベースの場所・ユーザー名・パスワードなどをまとめた文字列です。

Supabase ダッシュボード → Project Settings → Database → Connection string

2種類の接続モードがあります:

Transaction mode (トランザクションモード) と Session mode (セッションモード) の違い:



モード	ポート	用途	DATABASE_URL用
Transaction mode	6543	アプリからの通常接続（推奨）	はい
Session mode	5432	マイグレーション、Prisma Studio	DIRECT_URL用

たとえば: Transaction modeは「回転寿司」のようなもの。席（接続）を効率的に回転させて多くのお客様に対応します。Session modeは「予約席」のようなもの。1人のお客様がじっくり席を使います。

接続文字列の形式:

Transaction mode (推奨):

```
postgresql://postgres.[PROJECT_REF]:[PASSWORD]@aws-0-ap-northeast-1.pooler.supabase.com:6543/p
```

Session mode:

```
postgresql://postgres.[PROJECT_REF]:[PASSWORD]@aws-0-ap-northeast-1.pooler.supabase.com:5432/p
```

22.2.4 環境変数の設定（本番用）

取得した接続文字列を環境変数として設定します。

```
# .env.production (本番環境用の環境変数ファイル)

# Supabase Database
# DATABASE_URL: アプリからの接続に使用 (Transaction mode)
# pgbouncer=true を末尾につけることで、接続プールを経由する設定
DATABASE_URL="postgresql://postgres:[PASSWORD]@[HOST]:6543/postgres?pgbouncer=true"

# DIRECT_URL: マイグレーション実行時に使用 (Session mode)
# Prismaのマイグレーションは直接接続が必要なため、こちらを使う
DIRECT_URL="postgresql://postgres:[PASSWORD]@[HOST]:5432/postgres"
```

注意: `.env.production` ファイルは `.gitignore` に含まれていることを確認してください。パスワードをGitにコミットすると、セキュリティリスクになります。

本番データベースのマイグレーション

コマンド	用途	環境
<code>prisma db push</code>	スキーマを直接反映（履歴なし）	開発のみ
<code>prisma migrate dev</code>	マイグレーションファイル作成	開発
<code>prisma migrate deploy</code>	マイグレーション適用（安全）	本番

⚠ **本番で `prisma db push` は使わないでください。** データが失われる可能性があります。本番では必ず `prisma migrate deploy` を使用し、マイグレーション履歴で変更を追跡します。

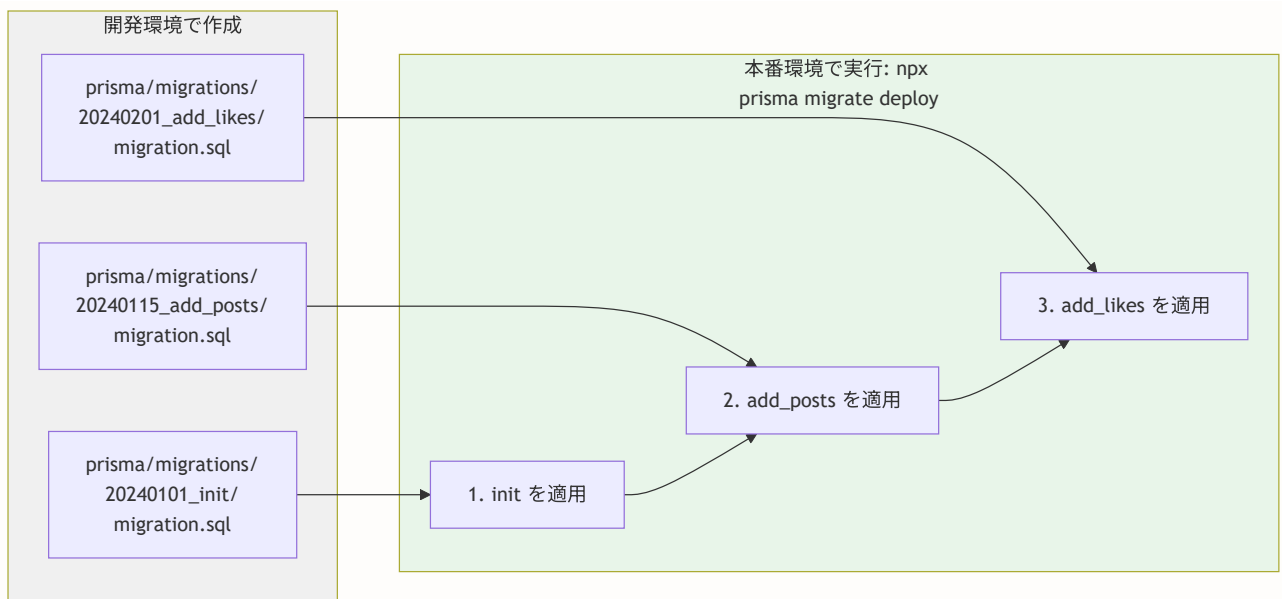
22.2.5 マイグレーション実行

開発中に作成したデータベーススキーマを本番データベースに適用します。

```
# 本番データベースにマイグレーションを適用するコマンド
# prisma migrate deploy は、既存のマイグレーションファイルを順番に実行する
# 新しいマイグレーションは作成せず、安全に適用のみ行う
npx prisma migrate deploy

# ※ db push は開発専用。本番では使わないこと！
# 理由: db push はスキーマの差分を直接適用するため、
#       データが失われる可能性がある
# npx prisma db push ← 本番では非推奨
```

マイグレーションの流れ:



よくあるトラブルと解決法

トラブル	原因	解決法
接続できない	パスワードが間違っている	Supabaseダッシュボードでパスワードをリセット
マイグレーションが失敗	DIRECT_URLを使っていない	<code>prisma migrate deploy</code> は DIRECT_URL (Session mode) が必要
タイムアウトする	リージョンが遠い	東京リージョンを選択しているか確認
<code>prepared statement already exists</code>	Transaction modeで Prisma Studio	Prisma Studioは DIRECT_URL を使う
接続数の上限に達した	同時接続が多すぎる	Transaction mode (PgBouncer) を使用

▶ 理解度チェック (クリックして展開)

22.3 Vercel へのデプロイ

▶ このセクションで学ぶこと (クリックして展開)

BON-LOGでの使用箇所: BON-LOGはVercelにデプロイされています。GitHubの `master` ブランチへの push で自動デプロイが実行されます。`next.config.ts` の `output: 'standalone'` は Docker ビルド向けの設定ですが、Vercel では自動的に無視されます。Vercel ダッシュボードで環境変数 (DATABASE_URL、NEXTAUTH_SECRET、R2 設定など) を設定する必要があります。

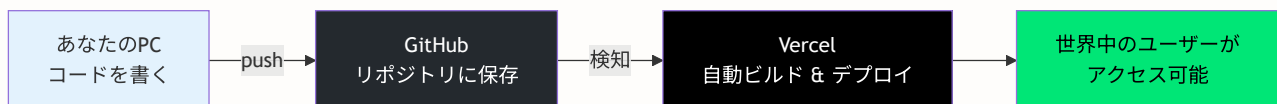
実装しない場合の影響: Vercel 以外のプラットフォーム (AWS、GCP、Fly.io 等) でも動作しますが、Next.js の全機能 (ISR、Server Actions、Edge Runtime) を最もシームレスに利用できるのは Vercel です。Vercel の無料プランはホビープロジェクト向けで、月間帯域幅 100GB まで利用できます。

22.3.1 Vercel とは

Vercel は、Next.js を開発した Vercel 社が運営するホスティングプラットフォームです。Next.js アプリのデプロイに最適化されており、ゼロコンフィグ (設定不要) でデプロイできます。

たとえば話で理解する:

Vercel は「出版社」のようなものです。あなた (開発者) が原稿 (コード) を書き、出版社 (Vercel) に渡すと、自動的に印刷 (ビルド) して本屋 (インターネット) に並べて (デプロイ) くれます。



Vercel の特徴:

特徴	説明
ゼロコンフィグ	Next.js を自動認識、特別な設定不要
自動デプロイ	GitHub に push するだけで自動的にデプロイ
プレビュー環境	PR ごとにプレビュー URL を自動生成
エッジネットワーク	世界中の CDN で高速配信
無料プラン	個人プロジェクトなら無料で利用可能
HTTPS 自動化	SSL 証明書が自動で設定される
サーバーレス関数	API Route が自動的にサーバーレスで実行

22.3.2 GitHubリポジトリとの連携

ステップ1: Vercelにサインアップ

1. [Vercel](#)にアクセス
2. 「Sign Up」 → 「Continue with GitHub」 をクリック
3. GitHubアカウントで認証

ステップ2: プロジェクトをインポート

1. Vercelダッシュボードで「Add New...」 → 「Project」 をクリック
2. 「Import Git Repository」 から `bonsai-sns-project` を選択
3. 「Import」 をクリック

```
+-----+
|           Import Project           |
+-----+
|                                     |
| Git Repository:                   |
| +-----+                         |
| | your-username/bonsai-sns-project | [Import] |
| +-----+                         |
|                                     |
| Framework Preset: Next.js (自動検出) |
|                                     |
| Root Directory: ./ (デフォルト)    |
|                                     |
| Build and Output Settings:        |
|   Build Command:    npm run build  |
|   Output Directory: .next           |
|   Install Command:  npm ci         |
|                                     |
+-----+
```

ポイント: Vercelは `package.json` を見てNext.jsプロジェクトを自動認識します。フレームワークプリセットが「Next.js」になっていることを確認してください。

22.3.3 環境変数の設定

Vercelダッシュボードで環境変数を設定します。これは非常に重要なステップです。環境変数が正しく設定されていないと、アプリは動作しません。

設定場所:

Vercel ダッシュボード → プロジェクト選択 → Settings → Environment Variables

環境変数の設定画面:

```

+-----+
| Environment Variables |
+-----+
|
| Key:  [DATABASE_URL          ] |
| Value: [postgresql:// ...    ] |
|
| Environment: |
|   [x] Production  [x] Preview  [ ] Development |
|
| [ Add ] |
+-----+

```

Production / Preview / Development の違い:

- **Production:** 本番環境（mainブランチのデプロイ先）
- **Preview:** プレビュー環境（PR作成時に自動生成される環境）
- **Development:** `vercel dev` コマンドでローカル実行する時

本番環境の環境変数設定 ローカルでは `.env.local` ファイルに書いていた環境変数を、Vercelでは管理画面で設定します：

1. [Vercel Dashboard](#) → プロジェクト選択
2. 「Settings」 → 「Environment Variables」
3. 各変数の名前と値を入力（Production / Preview / Development を選択）

重要: `.env.local` はVercelにアップロードされません。本番に必要な全ての環境変数をVercelダッシュボードで設定してください。

設定する環境変数の一覧:

```

# ===== データベース =====
# Supabaseの接続文字列
DATABASE_URL=postgresql:// ...      # Transaction mode (アプリ用)
DIRECT_URL=postgresql:// ...        # Session mode (マイグレーション用)

# ===== 認証 =====
# NextAuth.jsの設定
NEXTAUTH_URL=https://your-domain.vercel.app # デプロイ先のURL
NEXTAUTH_SECRET=your-production-secret      # 本番用の秘密鍵

# ===== 決済 =====
# Stripeの本番キー (sk_test_ ではなく sk_live_ を使う!)
STRIPE_SECRET_KEY=sk_live_xxxxx          # 本番用シークレットキー
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_live_xxxxx # 本番用公開キー
STRIPE_WEBHOOK_SECRET=whsec_xxxxx        # Webhook検証用シークレット
STRIPE_PREMIUM_PRICE_ID=price_xxxxx       # プレミアムプランの価格ID

# ===== キャッシュ =====
# Upstash Redisの接続情報
UPSTASH_REDIS_REST_URL=https:// ...      # RedisのREST APIエンドポイント
UPSTASH_REDIS_REST_TOKEN= ...             # 認証トークン

# ===== ストレージ =====
# Cloudflare R2 (画像保存用)
R2_ACCOUNT_ID= ...                       # CloudflareアカウントID
R2_ACCESS_KEY_ID= ...                     # R2アクセスキー
R2_SECRET_ACCESS_KEY= ...                 # R2シークレットキー
R2_BUCKET_NAME=bonlog-production         # バケット名
R2_PUBLIC_URL=https:// ...                # 公開URL

# ===== メール =====
# Resend (通知メール送信用)
RESEND_API_KEY=re_xxxxx                   # ResendのAPIキー

# ===== 監視 =====
# Sentry (エラー監視用)
SENTRY_DSN=https:// ...                   # サーバー側DSN
NEXT_PUBLIC_SENTRY_DSN=https:// ...       # クライアント側DSN

# ===== 定期実行 =====
CRON_SECRET=your-cron-secret              # Cronジョブの認証シークレット

```


セキュリティ上の注意点:

- `NEXT_PUBLIC_` で始まる変数はブラウザに公開されます。秘密情報には使わないでください
- `sk_live_` は本番Stripeキーです。テスト用の `sk_test_` と混同しないよう注意
- `NEXTAUTH_SECRET` は `openssl rand -base64 32` で生成した強力なランダム文字列を使用

22.3.4 ビルド設定

Vercelのビルド設定は通常、自動検出されるためそのまま問題ありません。

```
+-----+
| Build & Development Settings |
+-----+
|
| Build Command:      npm run build |
| (Next.jsのビルドコマンド) |
|
| Output Directory:   .next |
| (ビルド結果が出力されるディレクトリ) |
|
| Install Command:    npm ci |
| (依存パッケージのインストール) |
| ※ npm install より高速で確実 |
|
| Development Command: npm run dev |
| (vercel dev 使用時のコマンド) |
|
+-----+
```

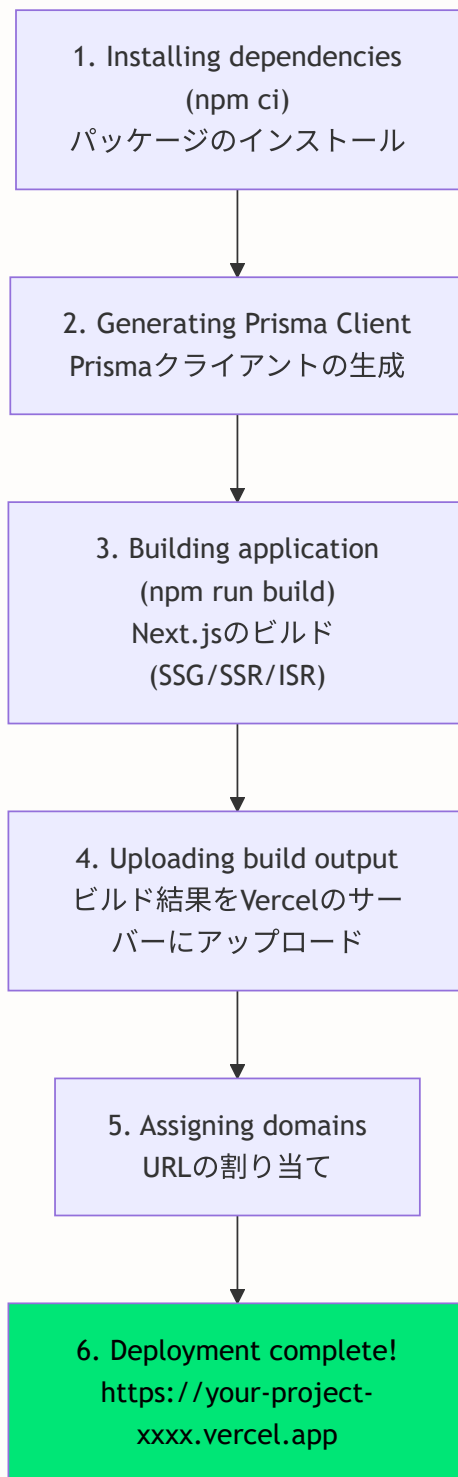
`npm ci` と `npm install` の違い:

- `npm ci`: `package-lock.json` に記載された正確なバージョンをインストール。CI/CD環境で推奨
- `npm install`: バージョン範囲内で最新をインストール。開発時に使用

22.3.5 デプロイ

「Deploy」ボタンをクリックすると、自動的にビルド・デプロイが開始されます。

デプロイの進行状況:

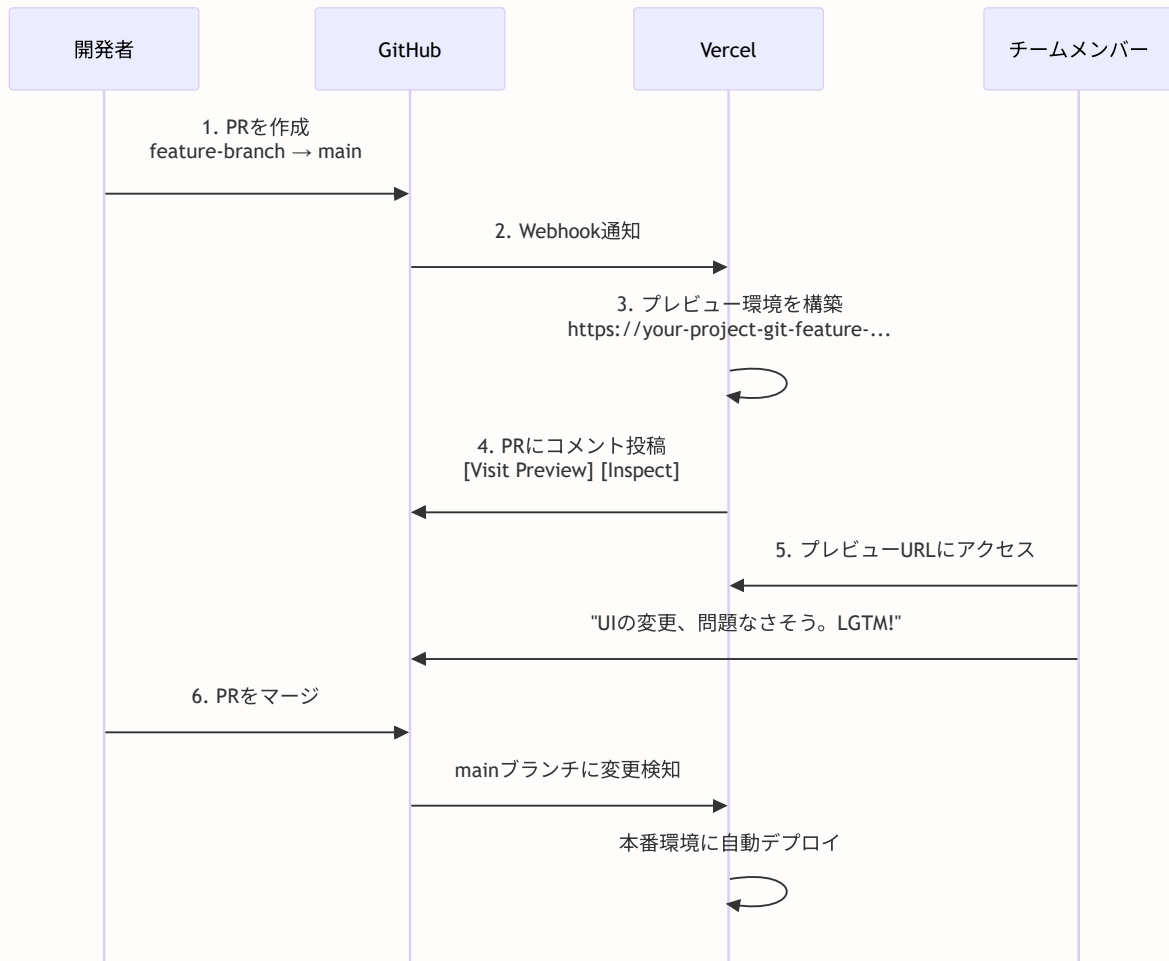


デプロイ完了後、自動的にURLが生成されます。このURLでアプリにアクセスできます。

初回デプロイの注意: 最初のデプロイは、データベースのマイグレーションも必要です。Vercelのデプロイ後、ローカルから `npx prisma migrate deploy` を実行するか、Vercelの「Build Command」を `npx prisma migrate deploy && npm run build` に変更してください。

22.3.6 プレビューデプロイ

Vercelの強力な機能の1つが「プレビューデプロイ」です。プルリクエストを作成すると、自動的にプレビュー環境が生成されます。



活用のコツ: チームでコードレビューする際、プレビューURLを共有すれば、コードを読まなくても実際の動作を確認できます。デザイナーやプロダクトマネージャーにも確認してもらいやすくなります。

22.3.7 ドメイン設定

デフォルトでは `your-project.vercel.app` というURLが割り当てられますが、独自のドメイン（例: `bonlog.app`）を設定できます。

カスタムドメインの追加

ステップ1: Vercelにドメインを追加

1. Vercel ダッシュボード → プロジェクト選択 → Settings → Domains
2. 「Add」をクリック
3. ドメイン名を入力: `bonlog.app`

ステップ2: DNSレコードを設定

ドメインレジストラ（お名前.com、Google Domains等）の管理画面で、以下のDNSレコードを追加します。

DNSレコードの設定:

```
# ルートドメイン (bonlog.app)
# Aレコード: ドメインをIPアドレスに紐づける
Type: A
Name: @           ← ルートドメインを意味する
Value: 76.76.21.21 ← VercelのIPアドレス

# www サブドメイン (www.bonlog.app)
# CNAMEレコード: 別のドメインを参照する
Type: CNAME
Name: www
Value: cname.vercel-dns.com ← Vercelのドメインサーバー
```

DNSレコードの種類を理解しよう:

レコード種類	役割	例
A レコード	ドメインをIPアドレスに紐づける	<code>bonlog.app</code> → <code>76.76.21.21</code>
CNAME レコード	ドメインを別のドメインに転送	<code>www.bonlog.app</code> → <code>cname.vercel-dns.com</code>
TXT レコード	ドメインの所有権を証明	Vercelの所有権確認で使用

サブドメインの設定

特定のサブドメイン（例: `app.bonlog.jp`）でアプリにアクセスさせたい場合:

```
Type: CNAME
Name: app           ← サブドメイン名
Value: cname.vercel-dns.com
```

これで `https://app.bonlog.jp` でアクセス可能になります。

SSL証明書: Vercelはカスタムドメインに対して自動的にSSL証明書（HTTPS）を発行します。手動での設定は不要です。

よくあるトラブルと解決法

トラブル	原因	解決法
ビルドが失敗する	環境変数が未設定	Vercelのダッシュボードで全ての環境変数を確認
ビルドが失敗する	TypeScriptの型エラー	ローカルで <code>npm run build</code> を実行して修正
ドメインが反映されない	DNS伝播に時間がかかる	最大48時間待つ（通常は数分～数時間）
404エラーが出る	ルーティング設定の問題	<code>vercel.json</code> の <code>rewrites</code> 設定を確認
APIが動かない	環境変数が Production に設定されていない	環境選択（Production/Preview/Development）を確認
画像が表示されない	<code>next.config.ts</code> の <code>remotePatterns</code> 未設定	外部画像のホスト名を許可設定に追加

▶ 理解度チェック（クリックして展開）

22.4 GitHub Actions CI/CD

▶ このセクションで学ぶこと（クリックして展開）

BON-LOGでの使用箇所: `.github/workflows/ci.yml` に定義されています。`main`/`master` ブランチへのpushとPR作成時に自動実行されます。`lint`、`test`、`build` の3ジョブが並列実行され、`e2e` ジョブは3つが全て成功した後に実行されます（現在は手動実行推奨のため実質的に無効化）。ユニットテストは `npx vitest run --coverage` で実行され、カバレッジレポートがアーティファクトとして7日間保存されます。

実装しない場合の影響: CI/CDがないと、コードのpush前にテストを手動実行しなければならず、バグが本番環境に混入するリスクが高まります。チーム開発では特に重要で、他のメンバーのコード変更が自分のコードに影響しているかどうかを自動的に確認できなくなります。

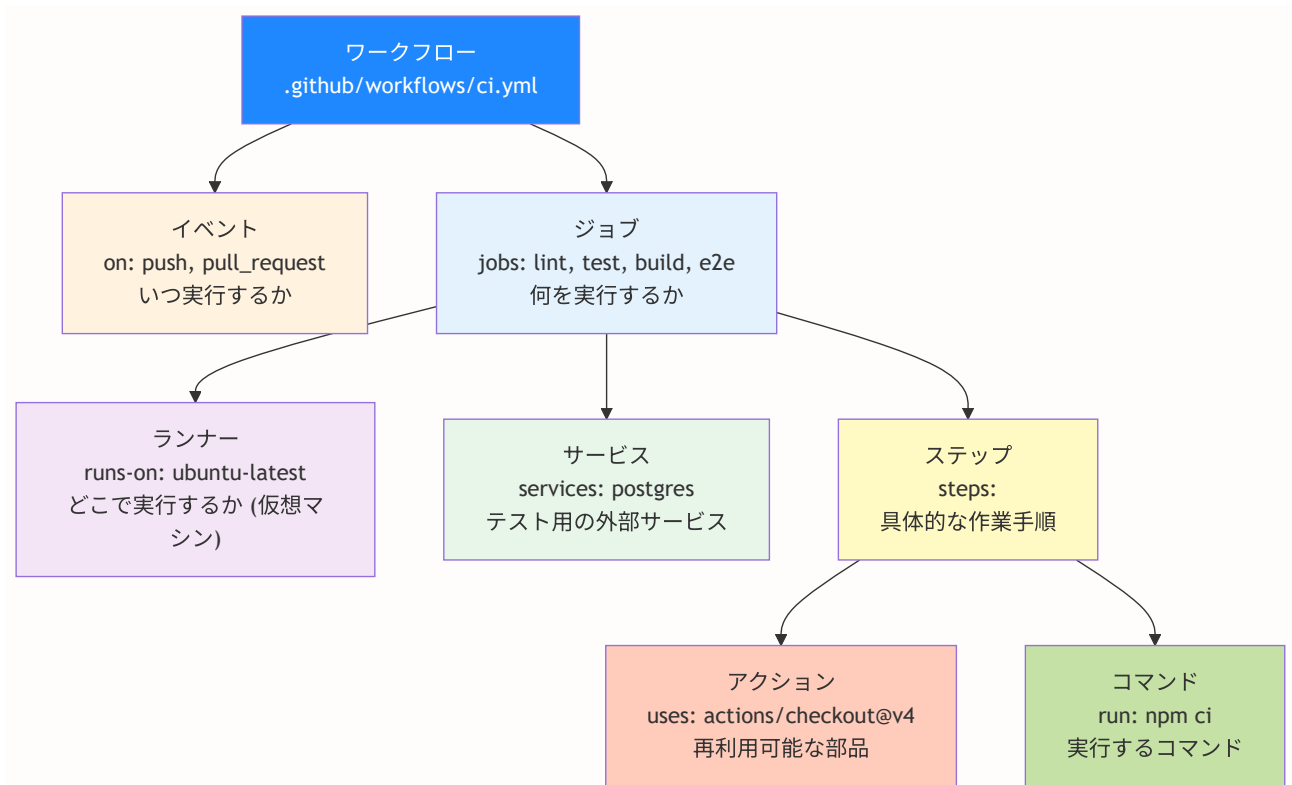
22.4.1 GitHub Actionsとは

GitHub Actionsは、GitHubに組み込まれたCI/CDサービスです。リポジトリに設定ファイル（YAMLファイル）を置くだけで、コードのプッシュやプルリクエストをきっかけに自動でタスクを実行できます。

たとえ話: GitHub Actionsは「工場の自動生産ライン」のようなものです。原材料（コード）が入ると、品質検査（テスト）→組立（ビルド）→出荷（デプロイ）を自動で行います。不良品（バグ）が見つかったら、ラインが止まって通知されます。

22.4.2 GitHub Actionsの基本用語

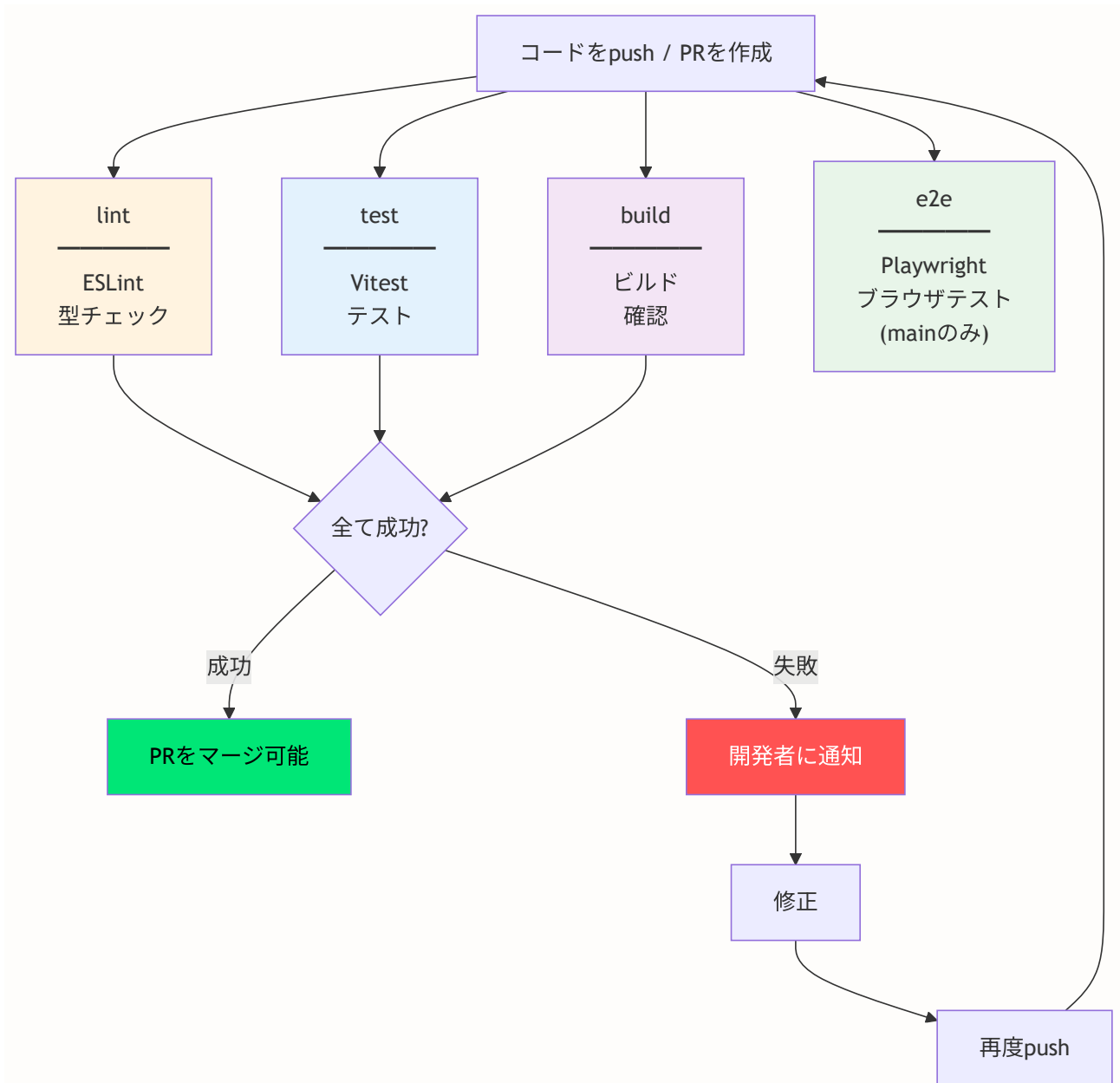
まず、GitHub Actionsで使われる用語を理解しましょう。



用語	説明	日常のたとえ
ワークフロー	CI/CDの全体的な定義ファイル	業務マニュアル全体
イベント	ワークフローを起動するきっかけ	「注文が入ったら」という条件
ジョブ	独立して実行される作業のまとまり	部署ごとの担当業務
ステップ	ジョブ内の個々の作業	業務手順の1つ1つ
アクション	再利用可能なステップの部品	標準作業手順書(SOP)
ランナー	ジョブを実行する仮想マシン	作業場 (ubuntu-latest = Linux環境)
サービス	テスト用のDockerコンテナ	テスト用の設備 (DB等)

22.4.3 パイプラインの全体設計

BON-LOGのCI/CDパイプラインは4つのジョブで構成されています。



ジョブ	内容	実行条件	所要時間目安
lint	ESLint + TypeScript型チェック (<code>tsconfig.check.json</code> 使用)	常時	1~2分
test	Vitestユニットテスト + カバレッジ	常時	2~5分
build	Next.jsビルド確認	常時	3~5分
e2e	Playwrightブラウザテスト (一時的に無効化、手動実行推奨)	mainのみ	5~15分

22.4.4 ワークフローファイルの詳細解説

`.github/workflows/ci.yml` ファイルの各行を詳しく解説します。


```

# ===== ワークフロー名 =====
# GitHubのActionsタブに表示される名前
name: CI

# ===== イベント定義 =====
# "どのタイミングでワークフローを起動するか" を定義
on:
  # mainまたはmasterブランチにpushされた時に実行
  push:
    branches: [main, master]
  # mainまたはmasterブランチへのプルリクエストが作成・更新された時に実行
  pull_request:
    branches: [main, master]

# 同じブランチの古いワークフローをキャンセル
# 例: PRに連続でpushした場合、前のワークフローを止めて最新だけ実行
concurrency:
  group: ${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

# 全ジョブ共通の環境変数
env:
  NODE_VERSION: '20'
  # Prisma用ダミー環境変数（ユニットテストでは実際のDB接続は不要）
  DATABASE_URL: "postgresql://dummy:dummy@localhost:5432/dummy"
  DIRECT_URL: "postgresql://dummy:dummy@localhost:5432/dummy"

# ===== ジョブ定義 =====
# ワークフロー内で実行するジョブ（作業）を定義
jobs:

  # =====
  # ジョブ1: lint（コード品質チェック）
  # =====
  # ESLintでコードスタイルをチェックし、
  # TypeScriptの型エラーがないかを確認する
  lint:
    name: Lint & Type Check
    # ubuntu-latest: 最新のUbuntu Linux仮想マシンで実行
    # GitHub側が用意してくれるクラウド上のマシン
    runs-on: ubuntu-latest
    steps:
      # ステップ1: リポジトリのコードをチェックアウト（ダウンロード）
      # actions/checkout@v4 は公式が提供する再利用アクション
      - name: Checkout
        uses: actions/checkout@v4

      # ステップ2: Node.jsの環境を準備
      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:

```

```

# 使用するNode.jsのバージョン
node-version: ${ env.NODE_VERSION }}
# npmのキャッシュを有効化（2回目以降のインストールが高速化）
cache: 'npm'

# ステップ3: 依存パッケージをインストール
# npm ci は package-lock.json の通りに正確にインストール
- name: Install dependencies
  run: npm ci

# ステップ4: Prismaクライアントを生成（型チェックに必要）
- name: Generate Prisma Client
  run: npx prisma generate

# ステップ5: ESLintでコードスタイルをチェック
# 命名規則、未使用変数、インポート順序などをチェック
- name: Run ESLint
  run: npm run lint

# ステップ6: TypeScriptの型チェック
# --noEmit: ファイルを出力せず、型チェックのみ実行
# -p tsconfig.check.json: テスト用の専用tsconfig（Next.js特有の設定を除外）
- name: Run TypeScript type check
  run: npx tsc --noEmit -p tsconfig.check.json

# =====
# ジョブ2: test（ユニットテスト）
# =====
# Vitestでユニットテストを実行し、カバレッジを計測する
# 注意: ユニットテストではDBサービスは不要（モックを使用するため）
test:
  name: Unit Tests
  runs-on: ubuntu-latest

  steps:
    # コードのチェックアウト
    - name: Checkout
      uses: actions/checkout@v4

    # Node.js環境のセットアップ
    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }}
        cache: 'npm'

    # 依存パッケージのインストール
    - name: Install dependencies
      run: npm ci

    # Prismaクライアントを生成
    # schema.prismaからTypeScriptの型定義を生成する
    - name: Generate Prisma Client

```

```

    run: npx prisma generate

# ユニットテストを実行（カバレッジ付き）
# --ci: CI環境向けの最適化（スナップショット自動更新無効化等）
# --maxWorkers=2: 並列数を制限してメモリ消費を抑える
- name: Run unit tests
  run: npx vitest run --coverage
  env:
    NODE_OPTIONS: "--max-old-space-size=4096"

# テストカバレッジをアーティファクトとしてアップロード
# GitHubのActionsタブからダウンロード可能
- name: Upload coverage report
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: coverage-report
    path: coverage/
    retention-days: 7

# =====
# ジョブ3: build（ビルド確認）
# =====
# 本番用にNext.jsをビルドできるか確認する
# ビルドエラーがあれば、このジョブが失敗する
build:
  name: Build
  runs-on: ubuntu-latest
  steps:
    - name: Checkout
      uses: actions/checkout@v4

    - name: Setup Node.js
      uses: actions/setup-node@v4
      with:
        node-version: ${ env.NODE_VERSION }
        cache: 'npm'

    - name: Install dependencies
      run: npm ci

    - name: Generate Prisma Client
      run: npx prisma generate

# ビルドを実行
# ビルド時にはDBに接続しないが、環境変数の参照でエラーにならないよう
# ダミーの値を設定している
- name: Build application
  run: npm run build
  env:
    NEXTAUTH_SECRET: "build-time-secret"
    NEXTAUTH_URL: "http://localhost:3000"
    NODE_OPTIONS: "--max-old-space-size=4096"

```

```

# =====
# ジョブ4: e2e (E2Eテスト)
# =====
# Playwrightでブラウザを自動操作し、実際のユーザー操作をテスト
# lint, test, build が全て成功してから実行
e2e:
  name: E2E Tests
  runs-on: ubuntu-latest
  needs: [lint, test, build]      # 前の3ジョブが成功してから実行
  # タイムアウト: 15分を超えたら強制終了（無限ループ防止）
  timeout-minutes: 15

# E2Eテストでは実際のDBが必要
services:
  postgres:
    image: postgres:16-alpine
    env:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: postgres
      POSTGRES_DB: bonsai_sns_test # テスト専用のDB名
    options: >-
      --health-cmd pg_isready
      --health-interval 10s
      --health-timeout 5s
      --health-retries 5
    ports:
      - 5432:5432

steps:
  - name: Checkout
    uses: actions/checkout@v4

  - name: Setup Node.js
    uses: actions/setup-node@v4
    with:
      node-version: ${ env.NODE_VERSION }
      cache: 'npm'

  - name: Install dependencies
    run: npm ci

  - name: Generate Prisma Client
    run: npx prisma generate

# E2Eテスト用データベースにスキーマを適用
  - name: Setup database
    run: npx prisma db push
    env:
      DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
      DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"

# テスト用の初期データ（シードデータ）を投入

```

```

# ログイン用のテストユーザーなどを作成する
- name: Seed test data
  run: npx prisma db seed
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"

# Playwrightが使うブラウザをインストール (chromiumのみで高速化)
# --with-deps: OSレベルの依存ライブラリも一緒にインストール
- name: Install Playwright browsers
  run: npx playwright install --with-deps chromium

# アプリケーションをビルド
- name: Build application
  run: npm run build
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
    NEXTAUTH_URL: "http://localhost:3000"
    NEXT_PUBLIC_APP_URL: "http://localhost:3000"
    AUTH_TRUST_HOST: "true"
    NODE_OPTIONS: "--max-old-space-size=4096"

# standaloneサーバーの準備
- name: Prepare standalone server
  run: |
    cp -r public .next/standalone/public 2>/dev/null || true
    cp -r .next/static .next/standalone/.next/static 2>/dev/null || true
    cp -r node_modules/.prisma .next/standalone/node_modules/.prisma 2>/dev/null || true
    cp -r node_modules/@prisma .next/standalone/node_modules/@prisma 2>/dev/null || true

# サーバーを起動してヘルスチェック
- name: Start server and check health
  run: |
    node .next/standalone/server.js &
    SERVER_PID=$!
    for i in $(seq 1 30); do
      if curl -s http://localhost:3000/login > /dev/null 2>&1; then
        echo "Server is ready"
        break
      fi
      sleep 2
    done
    kill $SERVER_PID 2>/dev/null || true
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
    NEXTAUTH_URL: "http://localhost:3000"
    AUTH_TRUST_HOST: "true"
    PORT: "3000"
    HOSTNAME: "0.0.0.0"

```

```

    NODE_ENV: "production"

# E2Eテストを実行（chromiumプロジェクトのみ）
- name: Run E2E tests
  run: npx playwright test --project=chromium --project=chromium-noauth
  env:
    DATABASE_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    DIRECT_URL: "postgresql://postgres:postgres@localhost:5432/bonsai_sns_test?sslmode=require"
    NEXTAUTH_SECRET: "ci-e2e-test-secret-key"
    NEXTAUTH_URL: "http://localhost:3000"
    AUTH_TRUST_HOST: "true"
    PORT: "3000"
    HOSTNAME: "0.0.0.0"
    NODE_ENV: "production"
  timeout-minutes: 15

# テスト結果のレポートをアーティファクトとしてアップロード
# if: always() は、テストが成功しても失敗してもレポートを保存する
# GitHubのActionsタブからダウンロード可能
- name: Upload Playwright report
  uses: actions/upload-artifact@v4
  if: always()
  with:
    name: playwright-report           # アーティファクトの名前
    path: playwright-report/         # レポートのディレクトリ
    retention-days: 7

```

22.4.5 ワークフローの実行結果の見方

GitHub Actionsの実行結果は、GitHubのリポジトリページから確認できます。

確認方法:

GitHub リポジトリ → Actions タブ

```
+-----+
| Actions |
+-----+
| CI      |
|         |
| [成功] feat: 投稿機能を追加      2分前  1m 45s |
| lint    [成功]                  |
| test    [成功]                  |
| build    [成功]                  |
|         |
| [失敗] fix: バグ修正              5分前  0m 32s |
| lint    [失敗] ← ここをクリックで詳細表示 |
| test    [成功]                  |
| build    [スキップ]              |
|         |
+-----+
```

プルリクエストでの表示:

```
+-----+
| Pull Request #42: 投稿機能の改善 |
+-----+
|                                     |
| All checks have passed            |
|                                     |
| [成功] lint      - ESLint + 型チェック |
| [成功] test      - ユニットテスト      |
| [成功] build      - ビルド確認          |
|                                     |
| [Merge pull request]              |
|                                     |
+-----+
```

22.4.6 ブランチ保護ルール

ブランチ保護ルールを設定すると、CI/CDが全て成功しないとマージできなくなります。これにより、バグのあるコードが本番に混入するのを防ぎます。

設定手順:

GitHub リポジトリ → Settings → Branches → Add branch protection rule

```

+-----+
| Branch protection rule |
+-----+
|
| Branch name pattern: main
|
| [x] Require a pull request before merging
|   (直接pushを禁止、PRが必要)
| [x] Require approvals: 1
|   (最低1人のレビュー承認が必要)
|
| [x] Require status checks to pass
|   (CIが全て成功している必要がある)
| [x] Require branches to be up to date
|   (最新のmainブランチとの同期が必要)
| Required checks:
|   - lint
|   - test
|   - build
|
| [x] Require conversation resolution
|   (全てのレビューコメントが解決済みである必要)
|
+-----+

```

保護ルールの効果:

保護ルールなし:

push to main → 本番
(テスト未実施) (バグ混入)

保護ルールあり:

push to main → ブロック!

```

|
v
PRを作成
|
v
CI実行 (自動テスト)
|
v
レビュー承認
|
v
マージ → 本番
          (品質保証済み)

```


よくあるトラブルと解決法

トラブル	原因	解決法
ワークフローが実行されない	YAMLファイルのパスが間違っている	<code>.github/workflows/</code> ディレクトリに配置されているか確認
<code>npm ci</code> が失敗する	<code>package-lock.json</code> が古い	ローカルで <code>npm install</code> 後にcommit
PostgreSQLに接続できない	サービスコンテナの起動に時間がかかっている	<code>health-check</code> の設定を確認
テストがタイムアウトする	テストが遅い、または無限ループ	<code>timeout-minutes</code> を適切に設定
キャッシュが効かない	キャッシュキーが変わっている	<code>cache: 'npm'</code> でpackage-lock.jsonベースのキャッシュを使用
E2Eテストが不安定	ブラウザの起動タイミング	<code>webServer</code> の設定で起動を待つ

▶ 理解度チェック（クリックして展開）

22.5 Docker（本番ビルド）

▶ このセクションで学ぶこと（クリックして展開）

BON-LOGでの使用箇所: プロジェクトルートの `Dockerfile`（本番用3ステージビルド）と `docker-compose.yml`（ローカル開発用）が実装されています。Dockerfileは `deps`（依存関係インストール）→ `builder`（ビルド）→ `runner`（実行）の3ステージ構成で、最終イメージには `node:20-alpine` の軽量ベースを使用し、非rootユーザー（nextjs/nodejs）で実行します。ヘルスチェックエンドポイント `/api/health` にポーリングして起動確認を行います。

実装しない場合の影響: VercelへのデプロイはDockerを使わないため、Vercelのみで運用する場合はDockerなしでも問題ありません。ただし「ローカルで動くのに本番で動かない」問題を再現・デバッグするためにDockerは非常に有用です。`next.config.ts` の `output: 'standalone'` はDockerとVercelの両方で機能し、Vercelでは自動的に無視されます。

22.5.1 なぜDockerで本番ビルドするのか

Dockerを使って、本番環境と同じ構成でローカルテストできます。「自分のPCでは動くのに、本番では動かない」という問題を防ぎます。

たとえ話: Dockerは「引越し用の段ボール箱」のようなものです。アプリに必要なもの（コード、ライブラリ、設定）をすべて1つの箱（コンテナ）に詰めて運ぶので、どこに置いても同じように動きます。

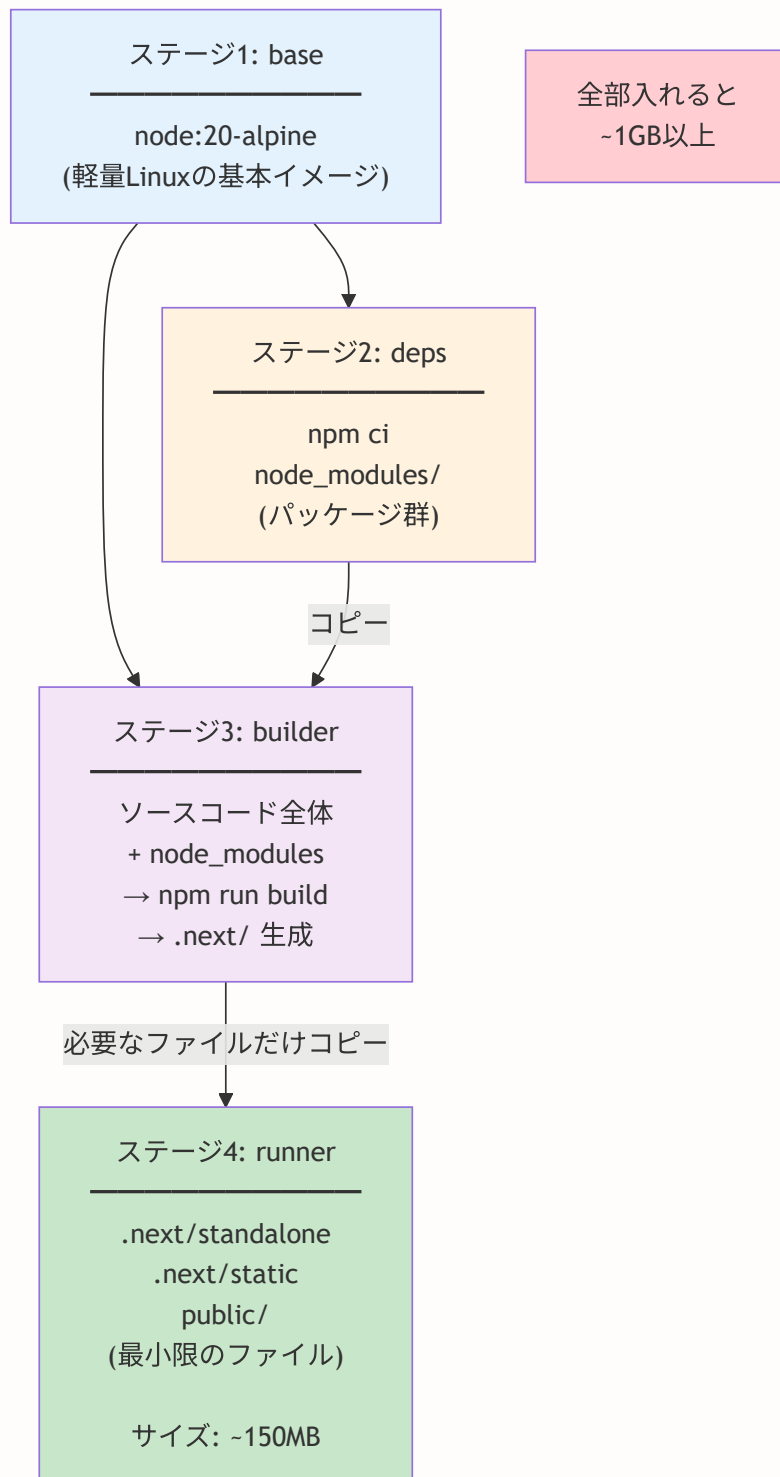
Dockerなしの場合:

Dockerありの場合:

```
<div class="mermaid-rendered">
<img src="data:image/svg+xml;base64,PHN2ZyBpZD0ibW1kXzIyX2RlcGxveV8xNSIgd2lkdgG9IjEwMCUiIHhtbG
</div>
```

22.5.2 マルチステージビルドの仕組み

BON-LOGのDockerfileは「マルチステージビルド」という技術を使っています。これは、ビルド用の大きな環境でアプリを作り、実行に必要な最小限のファイルだけを本番用イメージにコピーする手法です。



22.5.3 Dockerfileの詳細解説

BON-LOGの実際の `Dockerfile` : プロジェクトルートの `Dockerfile` は `deps` (依存関係) → `builder` (ビルド) → `runner` (実行) の3ステージ構成です。各ステージが `FROM node:20-alpine` を直接使用しています。以下は教育目的で `base` ステージを追加した解説版です。

```

# ===== ステージ1: base =====
# 全てのステージの基盤となるイメージ
# node:20-alpine は Node.js 20 が入った軽量Linux (Alpine Linux)
# Alpine Linux: 通常のLinuxイメージ(~900MB)に比べ ~50MB と非常に小さい
FROM node:20-alpine AS base

# ===== ステージ2: deps (依存関係インストール) =====
# パッケージのインストールだけを行うステージ
# ソースコードが変わっても、package.jsonが変わらなければ
# このステージはキャッシュされ、再実行されない(ビルド高速化)
FROM base AS deps
# 作業ディレクトリを /app に設定
WORKDIR /app

# package.json と package-lock.json だけを先にコピー
# (ソースコードより先にコピーすることでキャッシュ効率を上げる)
COPY package.json package-lock.json ./
# npm ci でパッケージをクリーンインストール
RUN npm ci

# ===== ステージ3: builder (ビルド) =====
# アプリケーションをビルドするステージ
FROM base AS builder
WORKDIR /app

# deps ステージからnode_modulesをコピー
COPY --from=deps /app/node_modules ./node_modules
# ソースコード全体をコピー
COPY . .

# 環境変数の設定 (ビルド時に使用)
# NEXT_TELEMETRY_DISABLED: Next.jsの利用統計送信を無効化
ENV NEXT_TELEMETRY_DISABLED 1
# NODE_ENV: 本番モードでビルド (最適化が有効になる)
ENV NODE_ENV production

# Prismaクライアントを生成
# schema.prismaからTypeScript型定義とクエリエンジンを生成
RUN npx prisma generate

# Next.jsの本番ビルドを実行
# ページの静的生成、コードの最適化・圧縮が行われる
RUN npm run build

# ===== ステージ4: runner (実行) =====
# 本番で実際にアプリを動かすステージ
# ビルドに使ったツールは含めず、実行に必要なファイルだけを持つ
FROM base AS runner
WORKDIR /app

# 本番モードであることを明示

```

```
ENV NODE_ENV production
# テレメトリーを無効化
ENV NEXT_TELEMETRY_DISABLED 1

# セキュリティ: root権限ではなく専用ユーザーで実行
# グループ "nodejs" (GID 1001) を作成
RUN addgroup --system --gid 1001 nodejs
# ユーザー "nextjs" (UID 1001) を作成
RUN adduser --system --uid 1001 nextjs

# builder ステージから必要なファイルだけをコピー
# public/: 静的ファイル(favicon, robots.txt等)
COPY --from=builder /app/public ./public
# .next/standalone: サーバーと最小限のnode_modulesが含まれる
COPY --from=builder /app/.next/standalone ./
# .next/static: CSSやJSのバンドルファイル
COPY --from=builder /app/.next/static ./next/static

# 専用ユーザーに切り替え (root権限で動かさない)
USER nextjs

# コンテナがリッスンするポートを宣言
EXPOSE 3000

# サーバーのポート番号
ENV PORT 3000
# 全てのIPアドレスからの接続を受け付ける
ENV HOSTNAME "0.0.0.0"

# アプリを起動するコマンド
# standalone モードでは server.js がエントリーポイント
CMD ["node", "server.js"]
```

22.5.4 next.config.ts (standalone設定)

Dockerで本番ビルドするには、Next.jsの `standalone` 出力モードを有効にする必要があります。

```
// next.config.ts (実際の実装 -- 抜粋)
import type { NextConfig } from 'next'
import { withSentryConfig } from '@sentry/nextjs' // Sentry統合

const nextConfig: NextConfig = {
  // standalone: 実行に必要なファイルだけを .next/standalone に出力する設定
  // これにより、node_modules全体をコピーせずに済み、イメージサイズが大幅に縮小
  // Vercelでは自動的に無視される（Vercel側がstandaloneを管理するため）
  output: 'standalone',

  images: {
    remotePatterns: [
      { protocol: 'https', hostname: '*.r2.dev' }, // Cloudflare R2
      { protocol: 'https', hostname: '*.supabase.co' }, // Supabase Storage
      { protocol: 'https', hostname: 'images.unsplash.com' }, // Unsplash
    ],
  },

  experimental: {
    serverActions: {
      bodySizeLimit: '10mb', // 画像アップロード用に10MBまで許可
    },
  },
}

// Sentry設定でラップ（エラー監視・ソースマップアップロード）
export default withSentryConfig(nextConfig, {
  org: process.env.SENTRY_ORG,
  project: process.env.SENTRY_PROJECT,
  silent: !process.env.CI,
})
```

standalone モードとは: 通常のNext.jsビルドでは `node_modules` 全体（数百MB）が必要ですが、**standalone** モードでは実行に必要な最小限のファイルだけを `.next/standalone` にコピーします。これにより、Dockerイメージのサイズを大幅に縮小できます。Vercel環境では自動的にこの設定が無視されるため、VercelとDockerの両方に対応した設定になっています。

22.5.5 docker-compose.yml（ローカル開発環境用）

docker-compose.ymlは、複数のコンテナ（アプリ、DB）をまとめて起動・管理するための設定ファイルです。BON-LOGのdocker-compose.ymlでは**profiles**（プロファイル）機能を使って、開発モードと本番モードを切り替えられるようになっています。

```

# Docker Compose設定ファイル
# 複数のサービスをまとめて定義・管理する
# 注意: version キーは不要 (Docker Compose V2以降では省略推奨)

services:
  # ===== PostgreSQLデータベース =====
  # プロファイルなし = 常に起動対象
  postgres:
    # 使用するイメージ: PostgreSQL 16 (Alpine Linux版で軽量)
    image: postgres:16-alpine
    container_name: bonsai-postgres
    restart: unless-stopped
    environment:
      # データベースの初期設定
      POSTGRES_USER: postgres      # 管理者ユーザー名
      POSTGRES_PASSWORD: postgres  # パスワード
      POSTGRES_DB: bonsai_sns      # データベース名
    ports:
      # ホストの5432番ポートをコンテナの5432番にマッピング
      # "ホスト:コンテナ" の形式
      - "5432:5432"
    volumes:
      # データを永続化 (コンテナを停止してもデータが残る)
      # postgres_data という名前のボリュームに保存
      - postgres_data:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres -d bonsai_sns"]
      interval: 10s
      timeout: 5s
      retries: 5

  # ===== アプリケーション (開発モード) =====
  # profiles: [dev] → docker compose --profile dev up で起動
  app:
    build:
      context: .
      dockerfile: Dockerfile.dev      # 開発用Dockerfile
    container_name: bonsai-app
    restart: unless-stopped
    ports:
      - "3000:3000"
    environment:
      # コンテナ間の通信では、サービス名がホスト名になる
      # "postgres" はサービス名 = ホスト名
      - DATABASE_URL=postgres://postgres:postgres@postgres:5432/bonsai_sns
      - NEXTAUTH_URL=http://localhost:3000
      - NEXTAUTH_SECRET=docker-dev-secret-change-in-production
      - NEXT_PUBLIC_APP_URL=http://localhost:3000
      - STORAGE_PROVIDER=local
      - EMAIL_PROVIDER=console
    volumes:

```



```

# ソースコードをマウント（ホトリロード用）
- ./app
- /app/node_modules
- /app/.next
depends_on:
  postgres:
    condition: service_healthy # postgresのヘルスチェックが通ってから起動
profiles:
  - dev # --profile dev 指定時のみ起動

# === アプリケーション（本番モード） ===
# profiles: [prod] → docker compose --profile prod up で起動
app-prod:
  build:
    context: .
    dockerfile: Dockerfile # 本番用Dockerfile（マルチステージビルド）
  container_name: bonsai-app-prod
  restart: unless-stopped
  ports:
    - "3000:3000"
  environment:
    - DATABASE_URL=postgresql://postgres:postgres@postgres:5432/bonsai_sns
    - NEXTAUTH_URL=http://localhost:3000
    - NEXTAUTH_SECRET=docker-dev-secret-change-in-production
    - NEXT_PUBLIC_APP_URL=http://localhost:3000
  depends_on:
    postgres:
      condition: service_healthy
  profiles:
    - prod # --profile prod 指定時のみ起動

# 名前付きボリュームの定義
# Docker管理下でデータを永続化する
volumes:
  postgres_data:

```

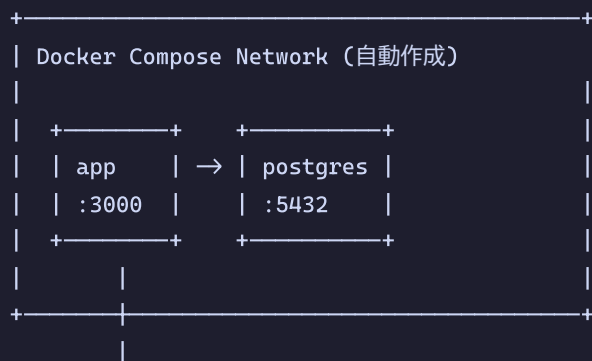
ポイント: profiles（プロファイル）とは

`profiles` を指定したサービスは、`--profile` フラグを付けないと起動しません。postgresサービスにはprofilesが設定されていないため、どのモードでも常に起動されます。

- `docker compose up -d postgres` -- PostgreSQLのみ起動（ローカルでNext.jsを直接起動する場合）
- `docker compose --profile dev up -d` -- PostgreSQL + 開発モードアプリ起動
- `docker compose --profile prod up -d` -- PostgreSQL + 本番モードアプリ起動

コンテナ間のネットワーク:

Docker Compose のネットワーク:



ホストPC:3000 でアクセス
http://localhost:3000

ポイント: Docker Compose内のコンテナ同士は、サービス名 (`postgres`) をホスト名として通信できます。外からのアクセスは `localhost:3000` です。

22.5.6 ビルド・実行方法

```
# === 方法1: docker build + docker run ===

# Dockerイメージをビルド（-t: イメージに名前をつける）
docker build -t bonsai-sns .

# コンテナを起動
# -p 3000:3000: ポートマッピング（ホスト:コンテナ）
# -e: 環境変数を設定
docker run -p 3000:3000 \
  -e DATABASE_URL="postgresql:// ... " \
  -e NEXTAUTH_SECRET="your-secret" \
  bonsai-sns

# === 方法2: docker compose（推奨） ===

# PostgreSQLのみ起動（ローカルでNext.jsを起動する場合）
docker compose up -d postgres

# 開発モードで全サービスを起動
docker compose --profile dev up -d

# 本番モードで全サービスを起動（ビルドテスト用）
docker compose --profile prod up -d

# ログを確認（リアルタイムで表示）
docker compose logs -f

# サービスの状態を確認
docker compose ps

# 全サービスを停止
docker compose down

# 全サービスを停止 + データも削除
docker compose down -v
```

よくあるトラブルと解決法

トラブル	原因	解決法
ビルドが遅い	キャッシュが効いていない	<code>package.json</code> を先にコピーする順序を確認
イメージが大きい	マルチステージビルドになっていない	4ステージ構成を確認
<code>standalone</code> ディレクトリがない	<code>next.config.ts</code> に <code>output: 'standalone'</code> がない	設定を追加
コンテナ間で通信できない	ホスト名が間違っている	サービス名（ <code>postgres</code> 等）を使用
ポートが使用中	他のプロセスが使っている	<code>docker compose down</code> または別のポートを使用
<code>permission denied</code>	ファイル権限の問題	Dockerfileで <code>USER nextjs</code> の前にファイルをコピー
appが起動しない	profilesを指定していない	<code>docker compose --profile dev up -d</code> を使用

▶ 理解度チェック（クリックして展開）

22.6 Sentry（エラー監視）

▶ このセクションで学ぶこと（クリックして展開）

BON-LOGでの使用箇所: `sentry.client.config.ts`（クライアントサイドエラー捕捉）、`sentry.server.config.ts`（サーバーサイドエラー捕捉）、`sentry.edge.config.ts`（Edgeランタイムエラー捕捉）として実装されています。`next.config.ts` で `withSentryConfig()` ラッパーが適用されており、ビルド時にソースマップが自動アップロードされます。環境変数 `SENTRY_DSN` と `NEXT_PUBLIC_SENTRY_DSN` の設定が必要です。

実装しない場合の影響: Sentryがないと、本番環境でのエラーをリアルタイムに検知できません。ユーザーからの報告に頼った「受け身のバグ対応」になります。`withSentryConfig()` を除去する場合は `next.config.ts` を元の `export default nextConfig` に戻す必要があります。Sentryは無料プランで月間5,000イベントまで利用できます。

22.6.1 なぜエラー監視が必要なのか

本番環境では、開発中には起きなかったエラーが発生します。ユーザーが報告してくれるとは限らないため、エラーを自動検知する仕組みが必要です。

たとえ話: Sentryは「ビルの警備システム」のようなものです。異常（エラー）が起きたら自動的に検知し、管理人（開発者）に通知します。どこで何が起きたのか、詳細な記録（ログ）も残します。

エラー監視なしの場合:

ユーザー: "画面が真っ白 ... "

|

v

ユーザーが離脱
(開発者は知らない)

エラー監視ありの場合:

ユーザー: "画面が真っ白 ... "

|

v

Sentry がエラーを即座に検知

|

v

開発者にSlack/メール通知
"PostPage でTypeError発生"
"該当コード: line 42"
"ブラウザ: Chrome 120"
"発生回数: 150回/1時間"

|

v

迅速に修正・デプロイ

22.6.2 Sentryのセットアップ

ステップ1: パッケージのインストール

```
# @sentry/nextjs: Next.js専用のSentryパッケージ
npm install @sentry/nextjs

# セットアップウィザードを実行（対話形式で設定ファイルを自動生成）
# -i nextjs: Next.jsプロジェクト用の設定
npx @sentry/wizard@latest -i nextjs
```

ウィザードを実行すると、以下のファイルが自動生成されます:

- `sentry.client.config.ts` - ブラウザ側のエラー捕捉設定
- `sentry.server.config.ts` - サーバー側のエラー捕捉設定
- `sentry.edge.config.ts` - エッジランタイムのエラー捕捉設定

ステップ2: Sentryアカウントでプロジェクトを作成

1. [Sentry](#)にアカウント登録
2. 「Create Project」 → 「Next.js」を選択
3. DSN（Data Source Name）が発行される

DSNとは: Sentryがエラーデータの送信先を特定するためのURLです。プロジェクトごとにユニークなDSNが発行されます。

22.6.3 クライアント側の設定

```
// sentry.client.config.ts
// ブラウザ（クライアント）側で発生するエラーを捕捉する設定

import * as Sentry from '@sentry/nextjs'

Sentry.init({
  // DSN: Sentryへのエラーデータ送信先URL
  // NEXT_PUBLIC_ プレフィックス付き = ブラウザでも使用可能
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,

  // tracesSampleRate: パフォーマンスデータの収集割合
  // 0.1 = 10%のリクエストでパフォーマンスデータを収集
  // 1.0 にすると100%収集するが、コストが増加する
  tracesSampleRate: 0.1,

  // environment: 環境の識別 ('development' or 'production')
  // Sentryダッシュボードで環境ごとにフィルタリングできる
  environment: process.env.NODE_ENV,

  // enabled: 本番環境のみで有効化
  // 開発中のエラーはSentryに送信しない（ノイズになるため）
  enabled: process.env.NODE_ENV === 'production',
})
```

22.6.4 サーバー側の設定

```
// sentry.server.config.ts
// サーバー（API Route、Server Component等）で発生するエラーを捕捉する設定

import * as Sentry from '@sentry/nextjs'

Sentry.init({
  // サーバー側のDSN（NEXT_PUBLIC_ なし = サーバー専用）
  dsn: process.env.SENTRY_DSN,

  // パフォーマンスデータの収集割合（10%）
  tracesSampleRate: 0.1,

  // 環境の識別
  environment: process.env.NODE_ENV,

  // 本番環境のみで有効化
  enabled: process.env.NODE_ENV === 'production',
})
```

`SENTRY_DSN` と `NEXT_PUBLIC_SENTRY_DSN` の違い:

- `SENTRY_DSN`: サーバー側でのみ使用。ブラウザには公開されない
- `NEXT_PUBLIC_SENTRY_DSN`: ブラウザでも使用。公開されるが、DSNは機密情報ではないため問題ない

22.6.5 エラーのキャプチャ

Server Actionsやapi routeでエラーを手動で捕捉・送信する方法です。


```
// lib/actions/post.ts
'use server'

// Sentryのインポート
import * as Sentry from '@sentry/nextjs'

export async function createPost(data: any) {
  try {
    // 投稿作成処理
    // ... (正常時のコード)
  } catch (error) {
    // エラーが発生したらSentryに送信
    Sentry.captureException(error, {
      // tags: エラーの分類に使うラベル
      // Sentryダッシュボードでフィルタリングに使える
      tags: { action: 'createPost' },

      // user: エラーが発生したユーザーの情報
      // 「特定のユーザーだけで発生する」問題の調査に役立つ
      user: { id: session.user.id },
    })

    // ユーザーには一般的なエラーメッセージを返す
    // （詳細なエラー内容はセキュリティ上、ユーザーに見せない）
    return { error: 'エラーが発生しました' }
  }
}
```

22.6.6 パフォーマンス監視

Sentryはエラーだけでなく、パフォーマンス（処理時間）も監視できます。

```
import * as Sentry from '@sentry/nextjs'

export async function getPostsWithTracing() {
  // startSpan: パフォーマンス計測区間を定義
  return await Sentry.startSpan(
    {
      // name: Sentryダッシュボードに表示される名前
      name: 'getPosts',
      // op: 操作の種類 ('db.query', 'http.request' 等)
      op: 'db.query',
    },
    async () => {
      // この中の処理時間が計測される
      return await prisma.post.findMany()
    }
  )
}
```

Sentryダッシュボードで確認できる情報:

```
+-----+
| Sentry ダッシュボード |
+-----+
|
| Issues (エラー一覧) |
| +-----+ |
| | TypeError: Cannot read property 'id' | |
| | 発生回数: 342回 最初: 2時間前 最後: 3分前 | |
| | ファイル: app/(main)/posts/[id]/page.tsx:42 | |
| | ブラウザ: Chrome 120 (85%), Safari 17 (15%) | |
| | OS: Windows (60%), macOS (30%), iOS (10%) | |
| +-----+ |
|
| Performance (パフォーマンス) |
| +-----+ |
| | getPosts 平均: 120ms P95: 350ms | |
| | getUser 平均: 45ms P95: 150ms | |
| | createPost 平均: 200ms P95: 500ms | |
| +-----+ |
|
+-----+
```

よくあるトラブルと解決法

トラブル	原因	解決法
エラーがSentryに表示されない	DSNが未設定	環境変数 <code>SENTRY_DSN</code> を確認
開発中のエラーが送信される	<code>enabled</code> 設定の問題	<code>enabled: process.env.NODE_ENV === 'production'</code> を確認
パフォーマンスデータが少ない	サンプリング率が低い	<code>tracesSampleRate</code> を上げる（コストに注意）
ソースマップが表示されない	ソースマップのアップロード未設定	<code>withSentryConfig</code> でソースマップ送信を設定

▶ 理解度チェック（クリックして展開）

22.7 Sentry詳細設定（クライアント/サーバー/Edge）

▶ このセクションで学ぶこと（クリックして展開）

22.7.1 Sentryの3つの設定ファイル

Next.jsアプリケーションは3つの異なるランタイムで動作します。それぞれに対応するSentry設定ファイルがあります。

ランタイム	設定ファイル	捕捉するエラー	具体例
ブラウザ	sentry.client.config.ts	ユーザーのブラウザで動作するJSのエラー	クリックイベント、API呼び出し、描画エラー等
Node.jsサーバー	sentry.server.config.ts	Server Components、API Routes、Server Actionsのエラー	DB接続エラー、認証エラー、バリデーションエラー等
Edgeランタイム	sentry.edge.config.ts	Middleware等のEdge Functionsのエラー	認証リダイレクト、ヘッダー操作のエラー等

なぜ3つ必要なのか？

Next.jsは1つのアプリケーション内で複数のランタイムを使い分けます。ブラウザのJavaScriptエンジン、サーバーのNode.js、そしてCDNエッジで動作するEdgeランタイムはそれぞれ異なる実行環境です。使えるAPIやキャプチャすべきエラーの種類が異なるため、設定を分けています。

22.7.2 クライアントサイド設定（sentry.client.config.ts）

ブラウザで発生するエラーを捕捉する設定ファイルです。BON-LOGでは以下のように構成されています。

```

// sentry.client.config.ts
import * as Sentry from '@sentry/nextjs'

// グローバルにSentryを公開（デバッグ・テスト用）
if (typeof window !== 'undefined') {
  (window as typeof window & { Sentry: typeof Sentry }).Sentry = Sentry
}

Sentry.init({
  // Sentry プロジェクトのDSN（Data Source Name）
  // 環境変数 NEXT_PUBLIC_ 付きでブラウザに公開される
  dsn: process.env.NEXT_PUBLIC_SENTRY_DSN,

  // パフォーマンスモニタリングのサンプリングレート
  // 本番: 10%のリクエストのみ計測（コスト節約）
  // 開発: 100%計測
  tracesSampleRate: process.env.NODE_ENV === 'production' ? 0.1 : 1.0,

  // 本番環境でのみ有効化
  enabled: process.env.NODE_ENV === 'production',

  // デバッグモード（通常はfalse）
  debug: false,

  // エラーフィルタリング
  beforeSend(event, hint) {
    // 開発環境ではエラーを送信しない
    if (process.env.NODE_ENV !== 'production') {
      return null
    }

    // ChunkLoadError（コードスプリットの読み込みエラー）を無視
    // ユーザーがデプロイ中にアクセスした場合に発生する
    if (event.exception?.values?.[0]?.type === 'ChunkLoadError') {
      return null
    }

    // Twitterアプリ内ブラウザの独自JSエラーを除外
    const userAgent = typeof navigator !== 'undefined' ? navigator.userAgent : ''
    if (userAgent.includes('Twitter')) {
      const errorMessage = event.exception?.values?.[0]?.value || ''
      if (
        errorMessage.includes("Can't find variable: CONFIG") ||
        errorMessage.includes('updateGapFiller')
      ) {
        return null
      }
    }
  },

  return event
},

```

```
// パターンマッチで無視するエラー
ignoreErrors: [
  // ブラウザ拡張機能由来のエラー
  'top.GLOBALS',
  /extensions\\//i,
  /^chrome:\\//i,
  /^chrome-extension:\\//i,
  // ネットワーク関連（一時的な接続エラー）
  'Network request failed',
  'Failed to fetch',
  'NetworkError',
  'AbortError',
  // レイアウト監視の非致命的エラー
  'ResizeObserver loop',
  'Non-Error promise rejection',
  // SNSアプリ内ブラウザ由来のエラー
  /Can't find variable: CONFIG/,
  /updateGapFiller/,
  /updateFooterPositions/,
],
})
```

重要な設定項目の解説

設定項目	説明
<code>dsn</code>	Sentryプロジェクトへの接続先。 <code>NEXT_PUBLIC_</code> 付きでブラウザに公開される
<code>tracesSampleRate</code>	パフォーマンス計測するリクエストの割合。0.1 = 10%
<code>enabled</code>	<code>false</code> にするとSentryが完全に無効化される
<code>beforeSend</code>	各エラーイベントの送信前に呼ばれるフック。 <code>null</code> を返すと送信しない
<code>ignoreErrors</code>	文字列・正規表現で指定したパターンに一致するエラーを自動的に無視

`beforeSend`でのエラーフィルタリングが重要な理由

本番環境では、アプリのバグではないエラーが大量に発生します。ブラウザ拡張機能が注入するスクリプトのエラー、SNSアプリ内ブラウザの独自JavaScriptエラー、一時的なネットワーク断絶など、開発者が対処不可能なエラーです。これらをフィルタリングしないと、本当に修正が必要なエラーがノイズに埋もれてしまいます。

22.7.3 サーバーサイド設定 (sentry.server.config.ts)

Node.jsサーバーで発生するエラーを捕捉する設定ファイルです。

```
// sentry.server.config.ts
import * as Sentry from '@sentry/nextjs'

Sentry.init({
  // サーバーサイドでは NEXT_PUBLIC_ なしの SENTRY_DSN を使用
  dsn: process.env.SENTRY_DSN,

  // パフォーマンストレーシングのサンプリングレート
  tracesSampleRate: process.env.NODE_ENV === 'production' ? 0.1 : 1.0,

  // 本番環境でのみ有効化
  enabled: process.env.NODE_ENV === 'production',

  debug: false,

  // サーバーサイド特有のエラーフィルタリング
  beforeSend(event) {
    if (process.env.NODE_ENV !== 'production') {
      return null
    }
    return event
  },

  // サーバーサイドで無視するエラー
  ignoreErrors: [
    // Next.jsの正常なリダイレクト・404（エラーではない）
    'NEXT_REDIRECT',
    'NEXT_NOT_FOUND',
    // DB接続の一時的なエラー（自動リトライで回復する）
    'Connection pool timeout',
    'Connection terminated unexpectedly',
  ],
})
```

クライアントとサーバーの設定の違い

項目	クライアント	サーバー
DSN環境変数	NEXT_PUBLIC_SENTRY_DSN	SENTRY_DSN
ブラウザ拡張エラー	フィルタリング対象	発生しない
NEXT_REDIRECT	発生しない	無視対象（正常動作）
DB接続エラー	発生しない	無視対象（一時的）
UserAgent判定	必要	不要

22.7.4 Edgeランタイム設定（sentry.edge.config.ts）

Middleware等のEdgeランタイムで発生するエラーを捕捉します。Edge環境はNode.jsのフルAPIが使えない制限された環境のため、設定はシンプルです。

```
// sentry.edge.config.ts
import * as Sentry from '@sentry/nextjs'

Sentry.init({
  dsn: process.env.SENTRY_DSN,

  // パフォーマンストレーシングのサンプリングレート
  tracesSampleRate: process.env.NODE_ENV === 'production' ? 0.1 : 1.0,

  // 本番環境でのみ有効化
  enabled: process.env.NODE_ENV === 'production',

  debug: false,
})
```

22.7.5 instrumentation.ts（サーバー初期化処理）

`instrumentation.ts` はNext.jsの特殊なファイルで、サーバーサイドの初期化処理を一元管理します。Sentryの初期化やセキュリティチェックなど、アプリケーション起動時に1回だけ実行すべき処理をここに記述します。

たとえば理解する: レストランの開店準備

`instrumentation.ts` は「レストランの開店準備マニュアル」のようなものです：

【開店前の準備】

1. 厨房の火を点ける → Sentryサーバー設定の読み込み
2. 監視カメラを起動する → エラー監視の開始
3. 食材の品質チェック → セキュリティチェックの実行
4. ホールの照明を点ける → Edge設定の読み込み

これらは「お客さんが来る前に1回だけ」行う準備です。

リクエストごとに繰り返す必要はありません。

BON-LOGの `instrumentation.ts` の完全なソースコードを1行ずつ解説します。

```
// instrumentation.ts
// ファイルの場所: プロジェクトルート直下 (app/ではなくルート)

/**
 * Next.js Instrumentation
 *
 * サーバーサイドの初期化処理を行います。
 * Sentryの初期化やその他のサーバーサイドツールの設定に使用します。
 *
 * @see https://nextjs.org/docs/app/building-your-application/optimizing/instrumentation
 */

// register関数: Next.jsが起動時に自動的に呼び出す特殊な関数
// async関数なので、非同期のimport (動的インポート) が使える
export async function register() {

  // NEXT_RUNTIME環境変数でどのランタイムで実行されているかを判定
  // 'nodejs' = 通常のNode.jsサーバー (Server Components、API Routes等)
  if (process.env.NEXT_RUNTIME === 'nodejs') {

    // サーバーサイドのSentry設定を動的にインポート
    // 動的インポート (await import) を使う理由:
    // → Node.js専用のモジュールをEdgeランタイムで読み込まないようにするため
    // → Edgeランタイムにはfs、net等のNode.jsモジュールが存在しない
    await import('./sentry.server.config')

    // セキュリティチェックを実行 (Node.jsランタイムのみ)
    // → 環境変数の検証、シークレットの強度チェック等
    const { enforceSecurityInProduction } = await import('./lib/security-checks')
    enforceSecurityInProduction()
  }

  // 'edge' = Edgeランタイム (Middleware等)
  if (process.env.NEXT_RUNTIME === 'edge') {

    // Edge用のSentry設定を動的にインポート
    // Edge環境ではNode.jsのAPIが使えないため、別の設定ファイルを使用
    await import('./sentry.edge.config')
  }
}

// onRequestError: リクエスト処理中にエラーが発生した時に呼ばれるコールバック
// Next.js 15+で追加された機能
// Server Components、Route Handlers、Server Actionsのエラーを一括で捕捉できる
export const onRequestError = async (
  // err: 発生したエラーオブジェクト
  // digest: Next.jsがエラーに付与する一意のID (クライアントに安全に渡せる)
  err: { digest: string } & Error,

  // request: エラーが発生したリクエストの情報
  request: {
```

```

    path: string // リクエストパス (例: '/api/posts')
    method: string // HTTPメソッド (例: 'GET', 'POST')
    headers: { [key: string]: string } // リクエストヘッダー
  },

  // context: エラーが発生したコンテキスト (どこで起きたか)
  context: {
    routerKind: 'Pages Router' | 'App Router' // ルーターの種類
    routePath: string // ルートパス (例: '/posts/[id]')
    routeType: 'render' | 'route' | 'action' | 'middleware' // ルートの種類
    renderSource: // レンダリングのソース
      | 'react-server-components'
      | 'react-server-components-payload'
      | 'server-rendering'
    revalidateReason: 'on-demand' | 'stale' | undefined // 再検証の理由
    renderType: 'dynamic' | 'dynamic-resume' // レンダリングタイプ
  }
) => {
  // Sentryにエラーを報告
  // ここでも動的インポートを使用 (ランタイムに関係なく呼ばれるため)
  const Sentry = await import('@sentry/nextjs')

  // captureException: Sentryにエラーを送信するメソッド
  // extra: エラーと一緒に送信する追加情報
  // → Sentryダッシュボードでエラーの発生状況を詳しく確認できる
  Sentry.captureException(err, {
    extra: {
      path: request.path, // どのパスでエラーが起きたか
      method: request.method, // どのHTTPメソッドで起きたか
      routePath: context.routePath, // どのルートで起きたか
      routeType: context.routeType, // ルートの種類 (render/route/action/middleware)
      routerKind: context.routerKind, // App Router/Pages Router
      renderSource: context.renderSource, // レンダリングソース
    },
  })
}

```

register 関数の実行タイミング

Next.jsサーバー起動のタイムライン:

```

[1] サーバードプロセス起動
  |
[2] register() 関数が呼ばれる ← ここで初期化
  |
  +-- NEXT_RUNTIME === 'nodejs' の場合:
  |   - sentry.server.config.ts を読み込み
  |   - セキュリティチェックを実行
  |
  +-- NEXT_RUNTIME === 'edge' の場合:
  |   - sentry.edge.config.ts を読み込み
  |
[3] リクエストの受付開始
  |
[4] エラー発生時 → onRequestError() が呼ばれる

```

onRequestError で送信される追加情報の活用例

Sentryダッシュボードでは、`extra` に含めた情報を使ってエラーの原因を素早く特定できます。

Sentryダッシュボードでの表示例:

Error: Cannot read properties of undefined (reading 'id')

追加情報	値	備考
path	/api/posts/abc123	
method	GET	
routePath	/posts/[id]	
routeType	route	API Routeでエラーが発生
routerKind	App Router	
renderSource	react-server-components	

→ 「/posts/[id]のRoute Handlerで、idパラメータの処理に問題がある」と即座に判断できる

なぜ動的インポート (`await import`) を使うのか？

通常のimport (トップレベル) :

```
import './sentry.server.config' // ← ファイル読み込み時に即実行される
→ Edge環境でもNode.js用モジュールが読み込まれてしまう
→ Edge環境にfsモジュールが存在しないためエラーになる
```

動的インポート:

```
if (process.env.NEXT_RUNTIME === 'nodejs') {
  await import('./sentry.server.config') // ← 条件が真の時だけ実行
}
→ Node.js環境の時だけNode.js用モジュールを読み込む
→ Edge環境ではこのブロックに入らないのでエラーにならない
```

22.7.6 ソースマップの設定

ソースマップとは、ビルド (minify・バンドル) 後のJavaScriptコードと元のTypeScriptソースコードを紐付けるファイルです。ソースマップがなければ、エラーの発生箇所が「bundle.js:1:23456」のように不明瞭になります。

ソースマップの仕組み:

[元のTypeScript]	[ビルド後のJS]
components/post/LikeButton.tsx	.next/static/chunks/app-xxx.js
42行目: await likePost(id)	1行目: ... a.LikePost(n) ...
^^^^^	^^^^^
+----- ソースマップで紐付け -----+	

Sentryのエラー表示:

- ✖ ソースマップなし: app-xxx.js:1:23456
- ✔ ソースマップあり: components/post/LikeButton.tsx:42 (await likePost(id))

Next.jsとSentryの連携では、`@sentry/nextjs` パッケージがビルド時に自動的にソースマップをSentryにアップロードします。`next.config.ts` で `withSentryConfig` ラッパーを使用することで設定されます。

```
// next.config.ts の Sentry設定部分 (イメージ)
import { withSentryConfig } from '@sentry/nextjs'

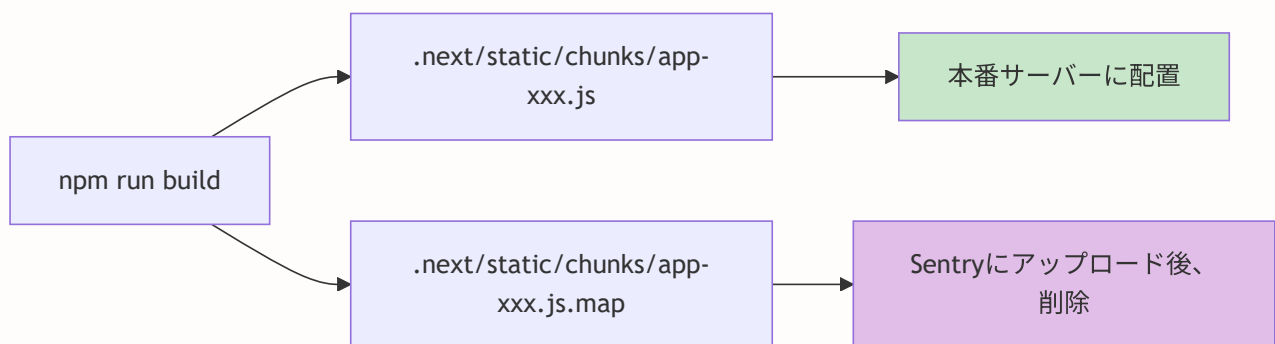
const nextConfig = {
  // ... 通常のNext.js設定
}

export default withSentryConfig(nextConfig, {
  // ビルド時にソースマップをSentryに自動アップロード
  silent: true, // アップロードログを非表示
  widenClientFileUpload: true, // クライアントファイルを広くアップロード
  hideSourceMaps: true, // 本番環境でソースマップを公開しない (セキュリティ)
})
```

ソースマップのセキュリティに関する注意

ソースマップの扱い方:

- ❌ **危険:** ソースマップを本番サーバーに公開する
 - 攻撃者がソースコードを読んで脆弱性を発見できてしまう
 - 認証ロジック、バリデーションの抜け穴を見つけられる
- ✅ **安全:** Sentryにだけアップロードし、本番サーバーには公開しない
 - Sentryは認証付きのダッシュボードなので、チームメンバーだけが閲覧
 - `hideSourceMaps: true` で本番サーバーからは非公開



ソースマップが無い場合と有る場合の比較

実際にSentryダッシュボードで見るエラーの違いを確認しましょう。

[ソースマップなし - 何が起きたのか分からない]

```
Error: Cannot read properties of undefined (reading 'id')
at Object.n [as likePost] (app-abc123.js:1:45678)
at Object.o (framework-xyz789.js:1:12345)
at e.dispatch (vendor-qwe456.js:1:67890)
```

- ファイル名が暗号のようで、何行目かも分からない
- デバッグに何時間もかかる

[ソースマップあり - 原因が一目瞭然]

```
Error: Cannot read properties of undefined (reading 'id')
at likePost (lib/actions/post.ts:42:15)
  |
  | 40 | export async function likePost(postId: string) {
  | 41 |   const session = await auth()
> | 42 |   const userId = session.user.id // ← session.userがundefined
  | 43 |   await prisma.like.create({
  |
at handleLike (components/post/LikeButton.tsx:18:5)
at onClick (components/post/LikeButton.tsx:28:20)
```

- 「lib/actions/post.ts の42行目で session.user が undefined」
- 「認証されていないユーザーがいいねボタンを押した」と即座に分かる

ソースマップの仕組み（内部構造）

ソースマップファイル（`.map`）の中身は以下のようなJSON構造です。

```
// app-abc123.js.map (実際のソースマップファイルの内容)
{
  "version": 3, // ソースマップ仕様のバージョン
  "file": "app-abc123.js", // ビルド後のファイル名
  "sourceRoot": "", // ソースのルートパス
  "sources": [ // 元のソースファイル一覧
    "components/post/LikeButton.tsx",
    "lib/actions/post.ts",
    "lib/db.ts"
  ],
  "names": [ // 元の変数名・関数名
    "likePost", "session", "userId", "prisma"
  ],
  "mappings": "AAAA,IAAM,KAAM, ..." // 位置の対応情報 (Base64 VLQエンコード)
}
```

`mappings` フィールドにはBase64 VLQエンコードされた位置情報が含まれており、ビルド後のコードの各文字が元のソースコードのどの位置に対応するかを記録しています。Sentryはこの情報を使って、エラーのスタックトレースを元のソースコードに変換します。

22.7.7 パフォーマンスモニタリング

`tracesSampleRate` で設定するパフォーマンスモニタリングは、リクエストの処理時間やデータベースクエリの実行時間を自動的に計測します。

パフォーマンスモニタリングで可視化される情報:

```
<div class="mermaid-rendered">
<img src="data:image/svg+xml;base64,PHN2ZyBpZD0ibW1kXzIyX2RlcGxveV8x0CIgd2lkdgGg9IjEwMCUiIHhtbG
</div>
```

ボトルネック: DB Query (`posts.findMany`) が全体の51%
→ インデックスの追加やクエリ最適化が必要

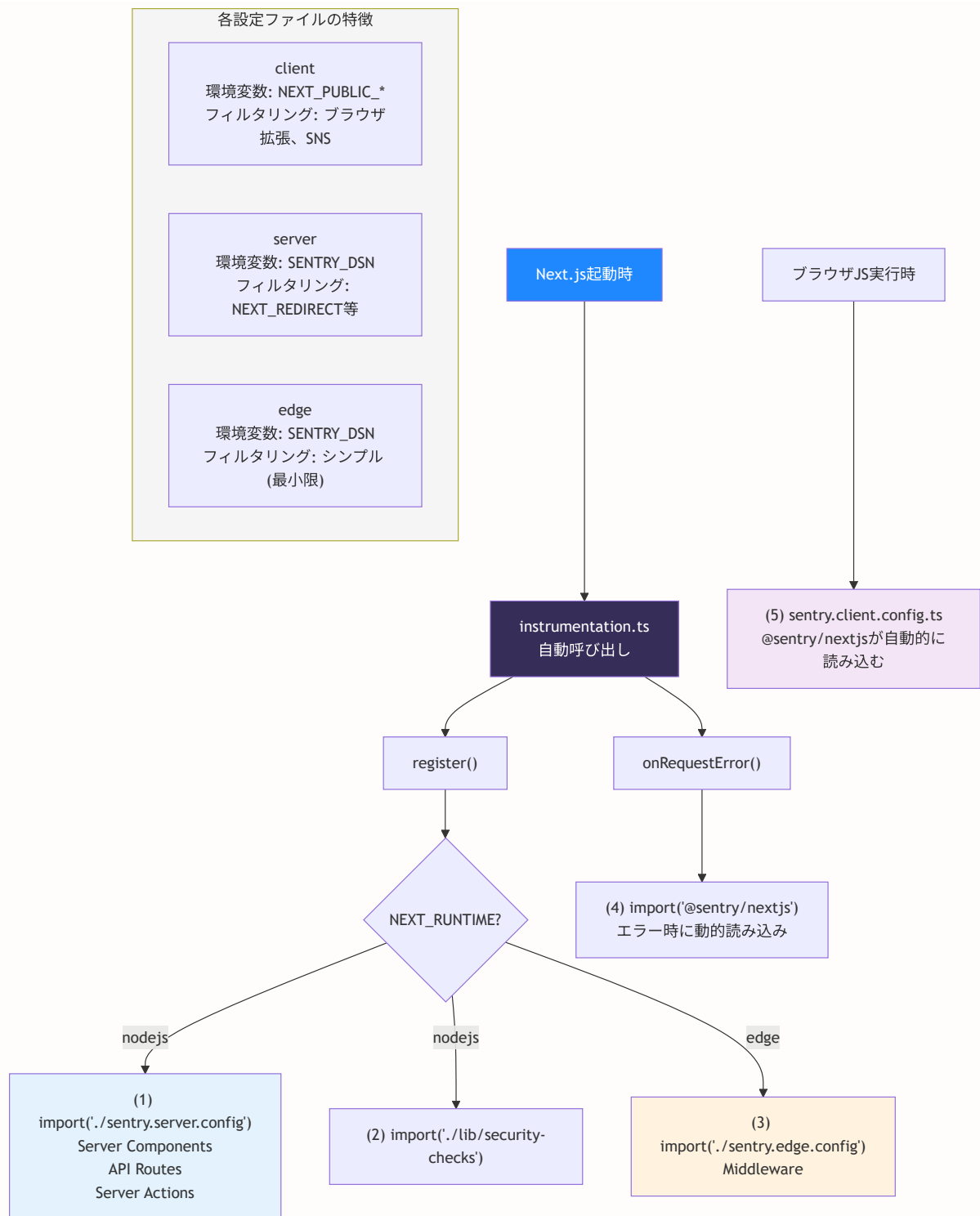
サンプリングレートの選び方

レート	用途	コスト
1.0 (100%)	開発環境、トラフィックが少ないサービス	高
0.1 (10%)	一般的な本番環境	中
0.01 (1%)	高トラフィックサービス	低

BON-LOGでは本番環境で `0.1` (10%) に設定しています。これはSentryの無料プラン枠内に収まる量であり、かつ統計的に十分なサンプル数を確保できるバランスです。

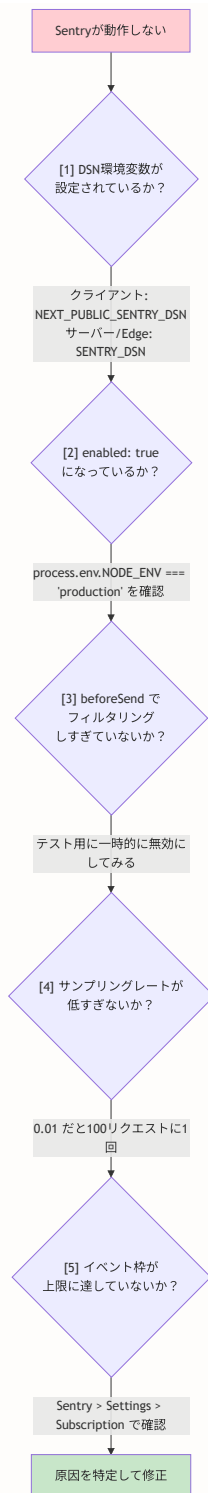
22.7.8 Sentry設定の全体像

3つの設定ファイルと `instrumentation.ts` の関係を整理します。



22.7.9 Sentryのトラブルシューティング

本番環境でSentryが正しく動作しない場合のチェックリストです。



テスト用エラーの送信方法

```
// 本番環境でSentryの動作確認をする場合
// app/api/test-sentry/route.ts (テスト後に削除すること！)
import * as Sentry from '@sentry/nextjs'

export async function GET() {
  try {
    // 意図的にエラーを発生させる
    throw new Error('Sentry Test Error - Delete this endpoint after testing')
  } catch (error) {
    // Sentryにエラーを手動で送信
    Sentry.captureException(error)
    return new Response('Sentry test error sent', { status: 200 })
  }
}
```

▶ 理解度チェック（クリックして展開）

22.8 PWA対応（Service Worker・オフライン）

▶ このセクションで学ぶこと（クリックして展開）

22.8.1 PWAとは

PWA（Progressive Web App）は、ウェブアプリをネイティブアプリに近い体験で提供する技術です。BON-LOGでは以下のPWA機能を実装しています。

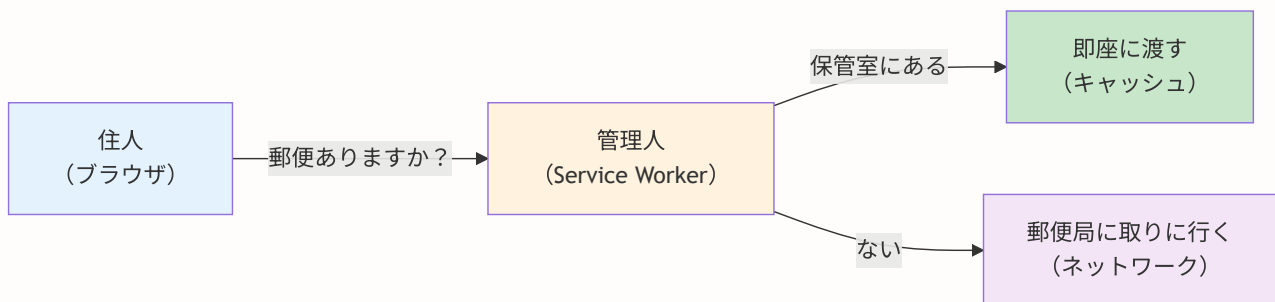
PWA機能	詳細
Service Worker	
	オフラインでもキャッシュ済みページを表示
	バックグラウンドでのリソース更新
	プッシュ通知の受信（将来機能）
キャッシュ戦略	
	静的アセット: Cache First（高速表示）
	APIレスポンス: Network First（最新データ優先）
	画像: Stale While Revalidate（即座表示 + バックグラウンド更新）
アプリ更新	
	新バージョン検出時にユーザーに通知
	「今すぐ更新」ボタンでService Workerを切り替え

22.8.2 Service Workerの登録（ServiceWorkerRegistration.tsx）

Service Workerは、ブラウザのバックグラウンドで動作するスクリプトです。BON-LOGでは

`ServiceWorkerRegistration` コンポーネントで登録を管理しています。

たとえば理解する: マンションの管理人



Service Worker = マンションの管理人のようなもの

- 住人が寝ていても働いている（バックグラウンド動作）
- 配達物を保管室に整理する（キャッシュ管理）
- 停電でも保管室の荷物は渡せる（オフライン対応）
- 管理人の交代には手続きが必要（アップデート管理）

BON-LOGの **ServiceWorkerRegistration** コンポーネントの完全なソースコードを1行ずつ解説します。

```
// components/pwa/ServiceWorkerRegistration.tsx
// このファイルの場所: components/pwa/ServiceWorkerRegistration.tsx

'use client'
// ↑ このコンポーネントはクライアントサイド（ブラウザ）で動作する
//   Service WorkerはブラウザAPIなのでサーバーサイドでは使えない

import { useEffect, useState, useRef, useCallback } from 'react'
// useEffect: コンポーネントのマウント時に副作用を実行
// useState: 状態管理（オンライン状態、更新プロンプト表示等）
// useRef: 値の保持（再レンダリングしても初期化されない）
// useCallback: 関数のメモ化（不要な再生成を防止）

export function ServiceWorkerRegistration() {
  // === 状態管理 ===

  // isOnline: インターネット接続状態
  // 初期値はブラウザの navigator.onLine を使用
  // SSR時（windowが存在しない場合）はtrue（オンライン）として初期化
  const [isOnline, setIsOnline] = useState(() => {
    if (typeof window === 'undefined') return true // SSR対策
    return navigator.onLine // ブラウザの接続状態
  })

  // showUpdatePrompt: 更新プロンプト（「アップデートがあります」ダイアログ）を表示するか
  const [showUpdatePrompt, setShowUpdatePrompt] = useState(false)

  // waitingWorker: アップデート待機中のService Worker
  // ユーザーが「今すぐ更新」を押した時に、このWorkerをアクティブにする
  const [waitingWorker, setWaitingWorker] = useState<ServiceWorker | null>(null)

  // isInitialized: 初期化済みフラグ（React Strict Modeでの二重実行防止）
  // useRefを使う理由: stateと違い変更しても再レンダリングが発生しない
  const isInitialized = useRef(false)

  // === Service Worker登録処理 ===
  // useCallbackでメモ化: 毎回新しい関数が作られるのを防止
  // → useEffectの依存配列に含めても無限ループにならない
  const registerSW = useCallback(async () => {
    try {
      // Service Workerの登録
      // '/sw.js': publicディレクトリにあるService Workerスクリプト
      // scope: '/': サイト全体をService Workerの管理対象にする
      const registration = await navigator.serviceWorker.register('/sw.js', {
        scope: '/',
      })
      console.log('[SW] Service Worker registered:', registration.scope)
    }
  }, [])

  // === 更新の監視 ===
  // 'updatefound'イベント: 新しいバージョンのService Workerが見つかった時に発火
  registration.addEventListener('updatefound', () => {

```

```

// registration.installing: インストール中の新しいService Worker
const newWorker = registration.installing

if (newWorker) {
  // 新しいWorkerの状態変化を監視
  newWorker.addEventListener('statechange', () => {
    // 条件: 新しいWorkerがインストール完了 && 既存のWorkerがアクティブ
    // この条件は「アップデートがある」ことを意味する
    // 初回インストール時は controller が null なので条件を満たさない
    if (newWorker.state === 'installed' && navigator.serviceWorker.controller) {
      console.log('[SW] New version available')
      setWaitingWorker(newWorker) // 待機中のWorkerを保存
      setShowUpdatePrompt(true)  // 更新プロンプトを表示
    }
  })
}

// 既にインストール済みの待機Workerがある場合
// (前回のアクセス時にインストールされたが、アクティブ化されていない場合)
if (registration.waiting) {
  setWaitingWorker(registration.waiting)
  setShowUpdatePrompt(true)
}

// 定期的に更新を確認 (1時間 = 60分 x 60秒 x 1000ミリ秒)
// ユーザーがページを開きっぱなしにしている場合でも更新を検出できる
const intervalId = setInterval(() => {
  registration.update() // サーバーにsw.jsの更新を問い合わせ
}, 60 * 60 * 1000)

return () => clearInterval(intervalId) // クリーンアップ
} catch (error) {
  console.error('[SW] Registration failed:', error)
}
}, [])

// === 初期化処理 (コンポーネントマウント時に1回だけ実行) ===
useEffect(() => {
  // 二重初期化を防止
  // React Strict Modeでは開発時にuseEffectが2回実行されるため
  if (isInitialized.current) return
  isInitialized.current = true

  // ブラウザがService Workerをサポートしているか確認
  // SSR時やIE等の古いブラウザではService WorkerのAPIが存在しない
  if (typeof window === 'undefined' || !('serviceWorker' in navigator)) {
    return
  }

  // 開発環境ではService Workerを無効化
  // 理由: Next.jsのHMR (Hot Module Replacement) とService Workerのキャッシュが競合する
  // → コードを変更しても古いキャッシュが返されて変更が反映されない問題が起きる

```

```

if (process.env.NODE_ENV === 'development') {
  console.log('[SW] Service Worker disabled in development mode')
  return
}

// === オンライン/オフラインイベントの監視 ===
const handleOnline = () => {
  setIsOnline(true)
  console.log('[SW] Back online')
}
const handleOffline = () => {
  setIsOnline(false)
  console.log('[SW] Gone offline')
}

// ブラウザの 'online' / 'offline' イベントを購読
window.addEventListener('online', handleOnline)
window.addEventListener('offline', handleOffline)

// === Service Workerの登録タイミング ===
// ページのロードが完了してから登録することで、初期表示を妨げない
if (document.readyState === 'complete') {
  registerSW() // 既にロード完了済み
} else {
  window.addEventListener('load', registerSW) // ロード完了を待つ
}

// === コントローラー変更の監視 ===
// 新しいService Workerがアクティブになった時にページをリロード
let refreshing = false // 二重リロード防止フラグ
const handleControllerChange = () => {
  if (!refreshing) {
    refreshing = true
    console.log('[SW] Controller changed, reloading page')
    window.location.reload() // ページをリロードして新しいService Workerを適用
  }
}
navigator.serviceWorker.addEventListener('controllerchange', handleControllerChange)

// === クリーンアップ ===
// コンポーネントがアンマウントされた時にイベントリスナーを解除
// メモリリークを防止する
return () => {
  window.removeEventListener('online', handleOnline)
  window.removeEventListener('offline', handleOffline)
  navigator.serviceWorker.removeEventListener('controllerchange', handleControllerChange)
}
}, [registerSW])

// === 更新を適用する関数 ===
// ユーザーが「今すぐ更新」ボタンを押した時に呼ばれる
const applyUpdate = () => {
  if (waitingWorker) {

```



```

// 待機中のService Workerに 'SKIP_WAITING' メッセージを送信
// これにより、待機中のWorkerが即座にアクティブになる
// → controllerchangeイベントが発火 → ページがリロードされる
waitingWorker.postMessage({ type: 'SKIP_WAITING' })
setShowUpdatePrompt(false)
}
}

// ===== 更新を後で行う関数 =====
const dismissUpdate = () => {
  setShowUpdatePrompt(false)
  // 次回アクセス時、または1時間後の自動チェック時に再度プロンプトが表示される
}

// ===== UIレンダリング =====
// 条件に応じて3つのUIのいずれかを表示

// 1. 更新プロンプト（新バージョンが利用可能な場合）
if (showUpdatePrompt) {
  return (
    <div className="fixed bottom-4 left-4 right-4 sm:left-auto sm:right-4 sm:w-80 z-50
      bg-card border rounded-lg shadow-lg p-4 animate-in slide-in-from-bottom
      role="alert" aria-live="polite">
      {/* ... 更新プロンプトのUI ... */}
    </div>
  )
}

// 2. オフラインバナー（ネットワーク切断時）
if (!isOnline) {
  return (
    <div className="fixed top-0 left-0 right-0 z-50 bg-amber-500 text-amber-950
      px-4 py-2 text-center text-sm font-medium"
      role="alert" aria-live="assertive">
      オフラインです - 一部の機能が制限されます
    </div>
  )
}

// 3. 通常時は何も表示しない
return null
}

```

状態遷移図

ServiceWorkerRegistrationコンポーネントの表示状態:

【初期状態】

```
isOnline=true, showUpdatePrompt=false
→ return null (何も表示しない)
|
+-- ネットワーク切断 → 【オフライン状態】
|   isOnline=false      → オフラインバナー表示
|   |
|   +-- ネットワーク復旧 → 【初期状態】に戻る
|   |
+-- 新バージョン検出 → 【更新プロンプト状態】
    showUpdatePrompt=true → 更新ダイアログ表示
    |
    +-- 「今すぐ更新」 → applyUpdate()
    |   → SKIP_WAITING → ページリロード
    |
    +-- 「後で」 → dismissUpdate()
        → 【初期状態】に戻る
```

Service Workerのライフサイクル

Service Workerのライフサイクル:

【1. 登録】

```
navigator.serviceWorker.register('/sw.js')
|
v
```

【2. インストール】 ← install イベント

プリキャッシュの実行

```
|
v
```

【3. 待機】 ← 旧バージョンがまだアクティブ

ユーザーが「更新」を押すか、全タブを閉じるまで待機

```
|
v
```

【4. アクティブ化】 ← activate イベント

古いキャッシュの削除

```
|
v
```

【5. 動作中】 ← fetch イベント

リクエストのインターセプト、キャッシュ戦略の適用

22.8.3 キャッシュ戦略 (sw.js)

BON-LOGのService Worker (`public/sw.js`) は、リクエストの種類に応じて3つのキャッシュ戦略を使い分けます。

```
// public/sw.js

// キャッシュ名 (バージョン管理用)
const CACHE_VERSION = 'v1'
const STATIC_CACHE = `bon-log-static-${CACHE_VERSION}`
const DYNAMIC_CACHE = `bon-log-dynamic-${CACHE_VERSION}`
const IMAGE_CACHE = `bon-log-images-${CACHE_VERSION}`

// インストール時にプリキャッシュするアセット
const PRECACHE_ASSETS = [
  '/',
  '/offline.html',
  '/favicon.ico',
  '/icon-192.png',
  '/icon-512.png',
  '/site.webmanifest',
]

// キャッシュしないパターン (常にネットワークから取得)
const NETWORK_ONLY_PATTERNS = [
  /\api\auth\/, // 認証関連
  /\api\webhooks\/, // Webhook
  /\api\cron\/, // Cron
  /\monitoring/, // Sentry
  /\_next\webpack-hmr/, // HMR (開発時)
]
```

3つのキャッシュ戦略の比較

+-----+	
キャッシュ戦略の比較	
+-----+	
[1. Cache First] ← 静的アセット (JS, CSS, フォント)	
キャッシュ → あり → キャッシュを返す	
→ なし → ネットワーク → キャッシュに保存	
特徴: 高速表示、バージョンで管理 (変更時はURL変更)	
[2. Network First] ← API、HTMLページ	
ネットワーク → 成功 → レスポンスを返す&キャッシュ保存	
→ 失敗 → キャッシュから返す	
特徴: 常に最新データ、オフライン時はキャッシュにフォールバック	
[3. Stale While Revalidate] ← 画像	
キャッシュ → あり → キャッシュを即返す&バックグラウンドで更新	
→ なし → ネットワークから取得	
特徴: 超高速表示、次回アクセスで最新版が表示される	
+-----+	

各戦略の実装を見てみましょう。

```

// Cache First戦略（静的アセット向け）
async function cacheFirst(request, cacheName) {
  const cachedResponse = await caches.match(request)
  if (cachedResponse) {
    return cachedResponse // キャッシュがあれば即座に返す
  }

  // キャッシュになればネットワークから取得してキャッシュに保存
  try {
    const networkResponse = await fetch(request)
    if (networkResponse.ok) {
      const cache = await caches.open(cacheName)
      cache.put(request, networkResponse.clone())
    }
    return networkResponse
  } catch (error) {
    return new Response('Offline', { status: 503 })
  }
}

// Network First戦略（API・動的コンテンツ向け）
async function networkFirst(request, cacheName) {
  try {
    const networkResponse = await fetch(request)
    if (networkResponse.ok) {
      const cache = await caches.open(cacheName)
      cache.put(request, networkResponse.clone())
    }
    return networkResponse
  } catch (error) {
    // オフライン時はキャッシュにフォールバック
    const cachedResponse = await caches.match(request)
    if (cachedResponse) return cachedResponse
    return new Response(JSON.stringify({ error: 'Offline' }), {
      status: 503,
      headers: { 'Content-Type': 'application/json' }
    })
  }
}

// Stale While Revalidate戦略（画像向け）
async function staleWhileRevalidate(request, cacheName) {
  const cache = await caches.open(cacheName)
  const cachedResponse = await cache.match(request)

  // バックグラウンドでネットワークから更新
  const fetchPromise = fetch(request)
  .then((networkResponse) => {
    if (networkResponse.ok) {
      cache.put(request, networkResponse.clone())
    }
  })
}

```

```

    return networkResponse
  })
  .catch(() => null)

  // キャッシュがあれば即座に返す、なければネットワークを待つ
  return cachedResponse || fetchPromise
}

```

22.8.4 オフラインフォールバック

HTMLページへのアクセスがオフラインでキャッシュにもない場合、`/offline.html` を表示します。

```

// Network First with Offline Fallback戦略 (HTMLページ向け)
async function networkFirstWithOfflineFallback(request) {
  try {
    const networkResponse = await fetch(request)
    if (networkResponse.ok) {
      const cache = await caches.open(DYNAMIC_CACHE)
      cache.put(request, networkResponse.clone())
    }
    return networkResponse
  } catch (error) {
    // キャッシュを確認
    const cachedResponse = await caches.match(request)
    if (cachedResponse) return cachedResponse

    // HTMLリクエストの場合はオフラインページを返す
    if (request.headers.get('accept')?.includes('text/html')) {
      const offlinePage = await caches.match('/offline.html')
      if (offlinePage) return offlinePage
    }
    return new Response('Offline', { status: 503 })
  }
}

```

22.8.5 更新プロンプトとオフラインバナー

ServiceWorkerRegistrationコンポーネントは、2つの重要なUIを提供します。

更新プロンプト: 新しいバージョンのService Workerが利用可能な時に表示されます。

```

+-----+
| [更新アイコン] アップデートがあります |
|                                         |
| 新しいバージョンが利用可能です。       |
| 更新して最新機能をお使いください。     |
|                                         |
| [今すぐ更新] [後で]                   |
+-----+

```

「今すぐ更新」を押すと、待機中のService Workerに `SKIP_WAITING` メッセージを送り、新バージョンをアクティブにしてページをリロードします。

```

const applyUpdate = () => {
  if (waitingWorker) {
    waitingWorker.postMessage({ type: 'SKIP_WAITING' })
    setShowUpdatePrompt(false)
  }
}

```

オフラインバナー: ネットワーク接続が切れた時に画面上部に表示されます。

```

+-----+
| [WiFiオフアイコン] オフラインです - 一部の機能が制限されます |
+-----+

```

22.8.6 キャッシュのバージョン管理

キャッシュはバージョン名で管理されます。Service Workerの更新時に古いキャッシュを自動的に削除します。

```
// アクティブイベントで古いキャッシュをクリーンアップ
self.addEventListener('activate', (event) => {
  event.waitUntil(
    caches.keys().then((cacheNames) => {
      return Promise.all(
        cacheNames
          .filter((cacheName) => {
            // 現在のバージョン以外のキャッシュを削除
            return cacheName.startsWith('bon-log-') &&
              cacheName !== STATIC_CACHE &&
              cacheName !== DYNAMIC_CACHE &&
              cacheName !== IMAGE_CACHE
          })
          .map((cacheName) => caches.delete(cacheName))
      )
    })
  )
})
```

バージョンを上げる場合は `CACHE_VERSION` を変更するだけです。

```
const CACHE_VERSION = 'v2' // v1 → v2 に変更すると古いキャッシュは自動削除
```

▶ 理解度チェック（クリックして展開）

22.9 SEO構造化データ（sitemap・robots・JSON-LD）

▶ このセクションで学ぶこと（クリックして展開）

22.9.1 sitemap.ts（サイトマップの自動生成）

サイトマップは、検索エンジンにウェブサイト内のページ一覧を伝えるXMLファイルです。Next.jsでは `app/sitemap.ts` に関数を定義するだけで、 `/sitemap.xml` が自動生成されます。


```
// app/sitemap.ts
import { MetadataRoute } from 'next'
import { prisma } from '@lib/db'

// データベース接続が必要なため動的生成
export const dynamic = 'force-dynamic'

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const baseUrl = process.env.NEXT_PUBLIC_APP_URL || 'https://bon-log.com'

  // 1. 静的ページ
  const staticPages: MetadataRoute.Sitemap = [
    {
      url: baseUrl,
      lastModified: new Date(),
      changeFrequency: 'daily', // 更新頻度のヒント
      priority: 1,             // 重要度 (0.0 ~ 1.0)
    },
    {
      url: `${baseUrl}/privacy`,
      lastModified: new Date(),
      changeFrequency: 'monthly',
      priority: 0.3,
    },
    {
      url: `${baseUrl}/terms`,
      lastModified: new Date(),
      changeFrequency: 'monthly',
      priority: 0.3,
    },
    {
      url: `${baseUrl}/shops`,
      lastModified: new Date(),
      changeFrequency: 'daily',
      priority: 0.8,
    },
    {
      url: `${baseUrl}/events`,
      lastModified: new Date(),
      changeFrequency: 'daily',
      priority: 0.8,
    },
  ]

  // 2. 公開ユーザーページ (DBから動的取得)
  const users = await prisma.user.findMany({
    where: { isPublic: true, isSuspended: false },
    select: { id: true, updatedAt: true },
    take: 1000,
    orderBy: { updatedAt: 'desc' },
  })
}
```

```

const userPages: MetadataRoute.Sitemap = users.map((user) => ({
  url: `${baseUrl}/users/${user.id}`,
  lastModified: user.updatedAt,
  changeFrequency: 'weekly' as const,
  priority: 0.6,
}))

// 3. 投稿ページ（公開ユーザーの投稿のみ、リポストは除外）
const posts = await prisma.post.findMany({
  where: {
    user: { isPublic: true, isSuspended: false },
    repostPostId: null,
  },
  select: { id: true, createdAt: true },
  take: 5000,
  orderBy: { createdAt: 'desc' },
})

const postPages: MetadataRoute.Sitemap = posts.map((post) => ({
  url: `${baseUrl}/posts/${post.id}`,
  lastModified: post.createdAt,
  changeFrequency: 'monthly' as const,
  priority: 0.5,
}))

// 4. 盆栽園ページ
const shops = await prisma.bonsaiShop.findMany({
  select: { id: true, updatedAt: true },
  take: 1000,
  orderBy: { updatedAt: 'desc' },
})

const shopPages: MetadataRoute.Sitemap = shops.map((shop) => ({
  url: `${baseUrl}/shops/${shop.id}`,
  lastModified: shop.updatedAt,
  changeFrequency: 'weekly' as const,
  priority: 0.7,
}))

// 5. イベントページ（今後開催予定のもの優先）
const events = await prisma.event.findMany({
  where: {
    OR: [
      { endDate: { gte: new Date() } },
      { endDate: null, startDate: { gte: new Date() } },
    ],
  },
  select: { id: true, createdAt: true },
  take: 500,
  orderBy: { startDate: 'asc' },
})

```

```
const eventPages: MetadataRoute.Sitemap = events.map((event) => ({
  url: `${baseUrl}/events/${event.id}`,
  lastModified: event.createdAt,
  changeFrequency: 'weekly' as const,
  priority: 0.7,
}))

// すべてのページを結合して返す
return [ ... staticPages, ... userPages, ... postPages, ... shopPages, ... eventPages ]
}
```

priority（優先度）の設定方針

優先度	対象	理由
1.0	トップページ	サイトの入り口、最も重要
0.8	盆栽園一覧、イベント一覧	メインコンテンツへの入り口
0.7	個別の盆栽園、イベント	検索対象として重要
0.6	ユーザープロフィール	中程度の重要性
0.5	個別の投稿	数が多いが個々の重要性は低め
0.3	利用規約、プライバシーポリシー	SEO的には低優先

22.9.2 robots.ts（クロール制御）

`robots.txt` は、検索エンジンのクローラーにどのページをインデックスすべきか/すべきでないかを伝えるファイルです。

```

// app/robots.ts
import { MetadataRoute } from 'next'

export default function robots(): MetadataRoute.Robots {
  const baseUrl = process.env.NEXT_PUBLIC_APP_URL || 'https://bon-log.com'

  return {
    rules: [
      {
        userAgent: '*', // すべてのクローラーに対するルール
        allow: '/', // 基本的にすべて許可
        disallow: [
          '/admin/', // 管理画面
          '/admin/*',
          '/api/', // APIエンドポイント
          '/api/*',
          '/login', // 認証ページ
          '/register',
          '/verify-email',
          '/settings/', // 設定ページ
          '/settings/*',
          '/feed', // タイムライン（認証必要）
          '/bookmarks', // ブックマーク（認証必要）
          '/notifications', // 通知（認証必要）
          '/messages/', // メッセージ（認証必要）
          '/posts/scheduled/', // 予約投稿（認証必要）
          '/analytics', // アナリティクス
          '/*.json$', // JSONファイル
          '/*?*$', // クエリパラメータ付きURL
        ],
      },
      {
        userAgent: 'Googlebot', // Googlebot専用ルール
        allow: '/',
        disallow: [
          '/admin/',
          '/api/',
          '/login',
          '/register',
          '/settings/',
          '/feed',
          '/bookmarks',
          '/notifications',
          '/messages/',
          '/posts/scheduled/',
          '/analytics',
        ],
      },
    ],
    sitemap: `${baseUrl}/sitemap.xml`, // サイトマップのURL
  }
}

```

```
}
}
```

クローリング制御の判断基準

インデックスすべきページ:

- ✓ 公開されたコンテンツ（投稿、ユーザープロフィール）
- ✓ 盆栽園情報、イベント情報
- ✓ 利用規約、ヘルプページ

インデックスすべきでないページ:

- ✗ 認証が必要なページ（/feed, /bookmarks, /notifications）
- ✗ 管理画面（/admin/）
- ✗ APIエンドポイント（/api/）
- ✗ 認証フォーム（/login, /register）
- ✗ 個人設定（/settings/）

22.9.3 JSON-LD構造化データ（JsonLd.tsx）

JSON-LD（JavaScript Object Notation for Linked Data）は、ページの内容を検索エンジンが理解しやすい形式で記述する仕組みです。BON-LOGでは `components/seo/JsonLd.tsx` に6種類の構造化データコンポーネントを実装しています。

JSON-LDの仕組み:

[通常のHTML]

```
<h1>〇〇盆栽園</h1>
<p>東京都〇〇区 ... </p>
<p>評価: 4.5/5</p>
→ 検索エンジン: 「テキストがあるが、何の情報かわからない」
```

[JSON-LD付きHTML]

```
<h1>〇〇盆栽園</h1>
<p>東京都〇〇区 ... </p>
<script type="application/ld+json">
{
  "@type": "LocalBusiness",
  "name": "〇〇盆栽園",
  "address": "東京都〇〇区 ... ",
  "aggregateRating": { "ratingValue": 4.5, "reviewCount": 10 }
}
</script>
→ 検索エンジン: 「これはローカルビジネスで、評価は4.5/5、レビュー10件だ」
→ 検索結果に星マーク付きで表示される可能性
```

6種類の構造化データコンポーネント

OrganizationJsonLd（組織情報）

ルートレイアウトに配置して、サイト全体の組織情報を表現します。

```
// 使用例: app/layout.tsx
<OrganizationJsonLd
  name="BON-LOG"
  url="https://bon-log.com"
  logo="https://bon-log.com/logo.png"
  description="盆栽愛好家のためのSNS"
/>
```

```
// 出力されるJSON-LD
{
  "@context": "https://schema.org",
  "@type": "Organization",
  "name": "BON-LOG",
  "url": "https://bon-log.com",
  "logo": "https://bon-log.com/logo.png",
  "description": "盆栽愛好家のためのSNS"
}
```

LocalBusinessJsonLd（盆栽園情報）

盆栽園の詳細ページに配置します。Googleマップやローカル検索結果での表示に影響します。

```
// 使用例: app/(main)/shops/[id]/page.tsx
<LocalBusinessJsonLd
  name="〇〇盆栽園"
  address="東京都文京区 ..."
  url="https://bon-log.com/shops/xxx"
  telephone="03-xxxx-xxxx"
  aggregateRating={{ ratingValue: 4.5, reviewCount: 10 }}
  geo={{ latitude: 35.6762, longitude: 139.6503 }}
/>
```

```
// 出力されるJSON-LD
{
  "@context": "https://schema.org",
  "@type": "LocalBusiness",
  "@id": "https://bon-log.com/shops/xxx",
  "name": "〇〇盆栽園",
  "address": {
    "@type": "PostalAddress",
    "streetAddress": "東京都文京区 ... ",
    "addressCountry": "JP"
  },
  "aggregateRating": {
    "@type": "AggregateRating",
    "ratingValue": 4.5,
    "reviewCount": 10,
    "bestRating": 5,
    "worstRating": 1
  },
  "geo": {
    "@type": "GeoCoordinates",
    "latitude": 35.6762,
    "longitude": 139.6503
  }
}
```

EventJsonLd（イベント情報）

イベント詳細ページに配置します。Googleカレンダーや検索結果のイベント表示に使用されます。

```
<EventJsonLd
  name="盆栽展示会2026"
  startDate="2026-04-01T10:00:00+09:00"
  endDate="2026-04-03T17:00:00+09:00"
  location={{ name: "東京ドーム", address: "東京都文京区 ... " }}
  url="https://bon-log.com/events/xxx"
  organizer="日本盆栽協会"
/>
```

ArticleJsonLd（記事・投稿情報）

投稿詳細ページに配置します。

```
<ArticleJsonLd
  headline="盆栽の育て方入門"
  datePublished="2026-01-01T12:00:00+09:00"
  author={ { name: "山田太郎", url: "https://bon-log.com/users/xxx" } }
  url="https://bon-log.com/posts/xxx"
  image="https:// ... "
/>
```

BreadcrumbJsonLd (パンくずリスト)

階層的なページ構造を検索エンジンに伝えます。検索結果でパンくずが表示され、ユーザーがサイトの構造を理解しやすくなります。

```
// 使用例: app/(main)/shops/[id]/page.tsx
<BreadcrumbJsonLd
  items={ [
    { name: "ホーム", url: "https://bon-log.com" },
    { name: "盆栽園", url: "https://bon-log.com/shops" },
    { name: "〇〇盆栽園", url: "https://bon-log.com/shops/xxx" },
  ] }
/>
```

```
// 出力されるJSON-LD
{
  "@context": "https://schema.org",
  "@type": "BreadcrumbList",
  "itemListElement": [
    { "@type": "ListItem", "position": 1, "name": "ホーム", "item": "https://bon-log.com" },
    { "@type": "ListItem", "position": 2, "name": "盆栽園", "item": "https://bon-log.com/shops" },
    { "@type": "ListItem", "position": 3, "name": "〇〇盆栽園", "item": "https://bon-log.com/shops/xxx" },
  ]
}
```

Google検索結果でのパンくず表示例

Google検索結果での表示:

```
bon-log.com > 盆栽園 > 〇〇盆栽園    ← パンくずリスト
〇〇盆栽園 - BON-LOG                  ← タイトル
東京都文京区にある歴史ある盆栽園。 ← 説明文
評価: ★★★★★ (4.5) - レビュー 10件 ← 評価 (LocalBusinessJsonLdから)
```


PersonJsonLd (ユーザープロフィール)

ユーザーのプロフィールページに配置して、人物情報を構造化データとして出力します。

```
// 使用例: app/(main)/users/[id]/page.tsx
<PersonJsonLd
  name="山田太郎"
  url="https://bon-log.com/users/xxx"
  image="https://r2.bon-log.com/avatars/xxx.jpg"
  description="盆栽歴10年の愛好家です。松柏類を中心に育てています。"
  location="東京都"
/>
```

```
// 出力されるJSON-LD
{
  "@context": "https://schema.org",
  "@type": "Person",
  "@id": "https://bon-log.com/users/xxx",
  "name": "山田太郎",
  "url": "https://bon-log.com/users/xxx",
  "image": "https://r2.bon-log.com/avatars/xxx.jpg",
  "description": "盆栽歴10年の愛好家です。松柏類を中心に育てています。",
  "homeLocation": {
    "@type": "Place",
    "name": "東京都"
  }
}
```

PersonJsonLdコンポーネントの実装を詳しく見てみましょう。

```
// components/seo/JsonLd.tsx (PersonJsonLd部分)

// プロパティの型定義
interface PersonJsonLdProps {
  name: string // 人物名 (必須)
  url: string // プロフィールページのURL (必須)
  image?: string // プロフィール画像URL (任意)
  description?: string // 自己紹介・説明 (任意)
  worksFor?: string // 所属組織名 (任意)
  location?: string // 居住地・所在地 (任意)
}

export function PersonJsonLd({
  name,
  url,
  image,
  description,
  worksFor,
  location,
}: PersonJsonLdProps) {
  // JSON-LDオブジェクトを構築
  const jsonLd = {
    '@context': 'https://schema.org', // Schema.orgの語彙を使用
    '@type': 'Person', // エンティティタイプ: 人物
    '@id': url, // 一意識別子としてURLを使用

    name, // 人物名
    url, // プロフィールURL

    // スプレッド構文とショートサーキット評価で、
    // 値がある場合のみプロパティを追加 (undefinedを含めない)
    ...(image && { image }), // プロフィール画像
    ...(description && { description }), // 説明

    // 所属組織 (Organization型としてネスト)
    ...(worksFor && {
      worksFor: {
        '@type': 'Organization',
        name: worksFor,
      },
    }),

    // 居住地 (Place型としてネスト)
    ...(location && {
      homeLocation: {
        '@type': 'Place',
        name: location,
      },
    }),
  }
}
```

```
// <script type="application/ld+json"> で出力
// dangerouslySetInnerHTML: JSON文字列をそのままHTMLに埋め込む
// → 検索エンジンのクローラーがこのJSONを読み取る
return (
  <script
    type="application/ld+json"
    dangerouslySetInnerHTML={{ __html: JSON.stringify(jsonLd) }}
  />
)
}
```

...(value && { key: value }) パターンの解説

このパターンは「条件付きプロパティ追加」です:

```
// imageがある場合 ("https://example.com/photo.jpg") :
...(image && { image })
// → ... ("https://example.com/photo.jpg" && { image: "https://example.com/photo.jpg" })
// → ... { image: "https://example.com/photo.jpg" }
// → image: "https://example.com/photo.jpg" がオブジェクトに追加される

// imageがない場合 (undefined) :
...(image && { image })
// → ... (undefined && { image: undefined })
// → ... (undefined) ← falseに評価され短絡
// → 何も追加されない ← JSON-LDにimageプロパティが含まれない
```

なぜこのパターンを使うのか？

→ JSON-LDに undefined や null のプロパティを含めると、
検索エンジンが不正な構造化データとして警告する場合があります

WebSiteJsonLd (ウェブサイト情報)

サイト内検索ボックスの表示に対応します。

```
<WebSiteJsonLd
  name="BON-LOG"
  url="https://bon-log.com"
  description="盆栽愛好家のためのSNS"
  searchUrl="https://bon-log.com/search"
/>
```

検索結果にサイト内検索ボックスが表示される可能性があります (Sitelinkサーチボックス)。

```
// 出力されるJSON-LD（サイト内検索対応）
{
  "@context": "https://schema.org",
  "@type": "WebSite",
  "name": "BON-LOG",
  "url": "https://bon-log.com",
  "potentialAction": {
    "@type": "SearchAction",
    "target": {
      "@type": "EntryPoint",
      "urlTemplate": "https://bon-log.com/search?q={search_term_string}"
    },
    "query-input": "required name=search_term_string"
  }
}
```

22.9.4 構造化データの配置場所まとめ

BON-LOGの各ページにどのJSON-LDコンポーネントを配置するかの一覧です。

ページ	使用する構造化データ
app/layout.tsx（ルートレイアウト）	OrganizationJsonLd, WebSiteJsonLd
app/(main)/shops/[id]/page.tsx	BreadcrumbJsonLd, LocalBusinessJsonLd
app/(main)/events/[id]/page.tsx	BreadcrumbJsonLd, EventJsonLd
app/(main)/posts/[id]/page.tsx	BreadcrumbJsonLd, ArticleJsonLd
app/(main)/users/[id]/page.tsx	BreadcrumbJsonLd, PersonJsonLd

22.9.5 構造化データのテスト方法

構造化データが正しく出力されているかを確認するためのツールと手順を紹介します。

1. Googleリッチリザルトテスト

手順:

1. <https://search.google.com/test/rich-results> にアクセス
2. テストしたいページのURLを入力
3. 「URLをテスト」をクリック
4. 結果を確認:
 - ✅ 有効なアイテムが検出されました (正常)
 - ⚠️ 警告があります (改善の余地あり)
 - ❌ エラーがあります (修正が必要)

2. Chrome DevToolsでの確認

手順:

1. Chrome DevToolsを開く (F12)
2. Elements タブを選択
3. Ctrl+F で "application/ld+json" を検索
4. `<script type="application/ld+json">` タグの中身を確認
5. `JSON.parse()` で正しいJSONか検証

例:

```
<script type="application/ld+json">
  { "@context": "https://schema.org", "@type": "LocalBusiness", ... }
</script>
```

3. Schema.orgバリデーター

手順:

1. <https://validator.schema.org/> にアクセス
2. JSON-LDコードを貼り付け
3. 「Validate」をクリック
4. Schema.orgの仕様に準拠しているか確認

22.9.6 サイトマップのパフォーマンス考慮事項

`app/sitemap.ts` はデータベースに大量のクエリを発行するため、パフォーマンスに配慮した設計が必要です。

サイトマップ生成のパフォーマンス設計:

【問題】

ユーザー1000件 + 投稿5000件 + 盆栽園1000件 + イベント500件
= 合計7500件のURLを生成

【対策】

1. `take` で取得件数を制限
 - ユーザー: 1000件まで
 - 投稿: 5000件まで
 - 更新日時の降順で最新のものを優先
2. `select` で必要なカラムのみ取得
 - `id` と `updatedAt/createdAt` だけ
 - 不要なリレーションを含めない
3. `force-dynamic` で動的生成
 - ビルド時にDB接続が不要
 - リクエスト時に最新データを取得

【注意】

サイトマップが50,000URLを超える場合は、
サイトマップインデックスファイルを使って分割する必要がある。
Next.jsでは `app/sitemap.ts` を配列形式にすることで
複数のサイトマップを生成できる。

▶ 理解度チェック（クリックして展開）

22.10 Web Vitals（パフォーマンス計測）

▶ このセクションで学ぶこと（クリックして展開）

22.10.1 Core Web Vitalsとは

Core Web Vitalsは、Googleが定義するウェブページのユーザー体験を測定する指標です。検索順位にも影響する重要な指標です。2021年からGoogleの検索ランキング要因に含まれています。

たとえば理解する: レストランの顧客体験

Web Vitals = レストランでの顧客体験を数値化したもの

【LCP】料理が出てくるまでの時間

「席に着いてからメイン料理が届くまで何分かかるか」

→ 2.5秒以内なら快適、4秒以上だとイライラ

【INP】注文への応答速度

「ウェイターに声をかけてから反応するまで何秒かかるか」

→ 200ms以内なら快適、500ms以上だと「聞こえてないのかな」

【CLS】テーブルの配置が突然変わる

「食事中にテーブルが急に動いて、飲み物をこぼしてしまう」

→ 0.1以下なら安定、0.25以上だとストレス

Core Web Vitals の3つの指標		
【LCP】 Largest Contentful Paint (最大コンテンツの描画時間)		
ユーザーが「ページが表示された」と感じるまでの時間		
✓ 良好: 2.5秒以下	⚠ 改善必要: 4.0秒以下	✗ 不良: 4.0秒超
【INP】 Interaction to Next Paint (操作から描画までの時間)		
ボタンクリックなどの操作から画面が反応するまでの時間		
✓ 良好: 200ms以下	⚠ 改善必要: 500ms以下	✗ 不良: 500ms超
【CLS】 Cumulative Layout Shift (レイアウトのずれ)		
ページ読み込み中に要素が予期せず移動する度合い		
✓ 良好: 0.1以下	⚠ 改善必要: 0.25以下	✗ 不良: 0.25超

22.10.2 WebVitalsReporterコンポーネント

BON-LOGでは `components/analytics/WebVitals.tsx` でWeb Vitalsの計測と報告を行います。

```

// components/analytics/WebVitals.tsx
'use client'

import { useReportWebVitals } from 'next/web-vitals'

// Web Vitals指標の型定義
interface WebVitalsMetric {
  id: string
  name: 'CLS' | 'FID' | 'FCP' | 'INP' | 'LCP' | 'TTFB'
  value: number
  rating: 'good' | 'needs-improvement' | 'poor'
  delta: number
  navigationType: string
}

// Google推奨の閾値
const thresholds = {
  LCP: { good: 2500, poor: 4000 }, // ミリ秒
  FID: { good: 100, poor: 300 }, // ミリ秒
  CLS: { good: 0.1, poor: 0.25 }, // スコア（単位なし）
  INP: { good: 200, poor: 500 }, // ミリ秒
  FCP: { good: 1800, poor: 3000 }, // ミリ秒
  TTFB: { good: 800, poor: 1800 }, // ミリ秒
}

export function WebVitalsReporter() {
  useReportWebVitals((metric) => {
    const { name, value, rating, id, delta, navigationType } = metric as WebVitalsMetric

    // 開発環境ではコンソールに出力
    if (process.env.NODE_ENV === 'development') {
      const threshold = thresholds[name as keyof typeof thresholds]
      const emoji = rating === 'good' ? '✅' : rating === 'needs-improvement' ? '⚠️' : '❌'

      console.log(
        `${emoji} Web Vitals [${name}]: ${value.toFixed(name === 'CLS' ? 3 : 0)}${name === 'CLS' ? ' (スコア)' : ' (ミリ秒)'}\n` +
        `  { id: ${id}, delta: ${delta}, navigationType: ${navigationType}, threshold: ${threshold} }`
      )
    }

    // 本番環境ではアナリティクスに送信
    // Google Analytics 4への送信例:
    // if (process.env.NODE_ENV === 'production') {
    //   if (typeof window !== 'undefined' && 'gtag' in window) {
    //     (window as any).gtag('event', name, {
    //       value: Math.round(name === 'CLS' ? value * 1000 : value),
    //       event_category: 'Web Vitals',
    //       event_label: id,
    //       non_interaction: true,
    //     })
    //   }
    // }
  })
}

```



```
    // }  
  })  
  
  // UIをレンダリングしない（計測のみ）  
  return null  
}
```

ルートレイアウトでの配置

```
// app/layout.tsx  
import { WebVitalsReporter } from '@components/analytics/WebVitals'  
  
export default function RootLayout({ children }: { children: React.ReactNode }) {  
  return (  
    <html>  
      <body>  
        <WebVitalsReporter />  
        {children}  
      </body>  
    </html>  
  )  
}
```

22.10.3 各指標の意味と改善方法

指標	原因	対策
LCP が遅い場合	大きな画像、重いフォント、サーバー応答が遅い	next/image で画像を最適化（WebP変換、遅延読み込み）
		LCP画像に priority 属性を付与
		フォントの preload
		サーバーのレスポンスタイムを改善（キャッシュ、DB最適化）
INP が遅い場合	重いJavaScript処理、メインスレッドのブロック	重い処理を Web Worker に移動
		React.memo で不要な再レンダリングを防止
		useTransition で低優先度の更新を遅延
		コンポーネントの動的インポート（next/dynamic）
CLS が大きい場合	画像のサイズ未指定、動的コンテンツの挿入	next/image に width/height を必ず指定
		フォントの font-display: swap を避ける（optional 推奨）
		広告やバナーの表示領域を事前に確保
		Skeleton UIで読み込み中のレイアウトを固定

22.10.4 開発環境でのコンソール出力例

開発中にChrome DevToolsのコンソールで以下のような出力を確認できます。

```

✔ Web Vitals [LCP]: 1245ms (good) {id: "v4-xxx", delta: 1245, navigationType: "navigate", th
✔ Web Vitals [CLS]: 0.012 (good) {id: "v4-yyy", delta: 0.012, navigationType: "navigate", th
⚠ Web Vitals [INP]: 312ms (needs-improvement) {id: "v4-zzz", delta: 312, navigationType: "nav
✔ Web Vitals [FCP]: 890ms (good) {id: "v4-aaa", delta: 890, navigationType: "navigate", thre
✔ Web Vitals [TTFB]: 234ms (good) {id: "v4-bbb", delta: 234, navigationType: "navigate", thre

```

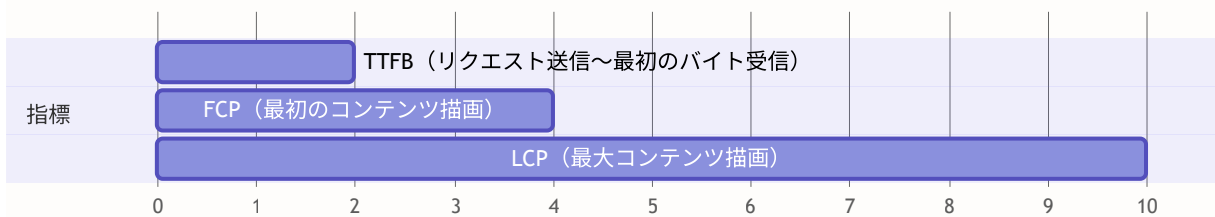
この出力から、INPに改善の余地があることが一目で分かります。

22.10.5 追加の指標（FCP、TTFB）

Core Web Vitals（LCP、INP、CLS）に加えて、WebVitalsReporterは2つの補助指標も計測します。

指標	正式名称	説明	良好	改善 必要	不良	改善方法
FCP	First Contentful Paint	ページにテキストや画像が最初に表示されるまでの時間。LCPよりも早いタイミングで計測される	1.8秒以下	3.0秒以下	3.0秒超	サーバーレスポンスの高速化、CSSの最適化
TTFB	Time to First Byte	ブラウザがサーバーからの最初のバイトを受信するまでの時間。サーバーの処理速度を反映する指標	800ms以下	1.8秒以下	1.8秒超	CDNの利用、サーバーのスペックアップ、DB最適化

時系列での指標の関係



22.10.6 本番環境でのアナリティクス連携

WebVitalsReporterをGoogle Analytics 4（GA4）やVercel Analyticsと連携する方法です。

```
// Google Analytics 4への送信例（本番環境で有効化する場合）
export function WebVitalsReporter() {
  useReportWebVitals((metric) => {
    const { name, value, rating, id } = metric as WebVitalsMetric

    // 開発環境: コンソールに出力
    if (process.env.NODE_ENV === 'development') {
      // ... (省略)
    }

    // 本番環境: GA4に送信
    if (process.env.NODE_ENV === 'production') {
      // gtagが存在するか確認（GA4のスク립トが読み込まれている場合）
      if (typeof window !== 'undefined' && 'gtag' in window) {
        // GA4にカスタムイベントとして送信
        (window as { gtag: (...args: unknown[]) => void }).gtag('event', name, {
          // CLSは0~1の小数値なので1000倍して整数にする
          // その他のメトリクスはミリ秒の整数値
          value: Math.round(name === 'CLS' ? value * 1000 : value),
          event_category: 'Web Vitals', // GA4でのカテゴリ分類
          event_label: id, // 個別の計測を識別
          non_interaction: true, // 直帰率に影響させない
        })
      }
    }
  })
}
return null
}
```

GA4でのWeb Vitals確認方法:

GA4ダッシュボード > レポート > エンゲージメント > イベント

イベント名	件数	平均値
LCP	1,234	2,100ms
INP	987	180ms
CLS	1,234	85 (0.085)
FCP	1,234	1,200ms
TTFB	1,234	450ms

→ 「LCPの平均が2,100ms。目標の2,500ms以内なので良好」 → 「INPの平均が180ms。目標の200ms以内なので良好」

22.10.7 BON-LOGでのパフォーマンス改善事例

実際にBON-LOGで行ったパフォーマンス改善の具体例です。

改善例1: LCPの改善 (3.2秒 → 1.8秒)

原因: タイムラインの最初の画像が遅延読み込みされていた

対策: `priority`属性を追加して優先読み込みに変更

```
``tsx
```

// Before: 遅延読み込み (デフォルト)

```
<Image src={post.imageUrl} alt=" ..." width={600} height={400} />
```

// After: 優先読み込み (LCP要素に指定)

```
<Image src={post.imageUrl} alt=" ..." width={600} height={400} priority />
```

改善例2: CLSの改善 (0.18 → 0.05) 原因: アバター画像のwidth/heightが未指定でレイアウトがずれていた

対策: 固定サイズのスケルトンUIを追加

// Before: サイズ未指定

```
<img src={user.avatarUrl} alt=" ..." />
```

// After: サイズ固定 + next/image

```
<Image src={user.avatarUrl} alt=" ..." width={40} height={40} />
```

改善例3: INPの改善 (350ms → 150ms) 原因: いいねボタンのクリック時に重いAPIコールがメインスレッドをブロック

対策: `useTransition`で低優先度の更新に変更

// Before: 即時更新 (メインスレッドをブロック)

```
const handleLike = async () => {
  await likePost(postId) // ← ここでUIがフリーズ
  mutate()
}
```

// After: 楽観的更新 + useTransition

```
const [isPending, startTransition] = useTransition()
const handleLike = () => {
  setLiked(!liked) // ← 楽観的にUI更新 (即座に反映)
  startTransition(async () => {
    await likePost(postId) // ← バックグラウンドで実行
  })
}
```

```
<details>
```

```
<summary><strong>理解度チェック</strong>（クリックして展開）</summary>
```

```
**Q1: WebVitalsReporterコンポーネントが`return null`を返す理由は何ですか？**
```

```
A: このコンポーネントはUIを表示する必要がなく、パフォーマンス計測とレポートのみを行います。`useReportWebVitals`は
```

```
**Q2: CLSの値だけ単位が`ms`ではなく無次元数である理由は何ですか？**
```

```
A: CLSはレイアウトのずれの度合いを表す累積スコアであり、「影響面積 x 移動距離」の計算で算出されます。時間の指
```

```
**Q3: `tracesSampleRate`の値とWeb Vitalsの計測は関係がありますか？**
```

```
A: `tracesSampleRate`はSentryのパフォーマンストレーシングのサンプリング率であり、`useReportWebVitals`は
```

```
**Q4: GA4への送信時にCLSだけ1000倍する理由は何ですか？**
```

```
A: GA4のイベントvalue値は整数を推奨しています。CLSは0.05や0.12のような小数值なので、1000倍して整数化（50や
```

```
**Q5: `non_interaction: true`を設定する理由は何ですか？**
```

```
A: Web Vitalsの計測はユーザーの意図的な操作ではなく、自動的に行われるものです。`non_interaction: true`を
```

```
</details>
```

```
---
```

22.11 本番セキュリティチェック

```
<details>
```

```
<summary><strong>このセクションで学ぶこと</strong>（クリックして展開）</summary>
```

- 本番環境のセキュリティ設定を自動検証する仕組み
- NEXTAUTH_SECRETの強度チェック
- 必須環境変数の存在確認
- セキュリティ警告のログ出力

```
</details>
```

22.11.1 セキュリティチェックの概要

```
BON-LOGでは`lib/security-checks.ts`で、アプリケーション起動時に本番環境のセキュリティ設定を自動的に検証し
```

```
<div class="mermaid-rendered">
```

```

```

```
</div>
```

22.11.2 セキュリティチェックの仕組み（全体構成）

`lib/security-checks.ts`は5つの関数で構成されており、それぞれが異なる検証を担当します。

security-checks.ts の関数構成:

enforceSecurityInProduction() ← エントリーポイント（起動時に呼ばれる） | +-- runSecurityChecks() ← すべてのチェックを実行して結果を返す || +-- validateAuthSecret() ← [1] シークレットの強度チェック | | +-- validateRequiredEnvVars() ← [2] 必須環境変数の存在チェック | +-- logSecurityWarnings() ← 結果をコンソールに出力 | +-- validateAuthSecret() ← [1] の結果をログ出力 | +-- validateRequiredEnvVars() ← [2] の結果をログ出力 | +-- 追加チェック ← [3] HTTPS、DEBUGモード等

開発環境: WARNING（警告）として出力 → 開発を妨げない 本番環境: ERROR（エラー）として出力 → 問題に気づかせる

なぜ起動時にセキュリティチェックを行うのか？

シナリオ: セキュリティチェックがない場合

[1] 開発者がNEXTAUTH_SECRETを設定し忘れてデプロイ | [2] アプリは正常に起動（エラーなし） | [3] ユーザーがログインする | [4] NextAuth.jsがデフォルトのシークレットでJWTを生成 | [5] 攻撃者がデフォルトのシークレットでJWTを偽造 | [6] 他のユーザーになりすましてアクセス可能に！ | [7] 数週間後に問題が発覚... 被害が拡大...

→ セキュリティチェックがあれば、[2]の段階で警告が出る → 問題に即座に気づいて対処できる

22.11.3 NEXTAUTH_SECRET の強度検証

認証に使用するシークレットキーの品質を検証します。

```

```typescript
// lib/security-checks.ts

// 弱いシークレットとして検出するパターン
const WEAK_SECRET_PATTERNS = [
 'your-development-secret',
 'your-secret-key',
 'change-in-production',
 'development-secret',
 'dev-secret',
 'test-secret',
 'secret',
 'password',
 '12345',
 'example',
 'changeme',
 'placeholder',
]

const MIN_SECRET_LENGTH = 32 // 最小32文字

export function validateAuthSecret(): {
 valid: boolean
 warnings: string[]
 errors: string[]
} {
 const secret = process.env.NEXTAUTH_SECRET
 const isProduction = process.env.NODE_ENV === 'production'
 const warnings: string[] = []
 const errors: string[] = []

 // 未設定チェック
 if (!secret) {
 if (isProduction) {
 errors.push('NEXTAUTH_SECRET が設定されていません。本番環境では必須です。')
 } else {
 warnings.push('NEXTAUTH_SECRET が設定されていません。')
 }
 return { valid: !isProduction, warnings, errors }
 }

 // 長さチェック
 if (secret.length < MIN_SECRET_LENGTH) {
 const message = `NEXTAUTH_SECRET が短すぎます (${secret.length}文字) 。${MIN_SECRET_LENGTH}文

```



```

 if (isProduction) errors.push(message)
 else warnings.push(message)
 }

 // 弱いパターンチェック
 const lowerSecret = secret.toLowerCase()
 for (const pattern of WEAK_SECRET_PATTERNS) {
 if (lowerSecret.includes(pattern.toLowerCase())) {
 const message = `NEXTAUTH_SECRET に弱いパターン '${pattern}' が含まれています。`
 if (isProduction) errors.push(message)
 else warnings.push(message)
 break
 }
 }

 // エントロピーチェック（ユニーク文字の割合）
 const uniqueChars = new Set(secret).size
 const entropyRatio = uniqueChars / secret.length
 if (entropyRatio < 0.3) {
 if (isProduction) {
 warnings.push(`NEXTAUTH_SECRET のエントロピーが低いです。`)
 }
 }

 return { valid: errors.length === 0, warnings, errors }
}

```

## 安全なシークレットの生成方法

```

OpenSSLで32バイトのランダム文字列を生成（推奨）
openssl rand -base64 32
出力例: K7xH3qR2mNvB8tF5jL9pW1zA4cE6gI0rS2uY7oD3nX=

Node.jsで生成する場合
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"

```

### 22.11.4 起動時のセキュリティ検証

`enforceSecurityInProduction` 関数は、本番環境でセキュリティチェックに失敗した場合にエラーをログ出力します。

```

export function enforceSecurityInProduction(): void {
 if (process.env.NODE_ENV !== 'production') {
 // 開発環境では警告のみ
 logSecurityWarnings()
 return
 }

 const result = runSecurityChecks()

 if (!result.valid) {
 const errors = [
 ... result.secretCheck.errors,
 ... (result.envCheck.missing.length > 0
 ? ['必須環境変数が不足: ${result.envCheck.missing.join(', ')}']
 : []),
]

 console.error('[SECURITY] 本番環境でセキュリティチェックに失敗しました:')
 errors.forEach((e) => console.error(`  - ${e}`))
 }

 logSecurityWarnings()
}

```

開発環境での出力例:

```

[SECURITY WARNING] NEXTAUTH_SECRET が短すぎます（16文字）。32文字以上を推奨します。
[SECURITY WARNING] NEXTAUTH_SECRET に弱いパターン 'dev-secret' が含まれています。

```

本番環境での出力例:

```

[SECURITY ERROR] NEXTAUTH_SECRET が設定されていません。本番環境では必須です。
[SECURITY ERROR] 必須環境変数が不足: DATABASE_URL, NEXTAUTH_URL

```

► 理解度チェック（クリックして展開）

## 22.12 Cronジョブ（定期実行タスク）

▶ このセクションで学ぶこと（クリックして展開）

### 22.12.1 Cronジョブとは

Cronジョブは、指定したスケジュールで自動的に実行されるタスクです。BON-LOGでは3つのCronジョブを使用しています。

Cronジョブ	スケジュール	機能	詳細
publish-scheduled	5分ごと	予約投稿の自動公開	公開時刻を過ぎた予約投稿を実際の投稿として公開
check-subscriptions	毎日深夜1時	サブスクリプション期限切れチェック	期限切れユーザーのプレミアムステータスをリセット、期限切れ通知メールの送信
cleanup-events	毎月1日の深夜0時	古いイベントの削除	終了から6ヶ月以上経過したイベントをDBから削除

### 22.12.2 Vercel Cron Jobsの設定

Vercelでは `vercel.json` にCronスケジュールを定義します。

```
// vercel.json
{
 "crons": [
 {
 "path": "/api/cron/publish-scheduled",
 "schedule": "* /5 * * * *"
 },
 {
 "path": "/api/cron/check-subscriptions",
 "schedule": "0 1 * * *"
 },
 {
 "path": "/api/cron/cleanup-events",
 "schedule": "0 0 1 * * *"
 }
]
}
```

## cron式の読み方

cron式の各フィールド: 分 時 日 月 曜日

位置	フィールド	範囲
1番目	分	0-59
2番目	時	0-23
3番目	日	1-31
4番目	月	1-12
5番目	曜日	0-7 (0=日曜)

cron式	意味
<span>* /5 * * * *</span>	5分ごと
<span>0 1 * * *</span>	毎日1時0分
<span>0 0 1 * *</span>	毎月1日0時0分
<span>0 * /6 * * *</span>	6時間ごと
<span>0 9 * * 1-5</span>	平日9時0分

### 22.12.3 Cron認証 (cron-auth.ts)

---

Cronエンドポイントは外部から不正に呼び出されるのを防ぐため、HMAC署名ベースの認証を実装しています。

```

// lib/cron-auth.ts
import crypto from 'crypto'

// 許容するタイムスタンプの誤差（5分以内）
const TIMESTAMP_TOLERANCE_MS = 5 * 60 * 1000

export function verifyCronAuth(
 authHeader: string | null,
 timestampHeader: string | null
): { valid: boolean; error?: string } {
 const cronSecret = process.env.CRON_SECRET

 // CRON_SECRETが未設定の場合
 if (!cronSecret) {
 if (process.env.NODE_ENV === 'production') {
 return { valid: false, error: 'CRON_SECRET is not configured' }
 }
 // 開発環境では認証をスキップ
 return { valid: true }
 }

 // レガシー認証 (Bearer token) のサポート
 if (authHeader === `Bearer ${cronSecret}` && !timestampHeader) {
 return { valid: true }
 }

 // HMAC認証
 if (!authHeader?.startsWith('HMAC ')) {
 return { valid: false, error: 'Invalid authorization scheme' }
 }

 if (!timestampHeader) {
 return { valid: false, error: 'Missing timestamp header' }
 }

 // タイムスタンプ検証 (リプレイ攻撃対策)
 const timestamp = parseInt(timestampHeader, 10)
 const timeDiff = Math.abs(Date.now() - timestamp)
 if (timeDiff > TIMESTAMP_TOLERANCE_MS) {
 return { valid: false, error: 'Request timestamp is too old' }
 }

 // 署名検証
 const providedSignature = authHeader.slice(5) // 'HMAC ' を除去
 const expectedSignature = crypto
 .createHmac('sha256', cronSecret)
 .update(timestampHeader)
 .digest('hex')

 // 定数時間比較 (タイミング攻撃対策)
 if (!crypto.timingSafeEqual(

```

```

 Buffer.from(providedSignature),
 Buffer.from(expectedSignature)
)) {
 return { valid: false, error: 'Invalid signature' }
 }

 return { valid: true }
}

```

## HMAC認証の仕組み

HMAC認証の流れ:

[呼び出し側 (Vercel Cron) ]

1. 現在のタイムスタンプを取得: "1706745600000"
2. HMAC-SHA256で署名を生成:  
`HMAC(timestamp="1706745600000", secret="your-cron-secret")`  
 → "a1b2c3d4e5f6 ... "
3. リクエストヘッダーに付与:  
`Authorization: HMAC a1b2c3d4e5f6 ...`  
`X-Cron-Timestamp: 1706745600000`

[受け側 (APIエンドポイント) ]

1. タイムスタンプが5分以内か検証 (リプレイ攻撃対策)
2. 同じ方法で期待される署名を計算
3. `timingSafeEqual` で署名を比較 (タイミング攻撃対策)
4. 一致すればリクエストを許可

## 22.12.4 予約投稿の自動公開 (publish-scheduled)

5分ごとに実行され、公開時刻を過ぎた予約投稿を実際の投稿として公開します。

```
// app/api/cron/publish-scheduled/route.ts
export async function GET(request: NextRequest) {
 // 認証チェック
 const authResult = verifyCronAuth(
 request.headers.get('authorization'),
 request.headers.get('x-cron-timestamp')
)
 if (!authResult.valid) {
 return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
 }

 try {
 const now = new Date()

 // 公開時刻が過ぎた予約投稿を取得（バッチサイズ制限：50件）
 const scheduledPosts = await prisma.scheduledPost.findMany({
 where: {
 status: 'pending',
 scheduledAt: { lte: now },
 },
 select: {
 id: true, userId: true, content: true,
 user: { select: { id: true, isSuspended: true } },
 media: { select: { url: true, type: true, sortOrder: true } },
 genres: { select: { genreId: true } },
 },
 take: 50,
 })

 let publishedCount = 0
 let failedCount = 0

 for (const scheduledPost of scheduledPosts) {
 try {
 // ユーザーが有効かチェック
 if (!scheduledPost.user || scheduledPost.user.isSuspended) {
 await prisma.scheduledPost.update({
 where: { id: scheduledPost.id },
 data: { status: 'failed' },
 })
 failedCount++
 continue
 }

 // トランザクションで投稿作成と予約投稿の更新をアトミックに実行
 await prisma.$transaction(async (tx) => {
 // 投稿を作成
 const post = await tx.post.create({
 data: { userId: scheduledPost.userId, content: scheduledPost.content },
 })
 })
 }
 }
 }
}
```



```

// メディアを作成
if (scheduledPost.media.length > 0) {
 await tx.postMedia.createMany({
 data: scheduledPost.media.map((m) => ({
 postId: post.id, url: m.url, type: m.type, sortOrder: m.sortOrder,
 })),
 })
}

// ジャンルを作成
if (scheduledPost.genres.length > 0) {
 await tx.postGenre.createMany({
 data: scheduledPost.genres.map((g) => ({
 postId: post.id, genreId: g.genreId,
 })),
 })
}

// 予約投稿のステータスを更新
await tx.scheduledPost.update({
 where: { id: scheduledPost.id },
 data: { status: 'published', publishedPostId: post.id },
})
publishedCount++
} catch (error) {
 // 個別のエラーをログして続行（他の投稿に影響させない）
 console.error(`Failed to publish post ${scheduledPost.id}:`, error)
 failedCount++
 await prisma.scheduledPost.update({
 where: { id: scheduledPost.id },
 data: { status: 'failed' },
 }).catch(() => {})
}
}

return NextResponse.json({
 success: true,
 publishedCount,
 failedCount,
})
} catch (error) {
 console.error('Cron job error:', error)
 return NextResponse.json({ error: 'Internal error' }, { status: 500 })
}
}

// Vercel Cron設定
export const dynamic = 'force-dynamic'
export const maxDuration = 60 // 60秒タイムアウト

```

## 設計のポイント

ポイント	説明
バッチサイズ制限	<code>take: 50</code> で一度に処理する数を制限。Vercelの実行時間制限（60秒）内に収める
個別エラーハンドリング	1件の失敗が他の投稿に影響しないよう、各投稿をtry-catchで囲む
トランザクション	投稿作成とステータス更新をアトミックに。途中で失敗しても不整合が起きない
ユーザー停止チェック	停止されたユーザーの予約投稿は <code>failed</code> にして公開しない

### 22.12.5 サブスクリプション期限管理（check-subscriptions）

毎日深夜1時に実行され、期限切れのプレミアム会員を処理します。

```
// app/api/cron/check-subscriptions/route.ts

// GET: 期限切れユーザーの処理
export async function GET(request: NextRequest) {
 // 認証チェック (省略)

 // 期限切れのプレミアム会員を取得
 const expiredUsers = await prisma.user.findMany({
 where: {
 isPremium: true,
 premiumExpiresAt: { lt: new Date() },
 },
 })

 // プレミアムステータスをリセット
 await prisma.user.updateMany({
 where: { id: { in: expiredUsers.map(u => u.id) } },
 data: { isPremium: false },
 })

 // 期限切れユーザーの予約投稿をキャンセル
 await prisma.scheduledPost.updateMany({
 where: {
 userId: { in: expiredUsers.map(u => u.id) },
 status: 'pending',
 },
 data: { status: 'cancelled' },
 })

 // メール通知を送信
 await Promise.allSettled(
 expiredUsers.map(user =>
 sendSubscriptionExpiredEmail(user.email, user.nickname)
)
)

 return NextResponse.json({ success: true, processedCount: expiredUsers.length })
}

// POST: 期限切れ間近 (3日以内) のユーザーに事前通知
export async function POST(request: NextRequest) {
 // 認証チェック (省略)

 const threeDaysLater = new Date(Date.now() + 3 * 24 * 60 * 60 * 1000)

 // 3日以内に期限切れになるユーザーを取得
 const expiringUsers = await prisma.user.findMany({
 where: {
 isPremium: true,
 premiumExpiresAt: { gt: new Date(), lte: threeDaysLater },
 },
 })
}
```

```
 })

 // メール通知+アプリ内通知を送信（重複通知を防ぐため24時間以内の通知を確認）
 for (const user of expiringUsers) {
 const existing = await prisma.notification.findFirst({
 where: {
 userId: user.id,
 type: 'subscription_expiring',
 createdAt: { gte: new Date(Date.now() - 24 * 60 * 60 * 1000) },
 },
 })
 if (!existing) {
 await prisma.notification.create({
 data: {
 userId: user.id,
 actorId: user.id,
 type: 'subscription_expiring',
 },
 })
 }
 }

 return NextResponse.json({ success: true, expiringCount: expiringUsers.length })
 }
}
```

### 22.12.6 イベントクリーンアップ (cleanup-events)

毎月1日の深夜0時に実行され、終了から6ヶ月以上経過したイベントを削除します。

```
// app/api/cron/cleanup-events/route.ts
export async function GET(request: NextRequest) {
 // シークレットキー検証
 if (!validateCronSecret(request)) {
 return NextResponse.json({ error: 'Unauthorized' }, { status: 401 })
 }

 try {
 // 6ヶ月前の日付を計算
 const sixMonthsAgo = new Date()
 sixMonthsAgo.setMonth(sixMonthsAgo.getMonth() - 6)

 // 終了日（または開始日）が6ヶ月以上前のイベントを削除
 const result = await prisma.event.deleteMany({
 where: {
 OR: [
 // 終了日がある場合：終了日が6ヶ月以上前
 { endDate: { not: null, lt: sixMonthsAgo } },
 // 終了日がない場合：開始日が6ヶ月以上前
 { endDate: null, startDate: { lt: sixMonthsAgo } },
],
 },
 })

 console.log(`[Cron] Deleted ${result.count} old events`)
 return NextResponse.json({
 success: true,
 deletedCount: result.count,
 cutoffDate: sixMonthsAgo.toISOString(),
 })
 } catch (error) {
 console.error('[Cron] Event cleanup error:', error)
 return NextResponse.json({ error: 'Internal error' }, { status: 500 })
 }
}
```

## 22.12.7 Cronジョブのモニタリング

各Cronジョブは実行結果をJSON形式で返すため、Vercelのダッシュボードやログで実行状況を確認できます。

```
// publish-scheduled の成功レスポンス例
{
 "success": true,
 "message": "Published 3 scheduled posts",
 "publishedCount": 3,
 "failedCount": 0
}

// check-subscriptions の成功レスポンス例
{
 "success": true,
 "message": "Processed 2 expired subscriptions",
 "processedCount": 2,
 "cancelledPostsCount": 5,
 "emailsSent": 2,
 "emailsFailed": 0
}

// cleanup-events の成功レスポンス例
{
 "success": true,
 "deletedCount": 15,
 "cutoffDate": "2025-08-10T00:00:00.000Z"
}
```

### Vercelダッシュボードでの確認方法

Vercelダッシュボード > プロジェクト > Settings > Cron Jobs

Job Name	Schedule	Last Run	Status
/api/cron/publish-scheduled	* / 5 * * * *	2 min ago	✓ Success
/api/cron/check-subscriptions	0 1 * * *	5 hours ago	✓ Success
/api/cron/cleanup-events	0 0 1 * *	15 days ago	✓ Success

▶ 理解度チェック（クリックして展開）

## 22.13 本番運用のチェックリストとパフォーマンス最適化

▶ このセクションで学ぶこと（クリックして展開）

## 22.13.1 デプロイ前チェックリスト

デプロイ前に以下の項目を確認しましょう。これは「出発前の持ち物チェック」のようなものです。忘れ物があると旅先で困るように、確認不足があると本番で問題が起きます。

デプロイ前チェックの流れ:

```
graph TD
 A[ローカル確認] --> B[CI/CD通過確認]
 B --> C[環境変数確認]
 C --> D[デプロイ実行]
 D --> E[デプロイ後スモークテスト]
```

### 必須チェック項目:

- ☐ 環境変数がすべて設定されている
  - DATABASE\_URL, NEXTAUTH\_SECRET, NEXTAUTH\_URL は特に重要
  - Vercelダッシュボードの Environment Variables で確認
- ☐ データベースマイグレーションが適用されている
  - `npx prisma migrate deploy` を実行済み
- ☐ テストがすべて通過している
  - GitHub ActionsのCIが全てグリーン（成功）
- ☐ ビルドエラーがない
  - ローカルで `npm run build` が成功することを確認
- ☐ Lighthouseスコアが良好（90点以上）
  - Chrome DevTools → Lighthouse タブで計測
- ☐ セキュリティヘッダーが設定されている
  - `X-Frame-Options`, `X-Content-Type-Options` 等
- ☐ CSP（Content Security Policy）が正しく設定されている
  - XSS攻撃を防ぐためのセキュリティポリシー

- ☐ robots.txtが設定されている
  - 検索エンジンのクロール制御
- ☐ sitemap.xmlが生成されている
  - 検索エンジンにページ構造を伝える
- ☐ OGP画像が設定されている
  - SNSでシェアされた時のプレビュー画像

## 22.13.2 デプロイ後チェックリスト（スモークテスト）

デプロイ後は「スモークテスト」を実施します。これは「新しい家に引っ越した後、電気・ガス・水道が使えるか確認する」ことに相当します。

- ☐ ヘルスチェックが通る
  - `curl https://your-domain.com/api/health` で 200 が返る
- ☐ ログインが正常に動作する
  - テストアカウントでログイン・ログアウト
- ☐ 主要機能が正常に動作する
  - 投稿の作成、コメント、いいね等
- ☐ エラー監視が有効になっている
  - Sentryダッシュボードで接続を確認
- ☐ SSL証明書が有効
  - ブラウザのアドレスバーに鍵アイコンが表示される
- ☐ カスタムドメインが正しく設定されている
  - `https://bonlog.app` でアクセスできる
- ☐ メール送信が動作する
  - テストメールが届く
- ☐ Stripe Webhookが動作する
  - Stripeダッシュボードでイベント配信を確認
- ☐ バックアップが設定されている
  - Supabaseのバックアップ設定を確認



### 22.13.3 定期メンテナンス

本番運用は「一度デプロイして終わり」ではありません。継続的なメンテナンスが必要です。

メンテナンススケジュール:

週次 (毎週月曜日)

+-----+	
- 依存関係の脆弱性チェック	
<code>npm audit</code> で確認	
- Sentryのエラー確認	
新しいエラーがないか確認	
+-----+	

月次 (毎月1日)

+-----+	
- パフォーマンスレビュー	
Lighthouseスコア、レスポンスタイム確認	
- ログの確認	
Vercel Functions のログを確認	
- 依存パッケージのアップデート	
<code>npm outdated</code> で確認	
+-----+	

四半期 (1月、4月、7月、10月)

+-----+	
- 不要なデータのクリーンアップ	
古いセッションデータ、一時ファイル等	
- バックアップの検証	
バックアップからの復元テスト	
- セキュリティ監査	
認証・認可の設定見直し	
+-----+	

- ☐ 週次: 依存関係の脆弱性チェック (`npm audit`)
- ☐ 月次: パフォーマンスレビュー (Lighthouse計測)
- ☐ 月次: ログの確認 (Vercel Functionsログ)
- ☐ 四半期: 不要なデータのクリーンアップ
- ☐ 四半期: バックアップの検証 (復元テスト)

### 22.13.4 パフォーマンス最適化

本番環境では、パフォーマンス (ページの表示速度) がユーザー体験に直結します。

## パフォーマンスが重要な理由:

表示速度	ユーザーの反応
1秒以内	快適に感じる
3秒以内	許容範囲
5秒以上	53%のユーザーが離脱
10秒以上	ほとんどのユーザーが離脱

## next.config.ts の最適化設定

```
// next.config.ts
// Next.jsアプリ全体のビルド・実行設定
import type { NextConfig } from 'next'

const nextConfig: NextConfig = {
 // === 画像最適化 ===
 images: {
 // formats: ブラウザが対応している場合に使用する画像形式
 // avif: 最も圧縮率が高い (JPEG比 約50%削減)
 // webp: 広くサポートされている (JPEG比 約25-35%削減)
 formats: ['image/avif', 'image/webp'],

 // remotePatterns: 外部画像の読み込みを許可するホスト
 // セキュリティ上、明示的に許可したホストのみ読み込み可能
 remotePatterns: [
 {
 protocol: 'https',
 // Cloudflare R2の画像を許可
 hostname: '*.r2.dev',
 },
],
 },

 // === 圧縮 ===
 // compress: レスポンスのgzip圧縮を有効化
 // 転送データ量を削減し、ページ表示を高速化
 compress: true,

 // === SWC圧縮 ===
 // swcMinify: Rust製のSWCコンパイラでJavaScriptを圧縮
 // Terserより高速で、ビルド時間を短縮
 swcMinify: true,

 // === Standalone出力 ===
 // output: Docker用。実行に必要な最小限のファイルのみ出力
 output: 'standalone',

 // === 実験的機能 ===
 experimental: {
 // ppr (Partial Prerendering): 部分的事前レンダリング
 // ページの静的部分を事前生成し、動的部分だけをリクエスト時に生成
 // これにより、初期表示が高速化される
 ppr: true,
 },
}

export default nextConfig
```

## 画像最適化

Next.jsの `Image` コンポーネントは、画像を自動的に最適化します。

```
import Image from 'next/image'

// ===== ヒーロー画像（ページ上部の大きな画像） =====
// priority: ページのLCP（Largest Contentful Paint）要素に指定
// LCPとは「ページで最も大きな要素が表示されるまでの時間」
// priority を付けると、他のリソースより優先的に読み込まれる
<Image
 src="/hero.jpg" // 画像のパス
 alt="Hero" // 代替テキスト（アクセシビリティ用）
 width={1200} // 画像の幅（px）
 height={630} // 画像の高さ（px）
 priority // 優先読み込みを有効化
/>

// ===== 投稿の画像（レスポンス対応） =====
// sizes: ブラウザの画面幅に応じて適切なサイズの画像を読み込む
// "(max-width: 768px) 100vw": スマホではビューポート幅の100%
// "600px": PC では 600px の画像を読み込む
// これにより、スマホでは小さい画像、PCでは大きい画像を配信
<Image
 src={post.imageUrl} // 動的な画像URL
 alt={post.title} // 投稿タイトルを代替テキストに
 width={600} // 画像の幅
 height={400} // 画像の高さ
 sizes="(max-width: 768px) 100vw, 600px" // レスponsiveサイズ
/>
```

### 画像最適化の効果:

最適化なし:

元画像: 2MB (JPEG)  
 全画面サイズで読み込み  
 即座に読み込み  
 表示速度: 遅い

最適化あり (next/image):

自動変換: 200KB (WebP/AVIF)  
 画面幅に応じて適切なサイズ  
 遅延読み込み (スクロールで表示時)  
 表示速度: 10倍高速

## フォント最適化

```
// app/layout.tsx
// Googleフォントを最適化して読み込む
import { Noto_Sans_JP } from 'next/font/google'

// Noto_Sans_JP: 日本語対応のGoogleフォント
const notoSansJP = Noto_Sans_JP({
 // subsets: 読み込む文字セット（使用する文字だけ読み込んで軽量化）
 subsets: ['latin'],

 // display: 'swap' はフォント読み込み中にシステムフォントを表示し、
 // 読み込み完了後に切り替える設定
 // ユーザーはテキストをすぐ読める（FOUT: Flash of Unstyled Text）
 display: 'swap',

 // variable: CSS変数名。Tailwind CSSなどで参照するために使う
 variable: '--font-noto-sans-jp',
})

export default function RootLayout({ children }) {
 return (
 // CSS変数をhtml要素に適用
 <html lang="ja" className={notoSansJP.variable}>
 <body>{children}</body>
 </html>
)
}
```

**フォント最適化の仕組み:** `next/font` はビルド時にフォントファイルをダウンロードし、自分のサーバーから配信します。Googleフォントのサーバーへの外部リクエストがなくなるため、プライバシー保護とパフォーマンス向上の両方を実現します。

▶ 理解度チェック（クリックして展開）

## 22.14 演習問題

この章で学んだ内容を実践するための演習問題です。難易度別に3段階に分かれています。

難易度	説明
基礎	手順通りに進めれば完成できる
応用	学んだ知識を組み合わせで解く
チャレンジ	追加の調査・設計力が必要

## 演習1: 環境変数の整理と確認 [基礎]

本番デプロイに必要な環境変数の一覧を整理し、設定状態を確認するスクリプトを作成してください。

### 要件:

- 必要な環境変数を一覧にまとめる
- 各環境変数が設定されているか確認するスクリプトを作成
- 未設定の変数があればエラーメッセージを表示

### ヒント:

```
// scripts/check-env.ts
// 必要な環境変数のリスト（変数名と説明）
const requiredEnvVars = [
 // name: 環境変数名
 // description: この環境変数の用途の説明
 { name: 'DATABASE_URL', description: 'PostgreSQLデータベースの接続文字列' },
 { name: 'NEXTAUTH_SECRET', description: 'NextAuth.jsの暗号化キー' },
 { name: 'NEXTAUTH_URL', description: 'アプリケーションのベースURL' },
 // ... 他の環境変数を追加
]

// 各環境変数の設定状態をチェック
let hasError = false
for (const envVar of requiredEnvVars) {
 // process.env から値を取得
 if (!process.env[envVar.name]) {
 // 未設定の場合はエラーを表示
 console.error(`[未設定] ${envVar.name}: ${envVar.description}`)
 hasError = true
 } else {
 // 設定済みの場合は確認メッセージ
 console.log(`[OK] ${envVar.name}`)
 }
}

// 未設定があれば終了コード1で終了（CIで失敗として扱われる）
if (hasError) {
 process.exit(1)
}
```

#### 完成の目安:

- 10個以上の環境変数をチェックできること
- CIパイプラインに組み込める形式であること

## 演習2: ステージング環境の構築 [応用]

本番環境とは別に、ステージング環境を構築してください。

ステージング環境の位置づけ:

開発環境 (localhost)	ステージング環境 (staging.bonlog.app)	本番環境 (bonlog.app)
+-----+	+-----+	+-----+
開発者が	本番と同じ構成で	ユーザー
自由に	→   テスト・QA	→   が利用
開発	チームで確認	
+-----+	+-----+	+-----+
develop ブランチ	develop ブランチ で自動デプロイ	main ブランチ で自動デプロイ

#### 要件:

- Vercelでステージング用プロジェクトを作成
- GitHubの `develop` ブランチと連携
- ステージング専用のSupabaseデータベースを使用
- 環境変数を本番と分離（特にStripeはテストモードを使用）
- ステージング環境のURL: `staging.bonlog.app`

#### ステップバイステップのガイド:

1. Supabaseで新しいプロジェクト `bonlog-staging` を作成
2. Vercelで新しいプロジェクトをインポート（同じリポジトリ）
3. Vercelの「Git」設定で Production Branch を `develop` に変更
4. 環境変数を設定（ステージング用のDB接続文字列、テスト用Stripeキー等）
5. `develop` ブランチにpushしてデプロイを確認

## 演習3: ロールバック手順の作成 [応用]

デプロイに問題があった場合のロールバック手順を文書化してください。

#### 要件:

- Vercelでの前バージョンへの切り戻し手順
- データベースマイグレーションのロールバック手順
- 環境変数の復元手順
- ダウンタイムを最小化する手順
- チームへの通知方法（テンプレート作成）



**ヒント: Vercelのロールバック手順:**

Vercelでのロールバック:

1. Vercelダッシュボード → Deployments
2. 正常に動作していたデプロイを選択
3. " ... " メニュー → "Promote to Production"
4. 確認画面で "Promote" をクリック

所要時間: 約30秒~1分

**作成するドキュメントの構成例:**

1. 障害検知 → 2. 影響範囲の確認 → 3. ロールバック判断  
→ 4. ロールバック実行 → 5. 動作確認 → 6. チーム通知  
→ 7. 原因調査 → 8. 再発防止策

## 演習4: ヘルスチェックAPIの作成 [応用]

アプリケーションの健全性を確認するヘルスチェックAPIを作成してください。

**要件:**

- `/api/health` エンドポイントを作成
- データベース接続の確認
- Redis接続の確認
- レスポンスタイムの計測
- 異常時は適切なHTTPステータスコードを返す

**ヒント:**

```
// app/api/health/route.ts
import { NextResponse } from 'next/server'
import { prisma } from '@lib/db'

export async function GET() {
 // 各サービスの状態を格納するオブジェクト
 const health = {
 status: 'ok' as 'ok' | 'error', // 全体のステータス
 timestamp: new Date().toISOString(), // チェック実行時刻
 services: {} as Record<string, any>, // 各サービスの状態
 }

 // データベース接続チェック
 try {
 const start = Date.now() // 開始時刻
 await prisma.$queryRaw`SELECT 1` // 最小限のクエリ
 const duration = Date.now() - start // 所要時間
 health.services.database = {
 status: 'ok',
 responseTime: `${duration}ms`,
 }
 } catch (error) {
 health.status = 'error' // 全体をエラーに
 health.services.database = {
 status: 'error',
 message: 'Database connection failed',
 }
 }

 // ステータスに応じたHTTPコードを返す
 // 200: 正常、503: サービス利用不可
 const statusCode = health.status === 'ok' ? 200 : 503
 return NextResponse.json(health, { status: statusCode })
}
```

## 演習5: GitHub Actionsのカスタマイズ [応用]

CI/CDパイプラインに以下の機能を追加してください。

要件:

- ビルドサイズが前回より10%以上増加した場合に警告を出す
- テストカバレッジが80%未満の場合にCIを失敗させる
- デプロイ成功時にSlack通知を送信する

**ヒント（Slack通知）：**

```
Slack通知のステップ例
- name: Slack Notification
 # デプロイ成功時のみ実行
 if: success()
 uses: 8398a7/action-slack@v3
 with:
 # 通知のステータス
 status: ${ job.status }
 # 通知メッセージに含めるフィールド
 fields: repo,message,commit,author,action,eventName,workflow
 env:
 # GitHubトークン（自動で提供される）
 GITHUB_TOKEN: ${ secrets.GITHUB_TOKEN }
 # Slack Webhook URL（Secretsに設定）
 SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK_URL }
```

**演習6: モニタリングダッシュボード [チャレンジ]**

本番環境の健全性を監視するダッシュボードを作成してください。

ダッシュボードのイメージ:

```

+-----+
| BON-LOG 監視ダッシュボード |
+-----+
|
| [全体ステータス: 正常]
|
| レスポンスタイム (過去24時間)
| 300ms | *
| 200ms | * * * *
| 100ms | * * * * * * *
| +---+---+---+---+---+---+---+---+---+ 時間
|
| エラー率 アクティブユーザー
| 1時間: 0.02% 現在: 142人
| 24時間: 0.05% 今日: 1,247人
| 7日間: 0.03% 今週: 5,831人
|
| DB接続: 12/100 Redis: 45MB/256MB
|
+-----+

```

#### 要件:

- `/admin/monitoring` ページを作成
- サーバレスポンスタイムのグラフ
- エラー率 (直近1時間、24時間、7日間)
- アクティブユーザー数
- データベース接続プール状態
- Redisメモリ使用量
- グラフ表示 (Chart.js または Recharts使用)
- 管理者のみアクセス可能 (認証チェック)

#### ヒント:

```
// lib/monitoring.ts
// システムメトリクスを取得する関数

export async function getSystemMetrics() {
 // Promise.all で並列実行（個別に await するより高速）
 const [
 errorRate, // エラー率
 responseTime, // 平均レスポンスタイム
 activeUsers, // アクティブユーザー数
 dbConnections // DB接続プール状態
] = await Promise.all([
 getErrorRate(), // Sentryから取得
 getAverageResponseTime(), // Vercel Analyticsから取得
 getActiveUserCount(), // DBから直近のアクティブセッション数
 getDatabasePoolStatus() // Prismaのメトリクスから取得
])

 return { errorRate, responseTime, activeUsers, dbConnections }
}

// エラー率の取得例
async function getErrorRate() {
 // 直近1時間のリクエスト数とエラー数から計算
 const oneHourAgo = new Date(Date.now() - 60 * 60 * 1000)
 const totalRequests = await getRequestCount(oneHourAgo)
 const errorRequests = await getErrorCount(oneHourAgo)
 return totalRequests > 0 ? errorRequests / totalRequests : 0
}
```

## 演習7: ゼロダウンタイムデプロイの実現 [チャレンジ]

データベースのマイグレーションを含むデプロイでも、ダウンタイム（サービス停止時間）をゼロにする戦略を設計・実装してください。

### 要件:

- 後方互換性のあるマイグレーション戦略の設計
- 段階的なスキーマ変更の実装
- ブルーグリーンデプロイメントの概念を活用

### ヒント:

ゼロダウンタイムのスキーマ変更（カラム名変更の例）：

- ステップ1: 新しいカラムを追加（旧カラムは残す）
- ステップ2: アプリを更新（新旧両方のカラムに書き込む）
- ステップ3: 既存データを新カラムにコピー
- ステップ4: アプリを更新（新カラムのみ使用）
- ステップ5: 旧カラムを削除

各ステップを別々のデプロイで実行することで、  
どの時点でもアプリが正常に動作する

## 22.15 よくある質問（FAQ）

この章の内容に関して、初学者からよく寄せられる質問とその回答をまとめました。デプロイ、CI/CD、本番運用に関する疑問を解消するために活用してください。

FAQの分類	
[一般・概念]	デプロイやCI/CDの基本的な疑問
[Vercel]	Vercelデプロイに関する質問
[Supabase/DB]	データベース運用の質問
[GitHub Actions]	CI/CDパイプラインの質問
[Docker]	コンテナ運用の質問
[Sentry]	エラー監視の質問
[PWA]	Service Workerとオフライン対応の質問
[SEO]	検索エンジン最適化の質問
[セキュリティ]	本番環境のセキュリティの質問
[パフォーマンス]	Web Vitalsと最適化の質問
[トラブル]	よくあるトラブルと解決方法

### 一般・概念に関するFAQ

**Q1: デプロイとリリースは同じ意味ですか？**

A: 厳密には異なります。

**デプロイ (Deploy) :**

- コードをサーバーに配置して動作可能にすること
- サーバーにファイルをアップロードして起動する作業

**リリース (Release) :**

- ユーザーが実際に新機能を使えるようにすること
- デプロイ後に機能フラグをONにする等

デプロイ = 配置      「料理をキッチンからカウンターに置く」  
 リリース = 提供      「カウンターからお客様のテーブルに運ぶ」

一般的にはほぼ同義で使われますが、  
 大規模サービスでは分離されることがあります。  
 例: コードはデプロイ済みだが、フィーチャーフラグで  
 一部ユーザーにのみ段階的にリリースする

**Q2: ステージング環境は必ず必要ですか？**

A: 個人開発や小規模プロジェクトでは必須ではありませんが、チーム開発や重要なサービスでは強く推奨されます。

**プロジェクト規模ごとの環境構成の目安:****【個人学習・プロトタイプ】**

- 開発(localhost) → 本番(Vercel)
- ステージング不要。本番デプロイ前にローカルで確認すれば十分

**【個人開発・小規模サービス】**

- 開発(localhost) → プレビュー(Vercel Preview) → 本番(Vercel)
- VercelのPRごとのプレビューデプロイで代用可能

**【チーム開発・商用サービス】**

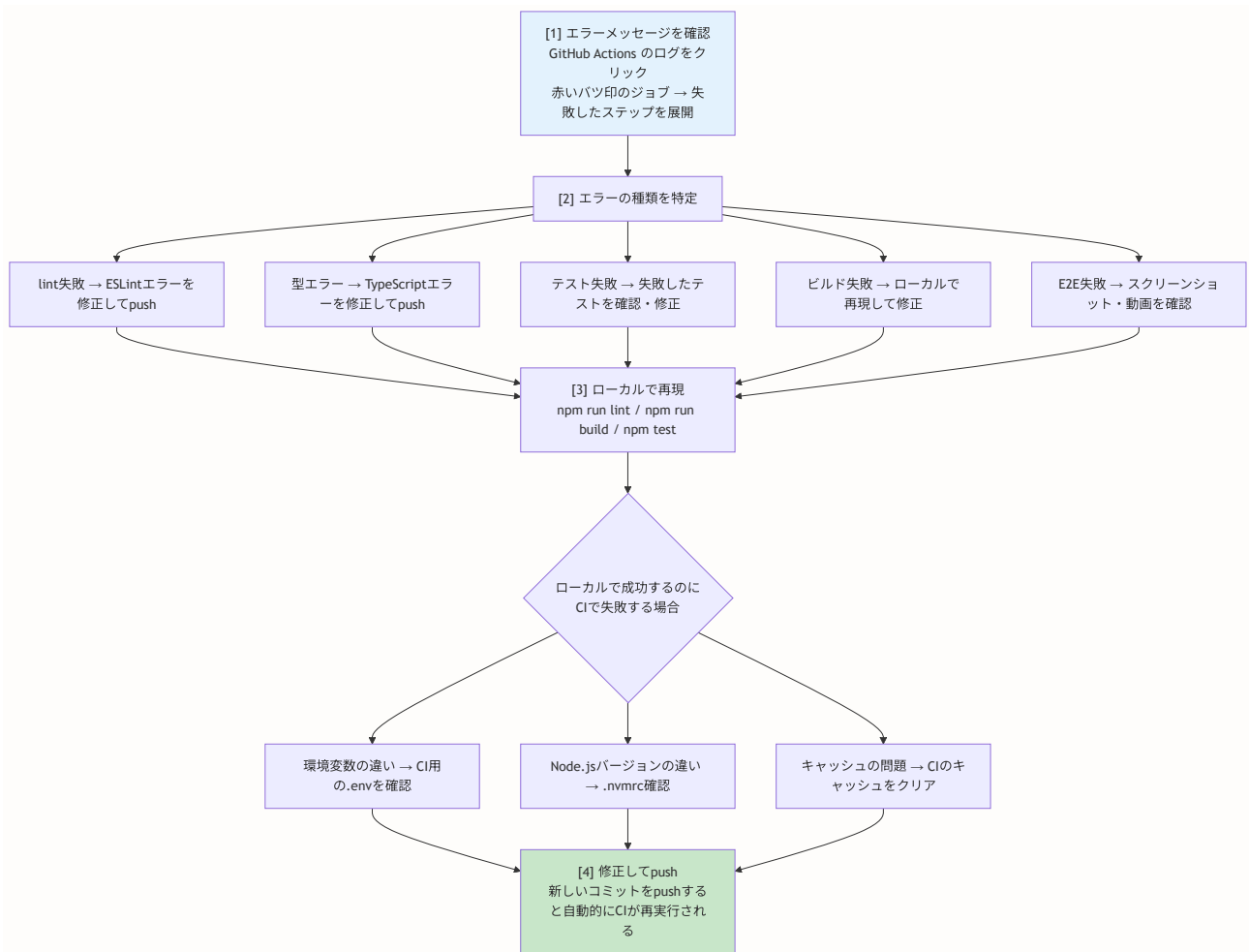
- 開発(localhost) → ステージング → 本番
- 専用のステージング環境を構築。QAチームが事前に検証

**【大規模サービス】**

- 開発 → テスト → ステージング → カナリア → 本番
- 段階的にトラフィックを増やしながら検証

**Q3: CI/CDパイプラインが失敗したときはどうすればよいですか？**

A: 以下の手順で対処します。



**Q4: 「デプロイ」と「ビルド」の違いは何ですか？**

A: ビルドはコードを実行可能な形に変換するプロセス、デプロイはビルド成果物をサーバーに配置して公開するプロセスです。



ソースコード → ビルド → 成果物 → デプロイ → 公開  
(TypeScript) (変換) (.next/) (配置) (アクセス可能)

ビルド (Build) の具体的な処理:

1. TypeScript → JavaScript に変換 (トランスパイル)
2. コードの圧縮・最適化 (ミニファイ・バンドル)
3. 静的ページの事前生成 (SSG)
4. CSS の処理 (Tailwind CSSのページ等)
5. 画像の最適化
6. ルートマニフェストの生成

デプロイ (Deploy) の具体的な処理:

1. ビルド成果物をサーバーにアップロード
2. 環境変数の設定
3. サーバーの起動・再起動
4. CDNキャッシュの更新
5. ヘルスチェック
6. DNSの切り替え (初回のみ)

## Vercelに関するFAQ

**Q5: Vercelの無料プランでどこまでできますか？**

A: 個人プロジェクトであれば、かなりの規模まで対応できます。

Vercel Hobbyプラン (無料) の主な制限:

項目	制限値
帯域幅	100GB/月
Serverless Function実行時間	10秒（デフォルト）
ビルド時間	6,000分/月
デプロイ数	100回/日
チームメンバー	1人（個人のみ）
プレビューデプロイ	制限なし
カスタムドメイン	制限なし
自動HTTPS	対応
環境変数	制限なし
Cron Jobs	2つまで（日次）

**注意点:**

- 商用利用にはProプラン（\$20/月）が推奨
- Serverless Functionは10秒を超えるとタイムアウト → Proプランでは60秒まで延長可能
- 帯域幅100GBを超えると追加課金 → 画像最適化やCDNキャッシュを活用して節約

**Q6: Vercelのプレビューデプロイとは何ですか？**

A: プルリクエスト（PR）を作成するたびに、そのPRの変更を反映した専用のURLが自動生成される機能です。

プレビューデプロイの流れ:



利点:

- ・ レビューアーがコードを読むだけでなく、実際に操作して確認できる
- ・ デザインの確認がスクリーンショットなしでできる
- ・ 本番環境に影響を与えずにテストできる
- ・ PRをマージするまで本番には反映されない

**Q7: Vercelでデプロイが失敗した場合、ユーザーに影響はありますか？**

A: 現在動作中のバージョンには影響しません。

Vercelのデプロイモデル:

【現在の本番】 ← ユーザーがアクセスしている  
 【新しいビルド】 ← バックグラウンドで実行

ビルド成功の場合:

新しいバージョンに自動的に切り替え  
 (アトミックデプロイ: 一瞬で切り替わる)

ビルド失敗の場合:

現在の本番はそのまま稼働し続ける  
 ← ユーザーには一切影響なし  
 開発者にエラー通知が送信される

これをイミュータブルデプロイ (Immutable Deploy) と呼びます。  
 各デプロイは独立しており、失敗しても既存環境を壊しません。

## Supabase/データベースに関するFAQ

### Q8: SupabaseとローカルのDockerデータベースの切り替えはどうしますか？

A: 環境変数 `DATABASE_URL` を切り替えるだけです。

```
// .env.local (開発環境 - Docker PostgreSQL)
DATABASE_URL="postgresql://postgres:postgres@localhost:5432/bonsai_sns"

// .env.production (本番環境 - Supabase)
DATABASE_URL="postgresql://postgres:xxxxx@db.xxxx.supabase.co:5432/postgres"

// 切り替えの仕組み:
// Prismaは起動時に DATABASE_URL を読み込む
// → 接続先が変わるだけで、コードの変更は不要
// → 同じ prisma.user.findMany() がどちらのDBにも対応

// ローカルとSupabaseのスキーマを同期する方法:
// 1. ローカルで schema.prisma を編集
// 2. npx prisma db push (ローカルに反映)
// 3. 確認後、DATABASE_URL を Supabase に切り替え
// 4. npx prisma migrate deploy (Supabase に反映)
```

### Q9: データベースのマイグレーションで失敗した場合、元に戻せますか？

A: Prisma Migrateには自動ロールバック機能はありませんが、手動で対応できます。

マイグレーション失敗時の対処:

【開発環境】

```
npx prisma db push --force-reset
```

→ データベースをリセットして最初からやり直し

→ 開発データは失われるが、seedデータで復元可能

【本番環境】

1. 失敗したマイグレーションの確認

```
npx prisma migrate status
```

2. 手動で逆のSQLを実行

例: ADD COLUMN で失敗 → DROP COLUMN を手動実行

```
psql -h db.xxxx.supabase.co -U postgres -c "ALTER TABLE users DROP COLUMN new_column;"
```

3. マイグレーション履歴を修正

```
npx prisma migrate resolve --rolled-back "migration_name"
```

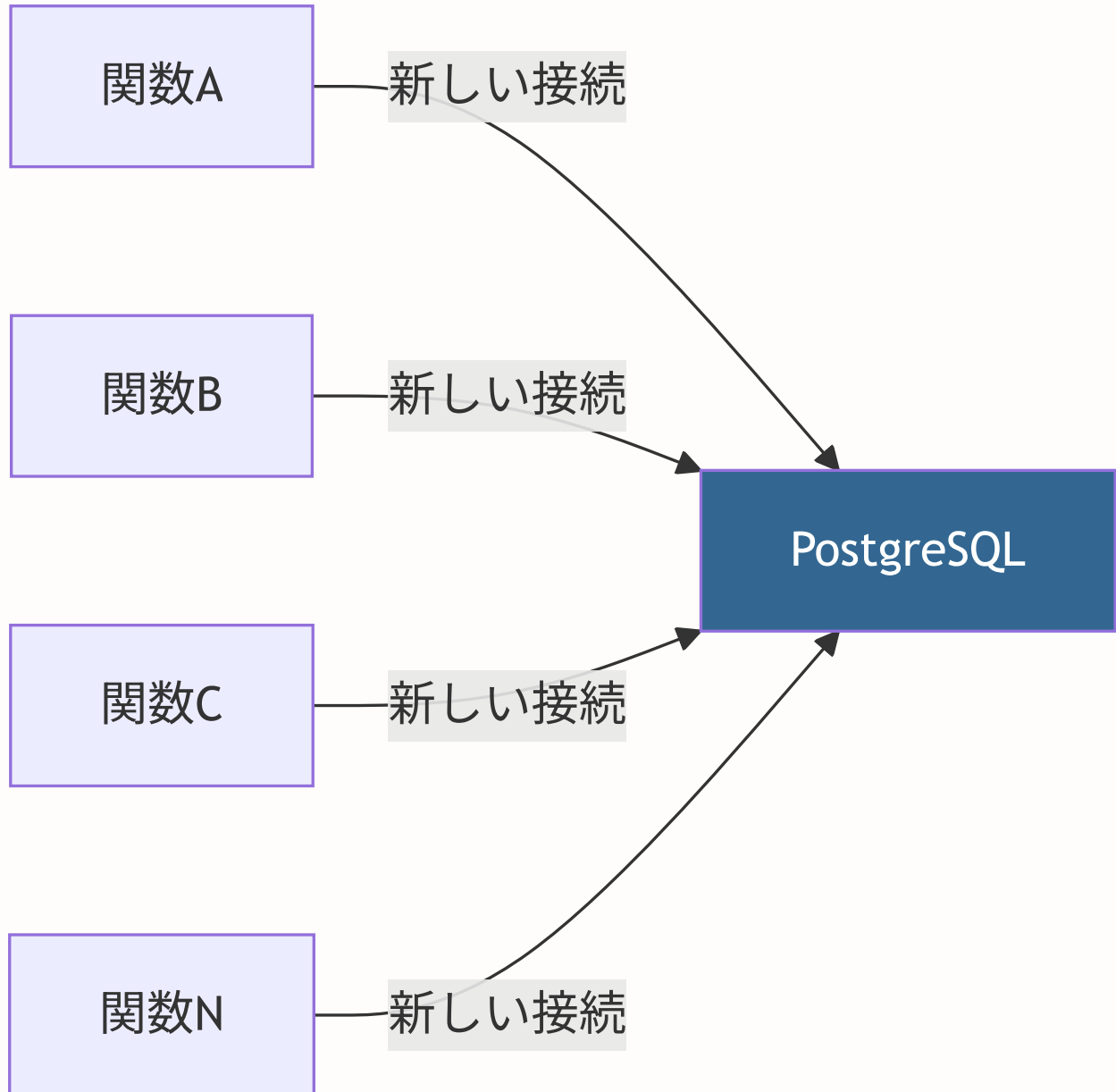
【ベストプラクティス】

- ・本番マイグレーション前にバックアップを取る
- ・破壊的変更（カラム削除等）は段階的に実行
- ・ステージングで事前にテスト

## Q10: Supabaseの接続プール（PgBouncer）とは何ですか？なぜ必要ですか？

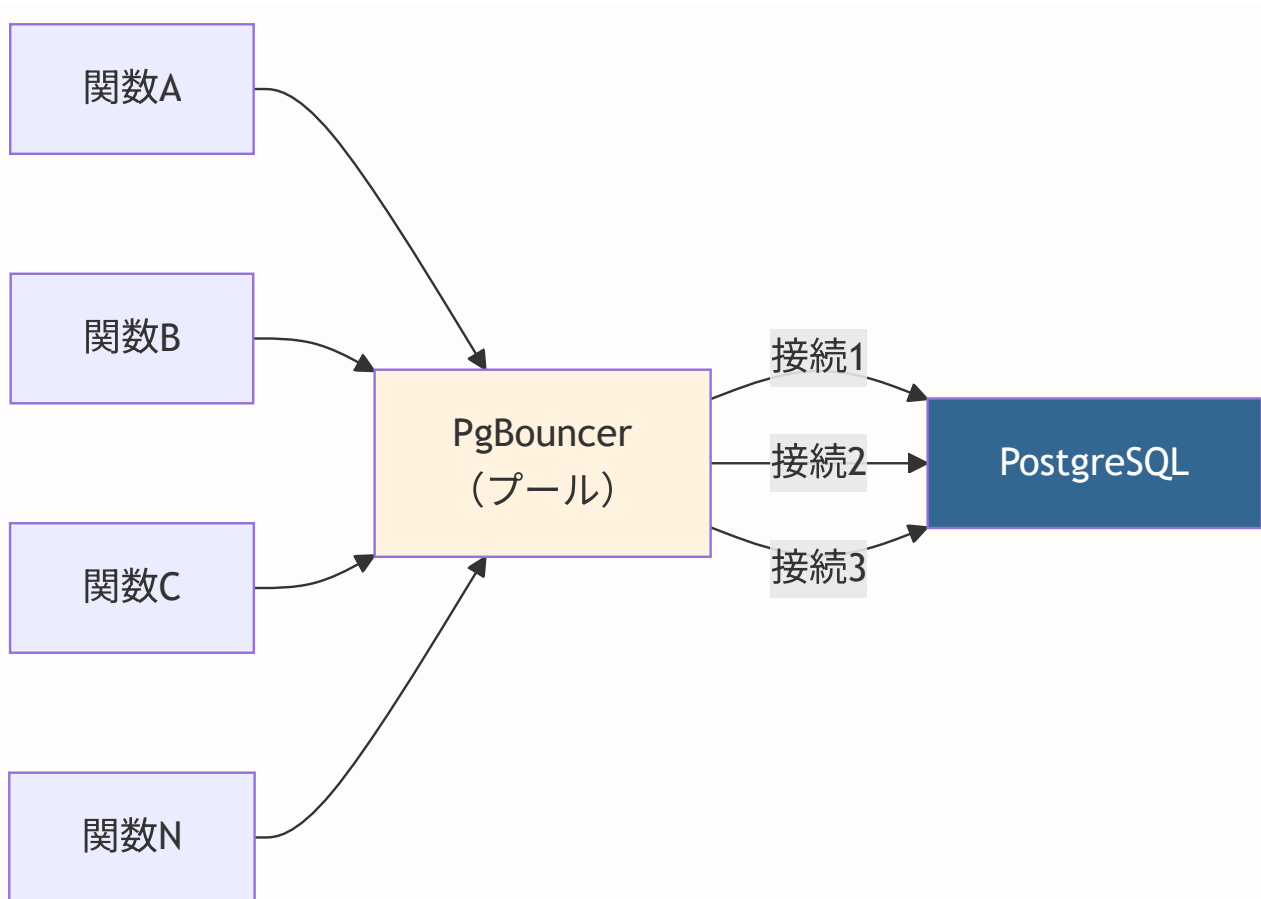
A: 接続プールは、データベースへの接続を使い回す仕組みです。Serverless環境では特に重要です。

接続プールなしの場合:



問題: 関数のたびに新しい接続が作られる → PostgreSQLの最大接続数（通常100程度）にすぐ到達 → 「too many connections」エラーが発生

**接続プールありの場合:**



解決: PgBouncerが接続を管理・再利用 → 実際のDB接続は少数で済む → Serverless環境でも安定動作

Supabaseでの設定	接続方式	用途
DATABASE_URL	PgBouncer経由 (ポート6543)	prisma.\$queryRaw、通常のCRUD操作
DIRECT_URL	直接接続 (ポート5432)	prisma migrate、スキーマ変更

## GitHub Actionsに関するFAQ

**Q11: GitHub Actionsのキャッシュが原因でCIが失敗することはありませんか？**

A: はい、キャッシュの破損や古いキャッシュが原因で失敗することがあります。

キャッシュ関連の問題と対処法:

【症状1】 ローカルでは成功するがCIで失敗する

→ `node_modules`のキャッシュが古い

対処: キャッシュキーに`package-lock.json`のハッシュを含める

# `ci.yml` のキャッシュ設定例

- uses: actions/cache@v4

with:

path: ~/.npm

key: \${{ runner.os }}-node-\${{ hashFiles('\*\*/package-lock.json') }}

# ↑ `package-lock.json` が変わるとキャッシュが更新される

【症状2】 同じコードなのにCIの結果が変わる

→ `npm install` で最新のパッチバージョンが入る

対処: `package-lock.json` をコミットし、`npm ci` を使用

【症状3】 キャッシュの容量制限に達する

→ GitHub Actionsのキャッシュは10GBまで

対処: 不要なキャッシュは自動的に削除される

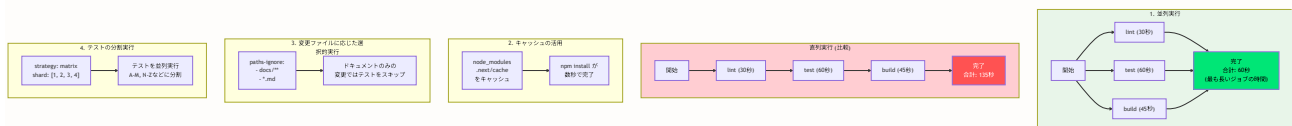
(7日間使われなかったキャッシュは削除)

[キャッシュを手動でクリアする方法]

GitHub → Settings → Actions → Caches → 削除

Q12: CIの実行時間を短縮するにはどうすればよいですか？

A: いくつかの手法があります。



## Dockerに関するFAQ

Q13: Docker Composeでデータベースだけ起動しているのに、アプリ全体をDockerで動かすこともできますか？

A: はい、BON-LOGの`docker-compose.yml`はプロファイルを使って切り替えられます。



3つの使い方:

[1] DBのみ (推奨開発スタイル)

```
docker compose up -d postgres
```

`npm run dev` ← ローカルでNext.jsを起動  
→ ホットリロードが速い、デバッグしやすい

[2] DB + Next.js開発モード

```
docker compose --profile dev up -d
```

→ すべてコンテナ内で動作。環境を揃えたい場合に

[3] DB + Next.js本番ビルド

```
docker compose --profile prod up -d
```

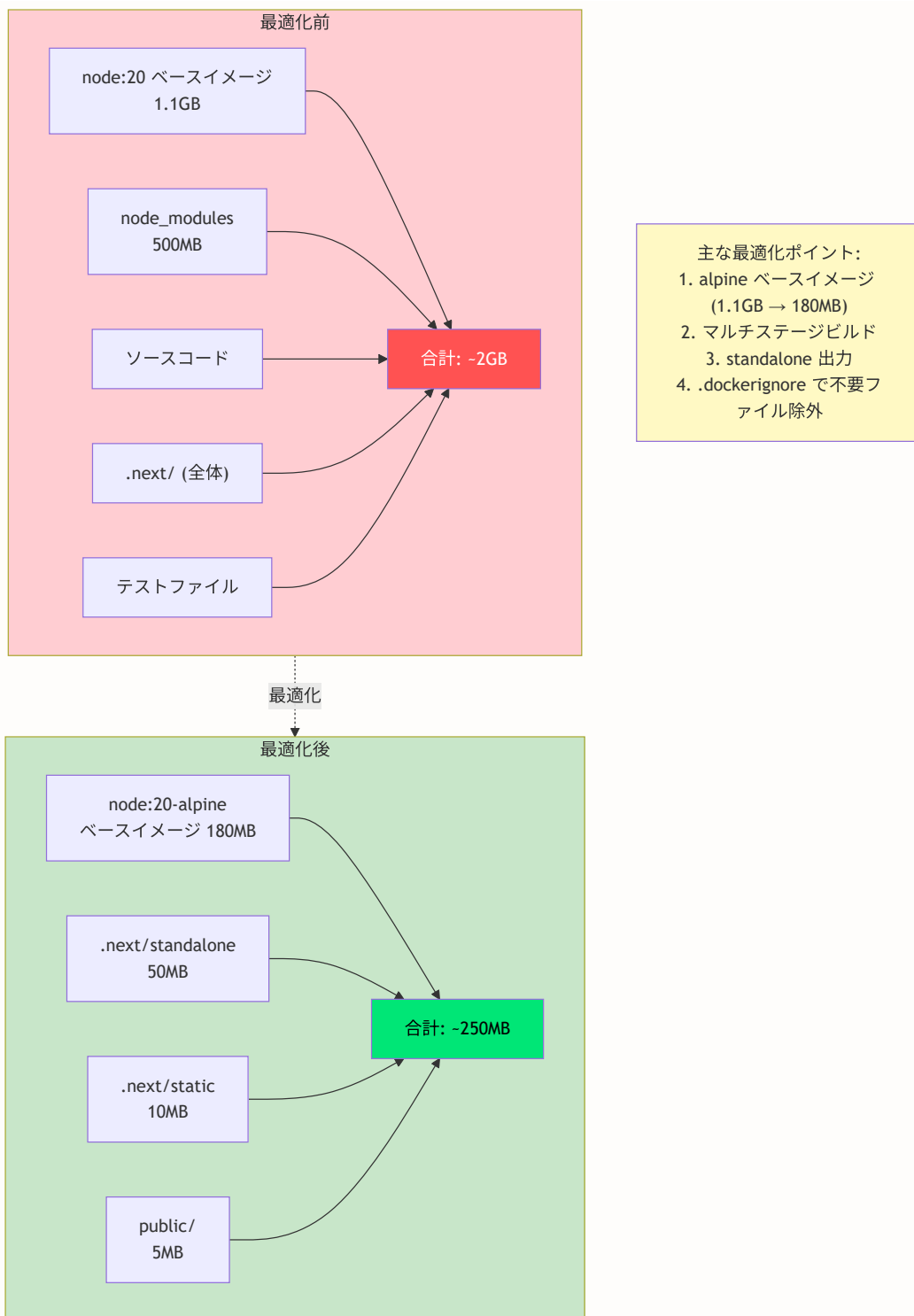
→ 本番と同じイメージで動作確認

使い分けの判断基準:

- ・ 普段の開発 → [1] DBのみ
- ・ チームで環境を揃えたい → [2] dev プロファイル
- ・ デプロイ前の最終確認 → [3] prod プロファイル
- ・ CI/CDでのテスト → [1] DBのみ + `npm test`

**Q14: Dockerイメージのサイズが大きくなってしまった場合の対処法は？**

A: マルチステージビルドと`.dockerignore`が重要です。



## Sentryに関するFAQ

**Q15: Sentryの無料プランでどれくらいのイベントを処理できますか？**

A: Developer（無料）プランでは月5,000イベントまでです。

プラン	イベント/月	料金	メンバー
Developer	5,000	無料	1人
Team	50,000	\$26/月～	無制限
Business	100,000	\$80/月～	無制限

5,000イベントの目安:

- エラー + パフォーマンスで5,000件
- `tracesSampleRate: 0.1` の場合、50,000リクエストのうち5,000件がパフォーマンスイベント
- エラーは全件送信なので、エラーが多いとすぐ上限に

イベント消費を抑えるコツ:

1. `tracesSampleRate` を低く設定 (0.05～0.1)
2. `beforeSend` でノイズエラーを除外
3. `ignoreErrors` で既知の無害エラーを除外
4. クライアントサイドの `enabled` を制御

**Q16: SentryのbeforeSendでエラーを除外しすぎると問題になりますか？**

A: はい、重要なエラーを見逃す可能性があります。

エラーフィルタリングのバランス:

【除外すべきエラー】

- ・ブラウザ拡張機能由来のエラー (`chrome-extension://`)
- ・サードパーティアプリのWebView由来のエラー
- ・ネットワークの一時的な切断エラー
- ・Next.jsの内部的なリダイレクトエラー (`NEXT_REDIRECT`)
- ・`ResizeObserver` loopエラー (UIに影響しない)

【除外してはいけないエラー】

- ・自分のコードのバグ (`TypeError`, `ReferenceError`等)
- ・APIのレスポンスエラー (500系)
- ・認証・認可の不正なエラー
- ・データベース接続のエラー

判断基準:

「このエラーをユーザーが報告してきたら、対処が必要か？」

- はい → Sentryに送信する
- いいえ → 除外してもよい

**Q17: ソースマップをSentryにアップロードする具体的な手順は？**

A: `next.config.ts` の `withSentryConfig` で自動的にアップロードされます。

```
// next.config.ts での設定（BON-LOGの実際の設定）

export default withSentryConfig(nextConfig, {
 // Sentry組織とプロジェクト設定
 org: process.env.SENTRY_ORG, // Sentryの組織スラッグ
 project: process.env.SENTRY_PROJECT, // プロジェクトスラッグ

 // ソースマップのアップロード設定
 silent: !process.env.CI, // CI以外ではログを抑制
 widenClientFileUpload: true, // クライアントファイルの検出範囲を拡大

 // 不要ファイルを除外（警告抑制）
 sourcemaps: {
 ignore: [
 '**/node_modules/**',
 '**/*_client-reference-manifest*',
 '**/*-manifest.js',
],
 },

 // Ad blockerを回避するトンネルルート
 // /monitoring へのリクエストが Sentry に転送される
 tunnelRoute: '/monitoring',
})
```

ソースマップのアップロードの流れ:

```
npm run build
|
v
Next.jsがソースマップを生成 (.map ファイル)
|
v
@sentry/nextjs がソースマップを検出
|
v
Sentry CLIがソースマップをSentryにアップロード
|
v
本番のバンドルファイルにはソースマップを含めない
→ ユーザーからはソースコードが見えない
→ Sentryダッシュボードでは元のTypeScriptソースで表示
```

必要な環境変数:

```
SENTRY_ORG=your-org
SENTRY_PROJECT=your-project
SENTRY_AUTH_TOKEN=sentrys_xxxxx ← Sentry APIトークン
```

## PWAに関するFAQ

**Q18: Service Workerを導入するデメリットはありますか？**

A: いくつかの注意点があります。

**Service Worker導入のメリットとデメリット:****[メリット]**

- ・オフラインでもキャッシュ済みページを表示できる
- ・リピータアクセスが高速化（キャッシュからの読み込み）
- ・プッシュ通知が可能（将来機能）
- ・ホーム画面への追加が可能

**[デメリット・注意点]**

- ・キャッシュの古さ問題
  - ユーザーが古いデータを見てしまう可能性
  - 対策: 適切なキャッシュ戦略の選択
- ・開発時の混乱
  - キャッシュが原因でコード変更が反映されない
  - 対策: 開発環境ではSW無効化（BON-LOGでは実装済み）
- ・デバッグの複雑さ
  - エラーの原因がSWか通常のコードか判別しにくい
  - 対策: Chrome DevTools > Application > Service Workers で制御
- ・ストレージ容量
  - キャッシュが肥大化するとデバイスの容量を圧迫
  - 対策: キャッシュのバージョン管理と古いキャッシュの自動削除

**BON-LOGでの対策:**

- ・開発環境では自動無効化
- ・CACHE\_VERSION でバージョン管理
- ・認証APIはキャッシュしない（NETWORK\_ONLY\_PATTERNS）
- ・更新プロンプトで手動更新を促す

**Q19: PWAとネイティブアプリの違いは何ですか？**

A: PWAはウェブ技術で作られたアプリで、アプリストアを経由せずにインストールできます。

項目	PWA	ネイティブ
インストール	ブラウザから	アプリストア
開発言語	HTML/CSS/JS	Swift/Kotlin
オフライン	対応（制限あり）	完全対応
カメラ	対応	対応
プッシュ通知	対応	対応
GPSの利用	対応	対応
ファイルアクセス	制限あり	フルアクセス
アプリストア掲載	不要	必要
審査	不要	必要
更新	自動	手動DL必要
開発コスト	低	高
パフォーマンス	やや劣る	最適化可能

BON-LOGがPWAを選択した理由:

- ウェブ開発の技術スタックで完結できる
- アプリストアの審査が不要で素早くリリース可能
- 1つのコードベースで全プラットフォーム対応
- オフラインでのタイムライン閲覧が十分にカバーできる

## SEOに関するFAQ

**Q20: サイトマップは手動で更新する必要がありますか？**

A: いいえ、BON-LOGでは自動生成されます。

```
// app/sitemap.ts の仕組み:

// export const dynamic = 'force-dynamic' により、
// /sitemap.xml へのリクエストのたびにデータベースから
// 最新のデータを取得して動的に生成します。

// 自動的にサイトマップに含まれるページ:
// ・静的ページ (/ , /privacy, /terms, /help, /shops, /events)
// ・公開ユーザーのプロフィールページ (最新1,000件)
// ・投稿ページ (リポスト除く、最新5,000件)
// ・盆栽園ページ (最新1,000件)
// ・開催予定のイベントページ (最新500件)

// 新しい投稿が作成されると、次回の /sitemap.xml へのアクセス時に
// 自動的に含まれます。手動更新は不要です。

// Google Search Console にサイトマップURLを登録しておく、
// Googlebotが定期的にサイトマップを取得して
// 新しいページを発見・インデックスします。
```

## Q21: JSON-LDを設定すると検索結果はどう変わりますか？

A: Googleの検索結果に追加情報が表示される可能性があります（リッチリザルト）。

### JSON-LDなしの検索結果:

BON-LOG - 盆栽愛好家のためのSNS <https://bon-log.com> 盆栽の育て方や作品を共有できるSNSサービスです...

### JSON-LDありの検索結果（リッチリザルト）:



JSON-LD種別	効果	表示例
Organization	検索結果にロゴが表示される可能性	-
LocalBusiness	盆栽園の営業時間、評価、地図が表示	4.2 (25件のレビュー) / 営業中 - 9:00～17:00 / 東京都文京区...
Event	イベントの日程、場所がカード形式で表示	盆栽展示会2026 / 4月1日 - 4月3日 / 東京ドーム
Article	投稿の著者名、公開日が表示	-
Breadcrumb	検索結果のURLがパンくず形式で表示	bon-log.com > 盆栽園 > ○○盆栽園

**注意:** JSON-LDを設定しても、リッチリザルトとして表示されるかはGoogleの判断に委ねられます。設定は「表示される可能性を高める」ものです。

## セキュリティに関するFAQ

**Q22: NEXTAUTH\_SECRETをどうやって生成すればよいですか？**

A: 以下のコマンドで安全なシークレットを生成できます。

```
macOS / Linux
openssl rand -base64 32
出力例: K7gNU3sdo+OL0wNhqoVWhr3g6s1xYv72oL/pe/UnoIs=

Node.js
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"

Windows PowerShell
[Convert]::ToBase64String((1..32 | ForEach-Object { Get-Random -Maximum 256 }) -as [byte[]])
```

生成したシークレットの設定場所:

[ローカル開発]

`.env.local` ファイルに設定:

`NEXTAUTH_SECRET="K7gNU3sdo+OL0wNhqoVWhr3g6s1xYv72oL/pe/Unols="`

[Vercel本番]

Vercelダッシュボード > Settings > Environment Variables

Key: NEXTAUTH\_SECRET

Value: K7gNU3sdo+OL0wNhqoVWhr3g6s1xYv72oL/pe/Unols=

Environment: Production, Preview

[GitHub Actions CI]

GitHub > Settings > Secrets > Actions > New repository secret

Name: NEXTAUTH\_SECRET

Value: K7gNU3sdo+OL0wNhqoVWhr3g6s1xYv72oL/pe/Unols=

注意:

- ・開発環境と本番環境で異なるシークレットを使用する
- ・シークレットを `.env` ファイルに書いてGitにコミットしない
- ・`.gitignore` に `.env.local` が含まれていることを確認

Q23: 環境変数の `NEXT_PUBLIC_` プレフィックスの重要性は？

A: `NEXT_PUBLIC_` が付いた環境変数はブラウザに公開されます。機密情報には絶対に付けないでください。

NEXT\_PUBLIC\_ の有無による違い:

NEXT\_PUBLIC\_APP\_URL="https://bon-log.com"

- ↑ ブラウザのJavaScriptに含まれる
- ↑ ユーザーのブラウザでアクセス可能
- ↑ DevToolsのSourcesタブで見える

DATABASE\_URL="postgresql://postgres:password@ ... "

- ↑ サーバーサイドでのみアクセス可能
- ↑ ブラウザには絶対に送信されない
- ↑ APIルート、Server Actions、Server Componentsでのみ使用

正しい使い分け:

分類	環境変数	説明
公開	NEXT_PUBLIC_	クライアント公開
非公開	NEXT_PUBLIC_APP_URL	公開されても問題ない値のみ
非公開	NEXT_PUBLIC_SENTRY_DSN	
非公開	NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY	
非公開	NEXT_PUBLIC_ なし	サーバーのみ
非公開	DATABASE_URL	漏洩すると大問題になる機密情報
非公開	NEXTAUTH_SECRET	
非公開	STRIPE_SECRET_KEY	
非公開	R2_SECRET_ACCESS_KEY	
非公開	CRON_SECRET	

## パフォーマンスに関するFAQ

**Q24: Web Vitalsのスコアが悪い場合、具体的にどう改善すればよいですか？**

A: 指標ごとに改善方法が異なります。

**LCP（最大コンテンツの描画時間）が遅い場合 -- 目標: 2.5秒以下**

原因	対策
大きな画像の読み込みが遅い	next/image を使用して自動最適化 / priority 属性をLCP要素に設定 / 適切な sizes 属性を指定
サーバーレスポンスが遅い	Server Componentでデータを並列取得（Promise.all） / データベースクエリの最適化（インデックス追加） / Redis等でキャッシュ
JavaScriptの実行が遅い	Client Componentを最小限に / dynamic import で遅延読み込み / 重いライブラリはServer Componentでのみ使用

**INP（操作から描画までの時間）が遅い場合 -- 目標: 200ms以下**

原因	対策
イベントハンドラ内の重い処理	useTransition で非ブロッキングに / 楽観的更新で即座にUI反映
大量のDOM更新	React.memo で不要な再レンダリングを防止 / 仮想リスト (react-virtual) の使用
サードパーティスクリプトの影響	分析スクリプトは遅延読み込み / Web Worker で別スレッド実行

**CLS（レイアウトのずれ）が大きい場合 -- 目標: 0.1以下**

原因	対策
画像にサイズが指定されていない	width と height を必ず指定 / next/image は自動的にサイズを確保
遅延読み込みされるコンテンツ	Suspense + Skeleton コンポーネントで領域を確保 / loading.tsx でローディングUI表示
Web フォントの読み込み	next/font で最適化 / display: 'swap' でFOUT対策
動的に挿入される広告やバナー	表示領域をあらかじめ確保 (min-height設定)

**Q25: Lighthouseスコアとは何ですか？どう計測しますか？**

A: Googleが提供するウェブページ品質評価ツールです。

カテゴリ	内容	目標スコア
Performance	表示速度	90以上
Accessibility	アクセシビリティ	90以上
Best Practices	ベストプラクティス	90以上
SEO	検索エンジン最適化	90以上
PWA	PWA対応度	- (チェックリスト形式)

計測方法:

**方法1: Chrome DevTools**

1. Chrome で対象ページを開く
2. F12 で DevTools を開く
3. 「Lighthouse」タブを選択
4. 「Analyze page load」をクリック
5. 結果を確認（30秒～1分で完了）

#### 方法2: PageSpeed Insights（オンライン）

1. <https://pagespeed.web.dev/> にアクセス
2. URLを入力して「分析」
3. モバイルとデスクトップの両方の結果を確認

#### 方法3: Lighthouse CI（CI/CDに組み込み）

```
GitHub Actionsで自動計測
- uses: treosh/lighthouse-ci-action@v11
 with:
 urls: |
 https://bon-log.com/
 https://bon-log.com/shops
 budgetPath: ./lighthouse-budget.json
```

注意:

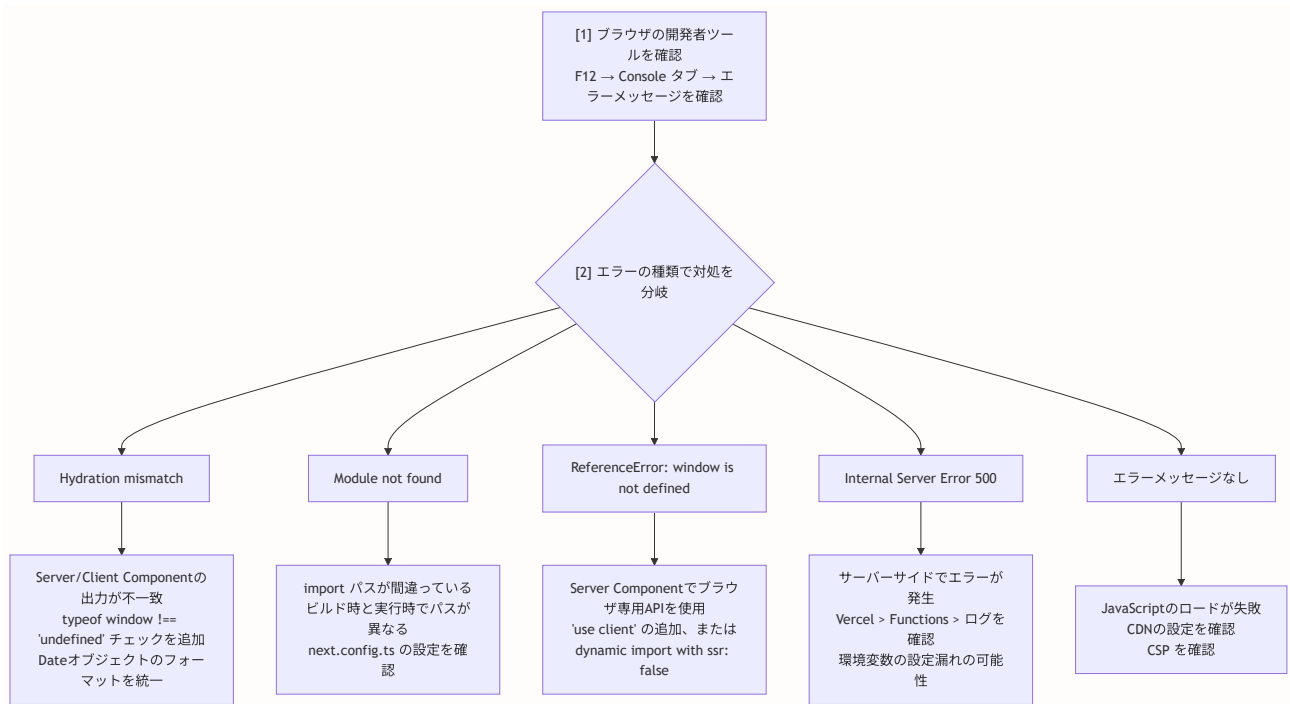
- 計測結果は毎回多少変動する
- シークレットウィンドウで計測する（拡張機能の影響を除外）
- モバイルの計測はCPUを4倍遅くするため、スコアが低めに出る

## トラブルシューティングFAQ

**Q26: デプロイ後にページが真っ白になる（White Screen of Death）場合は？**

A: いくつかの原因が考えられます。

**白い画面のデバッグ手順:**



### Q27: Vercelのビルドログで「Type error」が出る場合は？

A: ローカルでは出ないのにVercelで出る場合は、型チェックの厳密さが異なる可能性があります。

Vercelでの型エラーのデバッグ:

[まずローカルで再現]

```
npm run build
```

→ ローカルでも再現すれば、そのまま修正

[ローカルで再現しない場合]

1. TypeScriptバージョンの違い

→ `package.json` の `typescript` バージョンを確認

→ `npm ci` でクリーンインストール

2. Node.jsバージョンの違い

→ Vercelダッシュボード > Settings > Node.js Version

→ ローカルと合わせる (推奨: 20.x)

3. 環境変数の有無

→ ビルド時に参照している環境変数がVercelに設定されているか確認

→ 特に `process.env.NEXT_PUBLIC_*` の値

4. キャッシュの問題

→ Vercel > Settings > General > Build Cache を OFF にして再ビルド

[よくある型エラーパターン]

・ `params` が `Promise` 型になった (Next.js 15以降)

× `const { id } = params`

○ `const { id } = await params`

・ `searchParams` が `Promise` 型になった (Next.js 15以降)

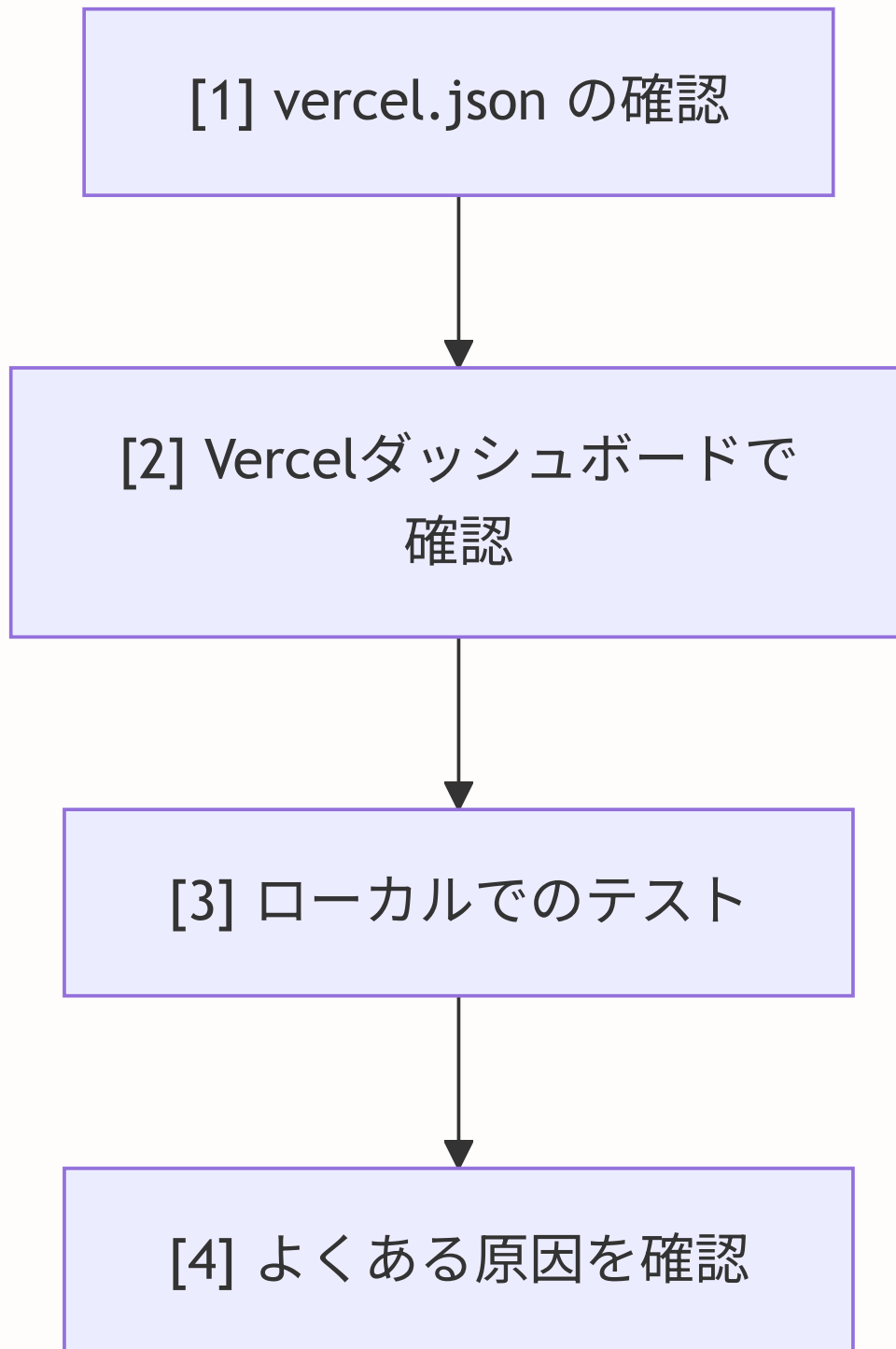
× `const q = searchParams.q`

○ `const { q } = await searchParams`

## Q28: Cronジョブが実行されない場合のデバッグ方法は？

A: Vercelのダッシュボードでログと設定を確認します。

### Cronジョブのデバッグ手順:



[1] vercel.json の確認



## チェック項目

パスが正しいか（先頭の `/` が必要）

cron式が正しいか

JSONのフォーマットが正しいか

## [2] Vercelダッシュボードで確認（Settings &gt; Cron Jobs）

## チェック項目

ジョブが表示されているか

最終実行時刻

ステータス（Success / Failed）

## [3] ローカルでのテスト

```
curlで直接APIを呼び出し
curl -H "Authorization: Bearer your-cron-secret" \
 http://localhost:3000/api/cron/publish-scheduled
```

## [4] よくある原因

原因	対処法
CRON_SECRET が未設定	Vercelの環境変数に設定する
認証ヘッダーの形式が間違い	<code>Authorization: Bearer &lt;secret&gt;</code> 形式を確認
Serverless Functionのタイムアウト（10秒制限）	Proプランなら <code>maxDuration: 60</code> で60秒まで延長
HobbyプランではCron Jobsが2つまで	Proプランなら無制限
データベース接続エラー	<code>DATABASE_URL</code> が正しく設定されているか確認

## Q29: 「Too many connections」エラーが出る場合は？

A: データベースの接続数が上限に達しています。

接続数エラーの原因と対策:

原因:

Serverless環境では、関数の呼び出しごとに新しいDB接続が作られる可能性がある。短時間に大量のリクエストが来ると、接続数がPostgreSQLの上限（通常100程度）を超える。

対策1: 接続プールを使用

→ SupabaseのPgBouncer経由で接続

→ DATABASE\_URL にポート 6543 を指定

対策2: Prismaの接続プール設定

```
// schema.prisma
datasource db {
 provider = "postgresql"
 url = env("DATABASE_URL")
 // 接続プールのサイズを制限
 // Serverless環境では小さめに設定
}
```

対策3: 接続数の監視

// Supabase Dashboard > Database > Connections

// で現在の接続数を確認

対策4: 不要な接続を閉じる

// API Route / Server Action の終了時に接続を解放

// Prismaの場合、\$disconnect() は通常不要

// （グローバルインスタンスが管理するため）

**Q30: Vercelのログはどこで確認できますか？**

A: Vercelダッシュボードの複数の場所で確認できます。

Vercelのログ確認場所:

[ビルドログ]

Vercel > プロジェクト > Deployments > 対象デプロイ > Build Logs  
→ npm install、ビルド、ソースマップアップロードのログ

[ランタイムログ (Serverless Function) ]

Vercel > プロジェクト > Logs  
→ API Routes、Server Actions のconsole.log出力  
→ リアルタイムで確認可能

[Cronジョブログ]

Vercel > プロジェクト > Settings > Cron Jobs  
→ 各Cronジョブの実行結果と出力

ログの保存期間（プランによる）:

Hobby（無料）: 1時間  
Pro: 3日間  
Enterprise: 14日間

[ログ確認のコツ]

- ・ランタイムログはリアルタイムなので、問題発生時にすぐ確認
- ・ console.error() の出力を重点的にチェック
- ・ 本番環境では構造化ログ（JSON形式）が検索しやすい

**Q31: CI/CDパイプラインにおけるシークレットの管理はどうすればよいですか？**

A: GitHubのRepository Secretsを使用します。

GitHubシークレットの設定手順:

1. GitHub > リポジトリ > Settings > Secrets and variables > Actions
2. "New repository secret" をクリック
3. Name (キー名) と Value (値) を入力して保存

使い方 (ci.yml内) :

```
env:
 DATABASE_URL: ${ secrets.DATABASE_URL }
 NEXTAUTH_SECRET: ${ secrets.NEXTAUTH_SECRET }
```

シークレットの特徴:

- ・一度設定すると値の確認 (閲覧) ができない
- ・ログに出力される場合は自動的に「\*\*\*」にマスクされる
- ・フォークしたリポジトリからはアクセスできない
- ・組織レベルで共有するシークレットも設定可能

注意:

- ・シークレットの値を変更したい場合は上書き設定する
- ・PRのワークフローではシークレットにアクセスできない場合がある (セキュリティ上の制限)
- ・GitHub Environmentsを使うと環境 (production、staging) ごとに異なるシークレットを設定できる

### Q32: 本番環境でデバッグするときのベストプラクティスは?

A: 本番環境に影響を与えずにデバッグするために、以下の手法を使います。

本番デバッグのベストプラクティス:

[絶対にやってはいけないこと]

- ・ `console.log` をたくさん追加して本番デプロイ
- ・ 本番DBに直接SQLを実行
- ・ デバッグ用のコードを本番ブランチにマージ

[推奨する方法]

1. Sentryのエラー詳細を活用
  - スタックトレース、ユーザー情報、リクエスト情報
  - ソースマップで元のTypeScriptコードを表示
2. Vercel Runtime Logsを監視
  - リアルタイムでサーバーログを確認
  - フィルタ機能でエラーのみ表示
3. ローカルで本番環境を再現
  - `.env.local` に本番のDB URLを設定（読み取りのみ）
  - 問題の再現を試みる
  - 注意: 本番DBへの書き込みは絶対に行わない
4. ステージング環境で再現
  - 本番と同じ構成のステージング環境を使う
  - ここでは自由にデバッグコードを追加できる
5. Feature Flagsの活用
  - 問題の原因を特定するために機能のON/OFFを切り替え
  - 特定のユーザーにのみデバッグ情報を表示

▶ FAQの理解度チェック（クリックして展開）

## 22.16 学習ロードマップ

この章で学んだCI/CDとデプロイの知識を土台として、さらにスキルアップするための詳細なロードマップを提供します。段階ごとに具体的な学習目標、参考リソース、実践課題を紹介します。

**STEP 1: 基礎固め（1～2週間）**

この章の内容を実際に手を動かして再現

**STEP 2: 実践（2～4週間）**

自分のプロジェクトに  
CI/CDを導入

**STEP 3: 深化（1～3ヶ月）**

モニタリング・パフォーマンス改善・セキュリティ強化

**STEP 4: 応用（3～6ヶ月）**

スケーラビリティ・マイクロサービス・IaC

**STEP 5: エキスパート（6ヶ月～）**

SRE・プラットフォームエンジニアリング

## STEP 1: 基礎固め（1～2週間）

この章の内容を実際に手を動かして体験します。知識を定着させるために「読む」だけでなく「やる」ことが最も重要です。

## STEP 1 のチェックリスト:

- ☐ Vercelの無料アカウントを作成
- ☐ 自分のNext.jsプロジェクトをVercelにデプロイ
- ☐ 環境変数をVercelに設定（最低限DATABASE\_URLとNEXTAUTH\_SECRET）
- ☐ GitHubリポジトリにCI/CDワークフローを追加
- ☐ ワークフローが自動実行されることを確認
- ☐ PRを作成してプレビューデプロイを体験
- ☐ Sentryの無料アカウントを作成
- ☐ 意図的にエラーを発生させてSentryで確認
- ☐ Docker Composeでローカル開発環境を構築

## 具体的な実践課題:

## 課題1: 「Hello World」デプロイ

1. `npx create-next-app@latest hello-deploy` で新しいプロジェクト作成
2. GitHubにリポジトリを作成してpush
3. Vercelにインポートしてデプロイ
4. カスタムドメインなしでデプロイされたURLを確認
5. READMEを編集 → push → 自動で再デプロイされることを確認

所要時間: 30分~1時間

学べること: デプロイの基本的な流れ

## 課題2: CI/CDパイプラインの構築

1. 上記プロジェクトに `.github/workflows/ci.yml` を追加
2. `lint` と `build` のジョブを定義
3. PRを作成してCIが自動実行されることを確認
4. 意図的にlintエラーを入れてCIが失敗することを確認
5. エラーを修正して再pushし、CIが成功することを確認

所要時間: 1~2時間

学べること: GitHub Actionsの基本

## STEP 2: 実践（2~4週間）

自分のプロジェクト（BON-LOGまたはオリジナル）にCI/CDを本格的に導入します。

**STEP 2 のチェックリスト:**

- [ ] Supabaseのプロジェクトを作成してDBを接続
- [ ] Prismaマイグレーションを本番に適用
- [ ] テストを追加してCIに組み込み
- [ ] Sentryのエラー通知をSlackまたはメールで受信
- [ ] PWAの基本設定 (manifest.json、Service Worker)
- [ ] robots.txtとsitemap.xmlの設定
- [ ] 本番環境のヘルスチェックAPIの実装
- [ ] ロールバック手順のドキュメント化

**学習の進め方のアドバイス:****効果的な学習方法:**

- [1] まずは動くものを作る  
完璧を目指すず、まず動作するCI/CDパイプラインを構築。  
設定の意味は後から理解すればOK。
- [2] エラーを恐れない  
CI/CDの失敗は学びのチャンス。  
エラーメッセージを読む力が最も重要なスキル。
- [3] 1つずつ追加する  
一度にすべてを設定しようとしなない。  
lint → test → build → deploy の順に段階的に追加。
- [4] 公式ドキュメントを読む習慣  
Vercel、GitHub Actions、Sentryの公式ドキュメントは  
最も信頼できる情報源。困ったらまず公式を確認。
- [5] 他のプロジェクトの設定を参考にする  
GitHubで「ci.yml」を検索して、  
実際のプロジェクトがどう設定しているか学ぶ。

**STEP 3: 深化 (1〜3ヶ月)**

モニタリング、パフォーマンス改善、セキュリティを深掘りします。

**STEP 3 の学習テーマ:****モニタリング強化**



テーマ	詳細
Sentryのカスタムメトリクス	ビジネスKPI（投稿数、アクティブユーザー数等）の追跡
アラートルールのカスタマイズ	エラー率が閾値を超えたらSlackに通知
ダッシュボードの構築	Grafana や Vercel Analyticsで可視化
ログの構造化	JSON形式のログ出力で検索・分析を容易に

## パフォーマンス最適化

テーマ	詳細
Core Web Vitals の改善	実際のユーザーデータ（RUM）で計測
データベースクエリの最適化	EXPLAIN ANALYZE でクエリプランを確認、インデックスの追加
CDNの活用	静的アセットのキャッシュ戦略、Edge Functionsの活用
バンドルサイズの最適化	@next/bundle-analyzer で分析、不要な依存関係の削除

## セキュリティ強化

テーマ	詳細
CSP（Content Security Policy）の設定	コンテンツの読み込み元を制限
Rate Limiting の実装	Upstash Redis でIP単位のレート制限
WAF（Web Application Firewall）の導入	VercelのFirewall機能
脆弱性スキャン	npm audit、Snyk、Dependabot
ペネトレーションテスト	OWASP Top 10 の確認

## 推奨リソース:

## 書籍・ドキュメント:

## [Vercel]

- ・ Vercel公式ドキュメント  
<https://vercel.com/docs>

## [GitHub Actions]

- ・ GitHub Actions公式ドキュメント  
<https://docs.github.com/ja/actions>

## [Next.js]

- ・ Next.js公式ドキュメント (Deploying セクション)  
<https://nextjs.org/docs/app/building-your-application/deploying>

## [Sentry]

- ・ Sentry for Next.js ドキュメント  
<https://docs.sentry.io/platforms/javascript/guides/nextjs/>

## [Docker]

- ・ Docker公式チュートリアル  
<https://docs.docker.com/get-started/>

## [Web Vitals]

- ・ web.dev (Google)  
<https://web.dev/articles/vitals>

## [セキュリティ]

- ・ OWASP Top 10  
<https://owasp.org/www-project-top-ten/>

## STEP 4: 応用 (3～6ヶ月)

大規模な運用に向けた技術を学びます。

### STEP 4 の学習テーマ:

スケーラビリティ

カテゴリ	テーマ
データベースのスケール ング	Read Replica（読み取り専用レプリカ）、Connection Pooling の最適化、データベ ースシャーディング
アプリケーションのスケー リング	マイクロフロントエンド、分散キャッシュ（Redis Cluster）、CDN Edge Computing
非同期処理	メッセージキュー（BullMQ、AWS SQS）、バックグラウンドジョブ、イベント駆動 アーキテクチャ

## Infrastructure as Code（IaC）

ツール	詳細
Terraform	クラウドインフラをコードで定義・管理
Pulumi	TypeScriptでインフラを記述
AWS CDK	AWSリソースをコードで管理

## コンテナオーケストレーション

ツール	詳細
Kubernetes（K8s）	Pods、Services、Deployments、Helm Charts、オートスケーリング
AWS ECS / Fargate	AWSのコンテナサービス
Google Cloud Run	GCPのサーバーレスコンテナ
Azure Container Apps	Azureのコンテナサービス

**STEP 4 の実践課題:**

課題: マルチリージョン対応のアーキテクチャ設計

要件:

- ・日本とアメリカの2リージョンで動作
- ・ユーザーの最寄りリージョンにルーティング
- ・データベースはプライマリ（日本）+ レプリカ（アメリカ）
- ・画像はCDNで世界中から高速アクセス

設計のヒント:

```
<div class="mermaid-rendered">
<img src="data:image/svg+xml;base64,PHN2ZyBpZD0ibW1kXzIyX2RlcGxveV8zMjIkdGg9IjEwMCUiIHhtbG
</div>
```

## STEP 5: エキスパート（6ヶ月以降）

SRE（Site Reliability Engineering）やプラットフォームエンジニアリングの領域に進みます。

**STEP 5 の学習テーマ:****SRE（サイト信頼性エンジニアリング）**

テーマ	詳細
SLI / SLO / SLA の定義と運用	SLI: Service Level Indicator（計測指標）、SLO: Service Level Objective（目標値）、SLA: Service Level Agreement（契約）
BON-LOGのSLO例	可用性: 99.9%（月間ダウンタイム43分以内）、レスポンスタイム: p99 < 500ms、エラー率: < 0.1%
インシデント管理	インシデント対応フロー、ポストモータム（事後分析）、エスカレーションポリシー
Chaos Engineering	意図的に障害を発生させて耐障害性を検証（例: DBを一時切断してアプリの振る舞いをテスト）
容量計画（Capacity Planning）	トラフィック増加に対する事前準備、負荷テスト（k6、Locust）

**プラットフォームエンジニアリング**

テーマ	詳細
開発者体験（DX）の向上	Internal Developer Platform（IDP）構築、セルフサービスのデプロイ環境、開発者向けドキュメントの整備
GitOps	ArgoCD / Flux、Gitリポジトリがインフラの唯一の真実、プルリクエストベースのインフラ変更
自動化の高度化	カスタムGitHub Actions、Terraformモジュールの作成、Policy as Code（OPA / Kyverno）

## キャリアパスのイメージ:

デプロイを学んだ先のキャリアパス:

### 【フルスタックエンジニア】

この章の知識 + フロントエンド + バックエンド  
→ 1人でサービスの開発からデプロイまで完結

### 【バックエンドエンジニア】

サーバー、DB、API設計 + CI/CD + インフラ  
→ サーバーサイドのアーキテクチャを設計・構築

### 【SRE（サイト信頼性エンジニア）】

CI/CD + モニタリング + インフラ + 自動化  
→ システムの信頼性と可用性を保証

### 【DevOps エンジニア】

CI/CD + IaC + コンテナ + クラウド  
→ 開発と運用の橋渡し、自動化推進

### 【プラットフォームエンジニア】

社内ツール + IDP + 自動化 + DX向上  
→ 開発チーム全体の生産性を向上

どのキャリアパスを選んでも、この章で学んだ  
CI/CD、デプロイ、モニタリングの知識は基盤になります。

## 学習スケジュールの目安

週ごとの学習スケジュール（STEP 1～2）:

週	Day 1-2	Day 3-4	Day 5-7
Week 1	Vercelアカウント作成、初回デプロイ	GitHub Actions CI/CD設定	Sentryの設定、エラー監視体験
Week 2	Docker環境構築	Supabase接続、マイグレーション	PWA基本設定
Week 3	SEO設定 (sitemap、robots、JSON-LD)	Web Vitals計測と改善	セキュリティチェック実装
Week 4	Cronジョブの設定	本番運用チェックリストの確認	演習問題に取り組む

### 1日の学習時間の目安: 1～2時間

曜日	内容	時間
平日	ドキュメントを読む、設定を変更する	30分～1時間
休日	手を動かして実践する	1～3時間

#### ▶ 学習ロードマップの理解度チェック（クリックして展開）

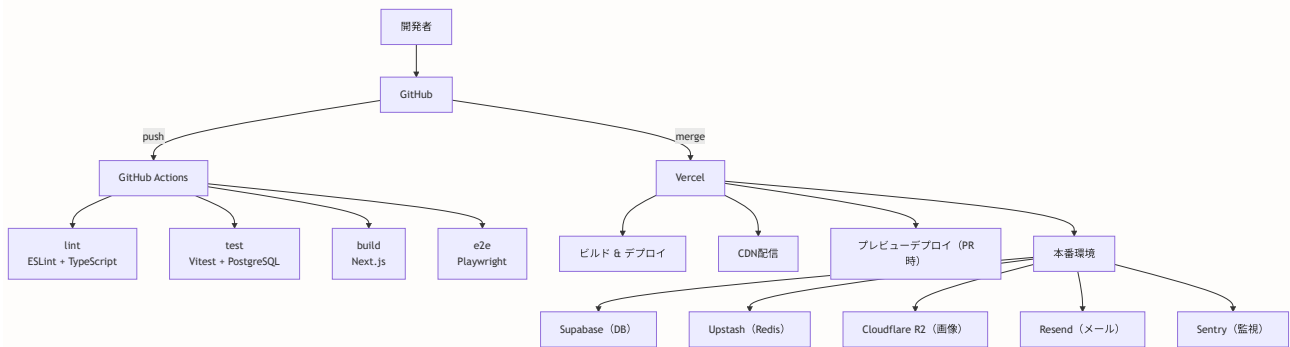
## 22.17 まとめ

この章では、CI/CDとデプロイについて包括的に学びました。

## 学んだ内容の振り返り

セクション	学んだ内容
22.1 デプロイの基本概念	デプロイとは何か、CI/CDの意味と重要性
22.2 Supabase（本番DB）	クラウドデータベースの作成と接続設定
22.3 Vercelデプロイ	Next.jsアプリの本番公開、プレビューデプロイ、ドメイン設定
22.4 GitHub Actions CI/CD	自動テスト・ビルド・デプロイパイプラインの構築
22.5 Docker本番ビルド	コンテナ化、マルチステージビルド、docker-compose
22.6 Sentry監視	エラー検知、パフォーマンス監視、アラート設定
22.7 Sentry詳細設定	クライアント/サーバー/Edge設定、エラーフィルタリング、パフォーマンスモニタリング
22.8 PWA対応	Service Worker、オフライン対応、キャッシュ戦略、更新プロンプト
22.9 SEO構造化データ	sitemap.ts、robots.ts、JSON-LDコンポーネント
22.10 Web Vitals	LCP/INP/CLS計測、WebVitalsReporterコンポーネント
22.11 本番セキュリティチェック	環境変数検証、シークレット強度チェック
22.12 Cronジョブ	予約投稿公開、サブスクリプション管理、イベントクリーンアップ
22.13 本番運用	チェックリスト、定期メンテナンス、パフォーマンス最適化
22.14 演習問題	基礎～チャレンジレベルの実践課題
22.15 FAQ	よくある質問と回答（デプロイ・CI/CD・運用全般）
22.16 学習ロードマップ	STEP 1～5の段階的な学習計画

## 技術の関連マップ



## 重要ポイントのおさらい

セクション	最も重要なポイント
Supabase	<code>DATABASE_URL</code> （Transaction mode）と <code>DIRECT_URL</code> （Session mode）の使い分け
Vercel	環境変数の正しい設定。 <code>NEXT_PUBLIC_</code> プレフィックスの意味
GitHub Actions	ワークフローファイルの構造（イベント→ジョブ→ステップ）
Docker	マルチステージビルドによるイメージサイズ最適化
Sentry	本番環境のみ有効化。サンプリング率の設定
Sentry詳細	クライアント/サーバー/Edge の3つの設定ファイル
PWA	Service Worker のキャッシュ戦略とオフライン対応
SEO	sitemap・robots・JSON-LD構造化データ
Web Vitals	LCP/INP/CLS のパフォーマンス計測
セキュリティ	NEXTAUTH_SECRET の強度検証、必須環境変数チェック
Cronジョブ	予約投稿公開、サブスク管理、イベントクリーンアップ
本番運用	デプロイ前後のチェックリスト。定期メンテナンスの習慣化
FAQ	環境変数のNEXT_PUBLIC_、Vercelプレビュー、DB接続プール
学習ロードマップ	基礎固め→実践→深化→応用→エキスパートの5段階

▶ 最終理解度チェック（クリックして展開）



## 22.18 次のステップ

この章でCI/CDとデプロイの基礎を学びました。ここからさらにレベルアップするためのステップを紹介します。

### すぐに取り組めること

- **Vercelの無料プランでデプロイしてみる**: 実際にデプロイすることが最大の学びになります
- **GitHub Actionsのワークフローを自分で書いてみる**: 本章のYAMLを参考に、自分のプロジェクトに適用
- **Sentryの無料プランでエラー監視を試す**: 開発環境でもエラー送信をテスト

### 中期的に学ぶべきこと

- **ユーザーフィードバックの収集と分析**: Sentryのフィードバックウィジェット、Google Analytics
- **A/Bテストの実施**: 機能のON/OFFをユーザーグループごとに切り替える
- **パフォーマンスの継続的な改善**: Core Web Vitals、Lighthouse CI

### 長期的な発展

- **スケーラビリティの向上**: データベースのリードレプリカ、CDNの最適化
- **マイクロサービスへの移行**: 機能ごとにサービスを分割
- **Kubernetesの学習**: より高度なコンテナオーケストレーション
- **laC (Infrastructure as Code)**: Terraform等でインフラをコードで管理

学習のロードマップ:

【この章】	【次の目標】	【さらに先】
CI/CD基礎	→ パフォーマンス	→ スケーラビリティ
Vercel	→ マルチリージョン	→ Kubernetes
Docker	→ Docker Compose	→ Container Orchestration
Sentry	→ カスタムメトリクス	→ APM (Application Performance Monitoring)
GitHub Actions	→ 高度なワークフロー	→ GitOps

これで、BON-LOGの開発から本番運用までの完全なフローを習得しました。おめでとうございます！

継続的に改善を重ね、ユーザーにとって価値あるサービスを提供し続けましょう。本番環境でのトラブルは必ず起きますが、この章で学んだCI/CD、監視、ロールバックの仕組みがあれば、迅速に対処できます。